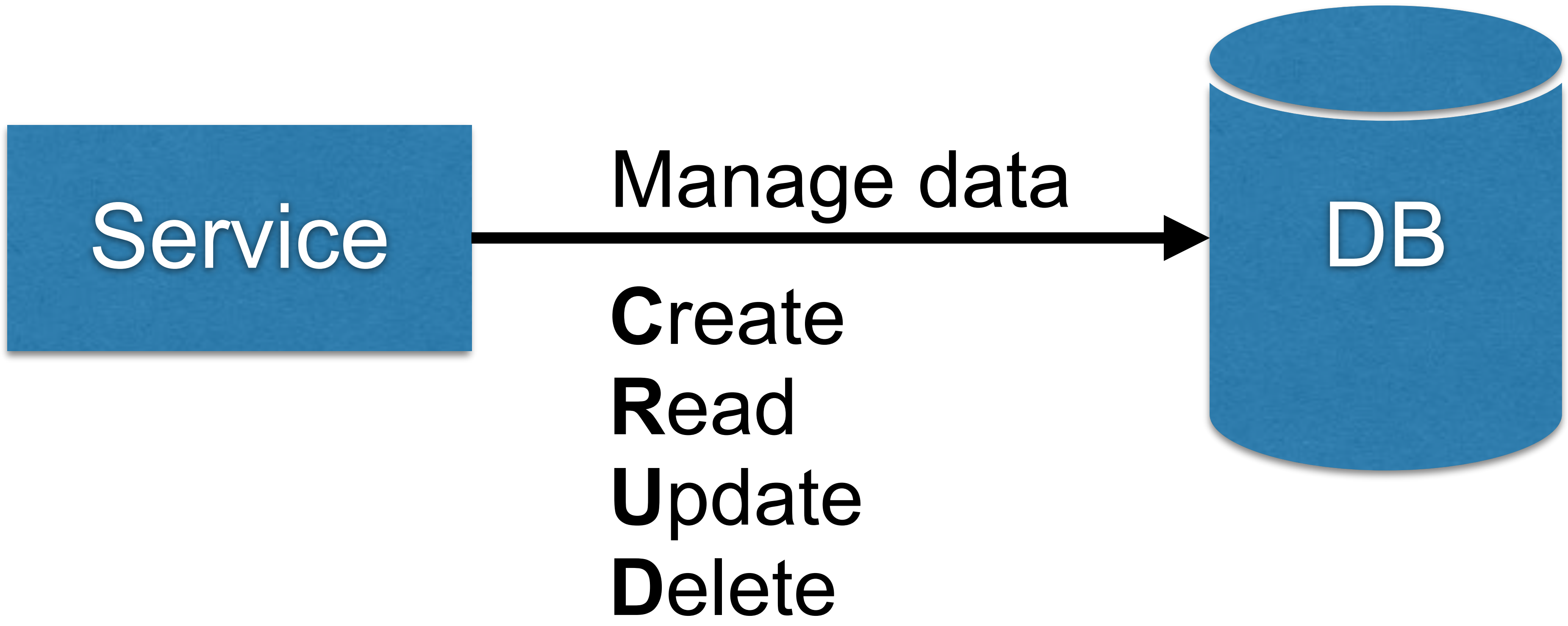
Database Design Tuning

https://github.com/up1/workshop-database

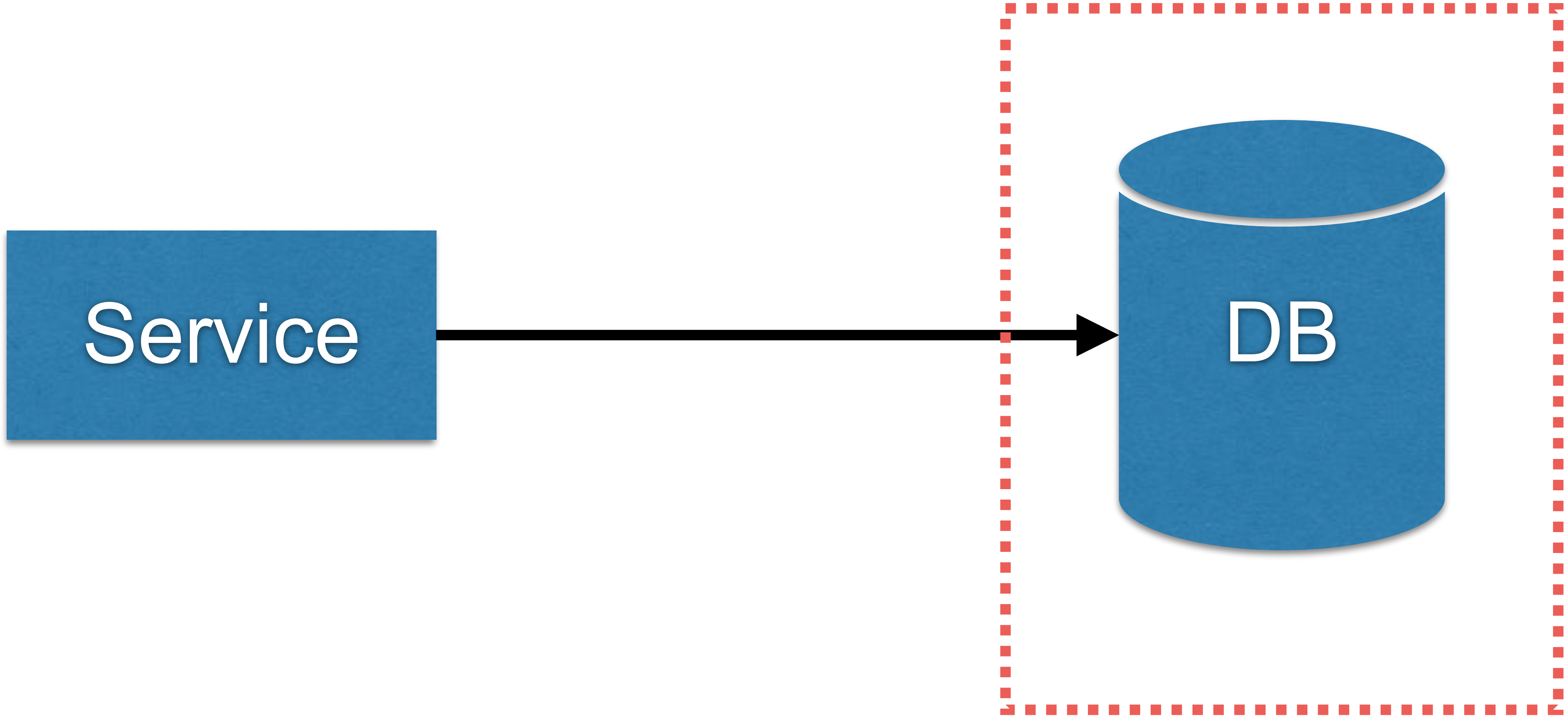
Database

Database



Working with Database

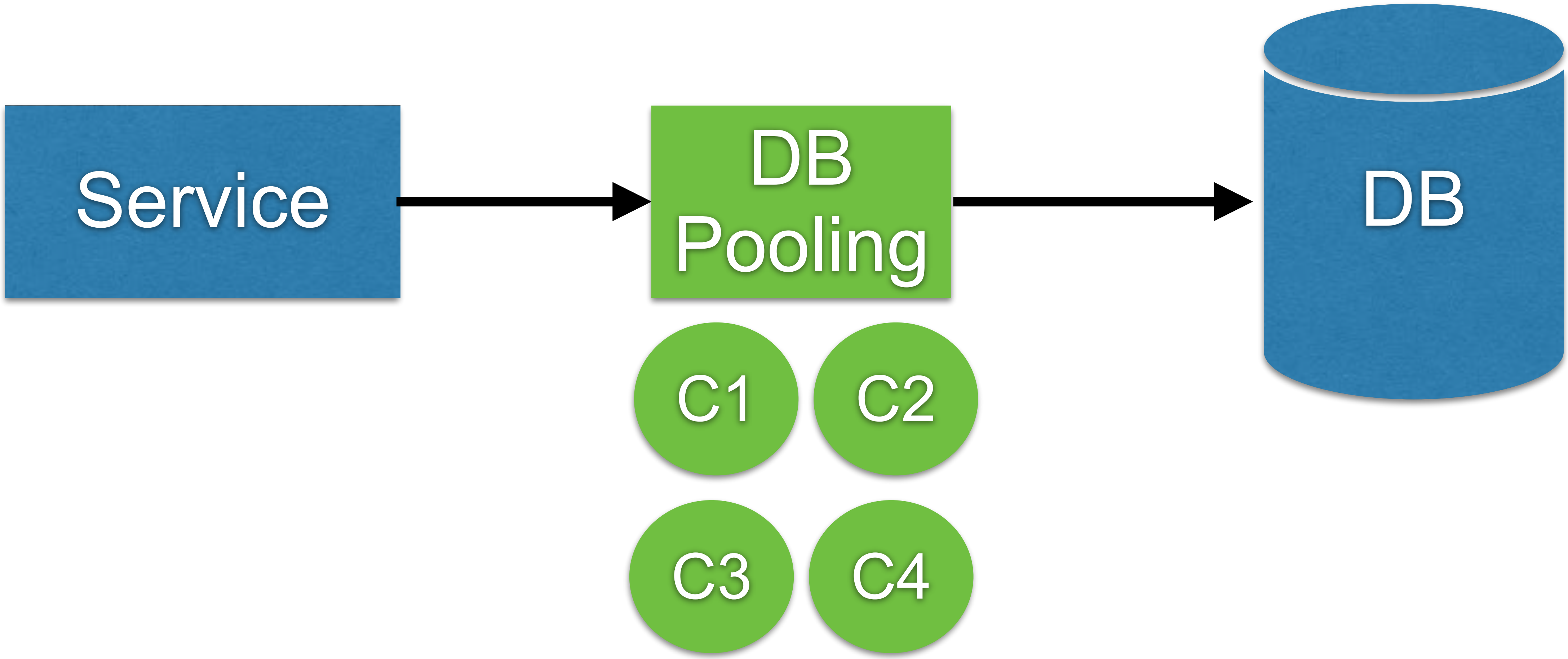
Directly to database



Data modeling
Database architecture

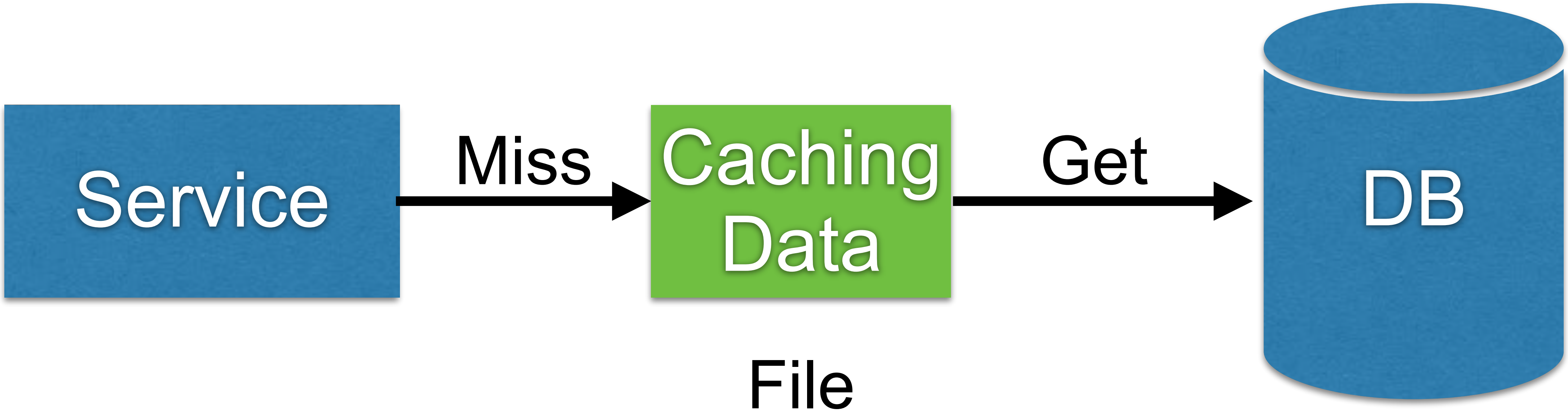
Working with Database

Connection pool



Working with Database

Caching data layer



File
Memory
Other database model

Scalability metrics

Throughput (transactions per second)
Latency (response time)
Availability (uptime)
Durability (data safety)

Database ?

Structured data collection

Records

Relationships

Database Management System ?

Software systems for storing and querying databases

User's perspective

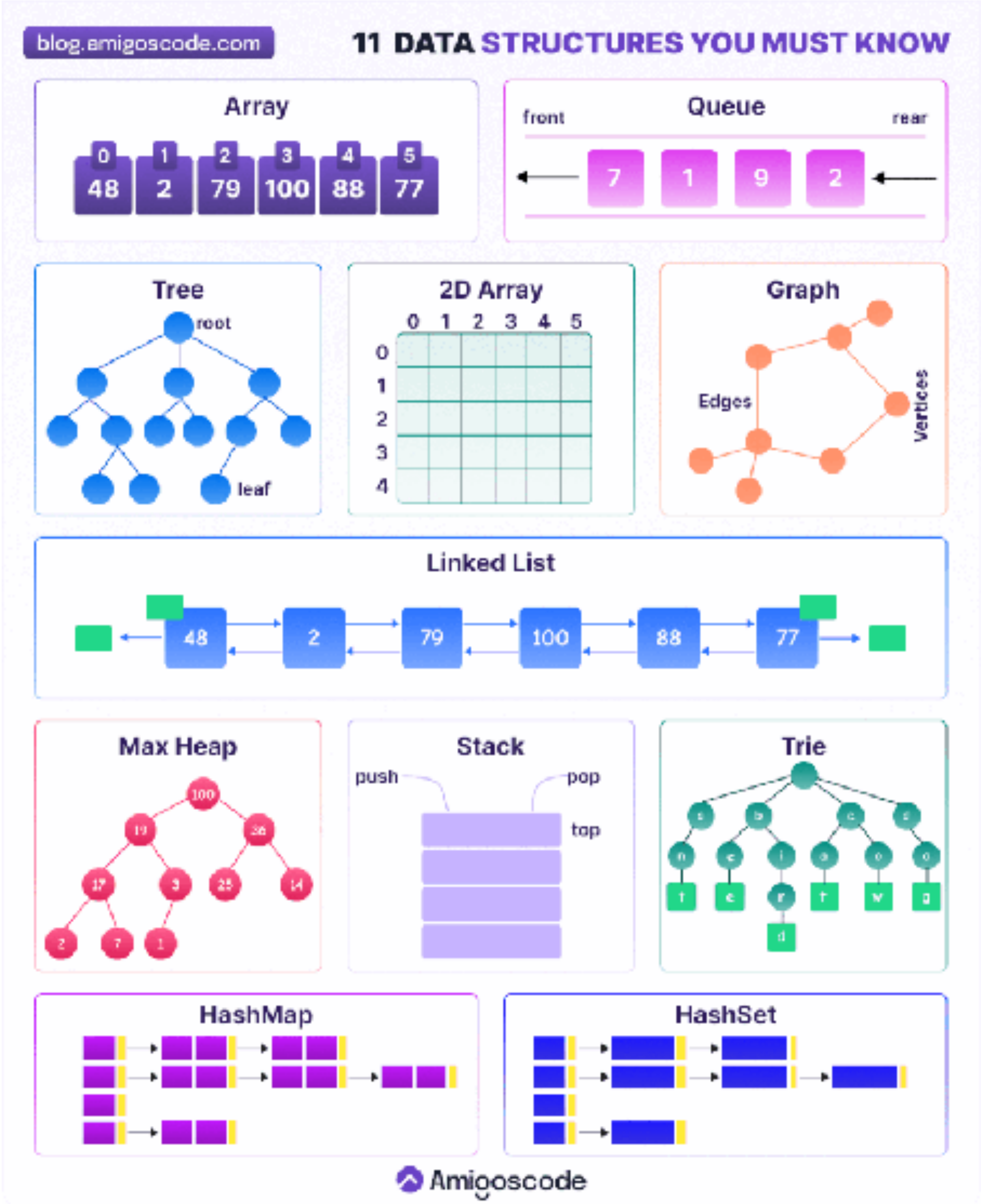
Data modeling
Querying data
Store data

Read

Write

Database Models ?

Data Structure



https://blog.amigoscode.com/p/11-data-structures-every-developer

Database Models ?

424 systems in ranking, July 2025

Rank Jul 2025	Rank Jun 2025	Rank Jul 2024	DBMS	Database Model	Score Jul 2025	Score Jun 2025	Score Jul 2024
1.	1.	1.	Oracle	Relational, Multi-model	1217.05	-13.33	-23.31
2.	2.	2.	MySQL	Relational, Multi-model	940.73	-12.85	-98.73
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	771.14	-5.61	-36.51
4.	4.	4.	PostgreSQL	Relational, Multi-model	680.89	+0.23	+41.98
5.	5.	5.	MongoDB	Document, Multi-model	403.83	+0.99	-25.99
6.	6.	↑7.	Snowflake	Relational	176.17	+1.68	+39.64
7.	7.	↓6.	Redis	Key-value, Multi-model	149.72	-2.01	-7.05
8.	8.	↑9.	IBM Db2	Relational, Multi-model	127.51	+2.38	+3.11
9.	9.	↓8.	Elasticsearch	Multi-model	118.83	-2.45	-12.00
10.	10.	10.	SQLite	Relational	115.44	-1.60	+5.49
11.	11.	↑12.	Apache Cassandra	Wide column, Multi-model	108.76	+0.49	+9.63
12.	12.	↑15.	Databricks	Multi-model	108.04	+3.36	+24.74
13.	13.	↑14.	MariaDB	Relational, Multi-model	95.44	+0.91	+4.85
14.	14.	↓11.	Microsoft Access	Relational	90.47	+2.19	-10.17
15.	15.	↑17.	Amazon DynamoDB	Multi-model	84.15	+0.81	+13.20

https://db-engines.com/en/ranking

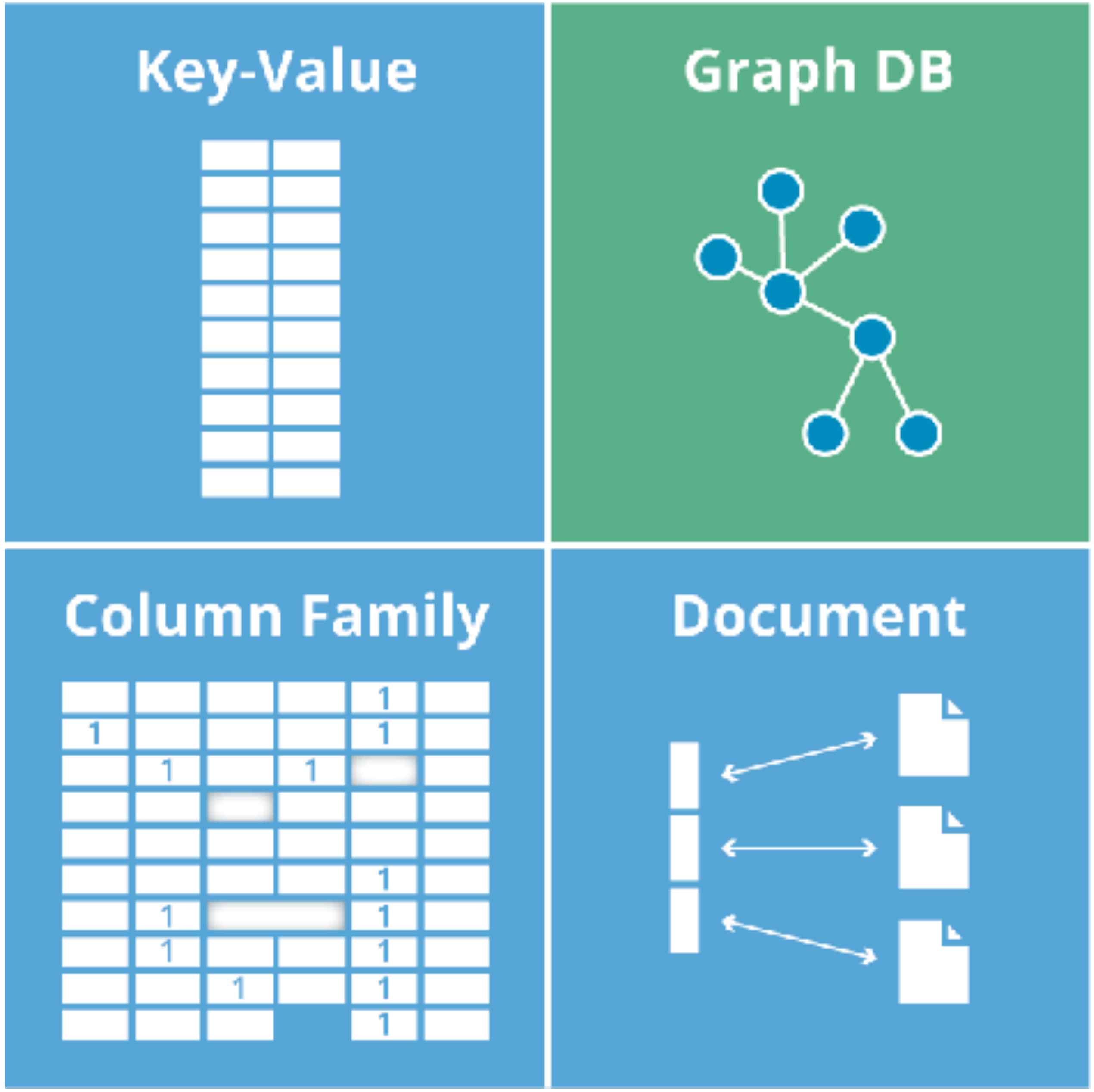
Database Model

Relational

Document
Key-value
Search engine
Wide column
Graph
Time series
Vector

Non-Relational
NoSQL

NoSQL



Relational Database

PostgreSQL

MySQL™

ORACLE®
DATABASE

Microsoft® SQL Server®

NoSQL

APACHE HBASE

Cassandra

riak

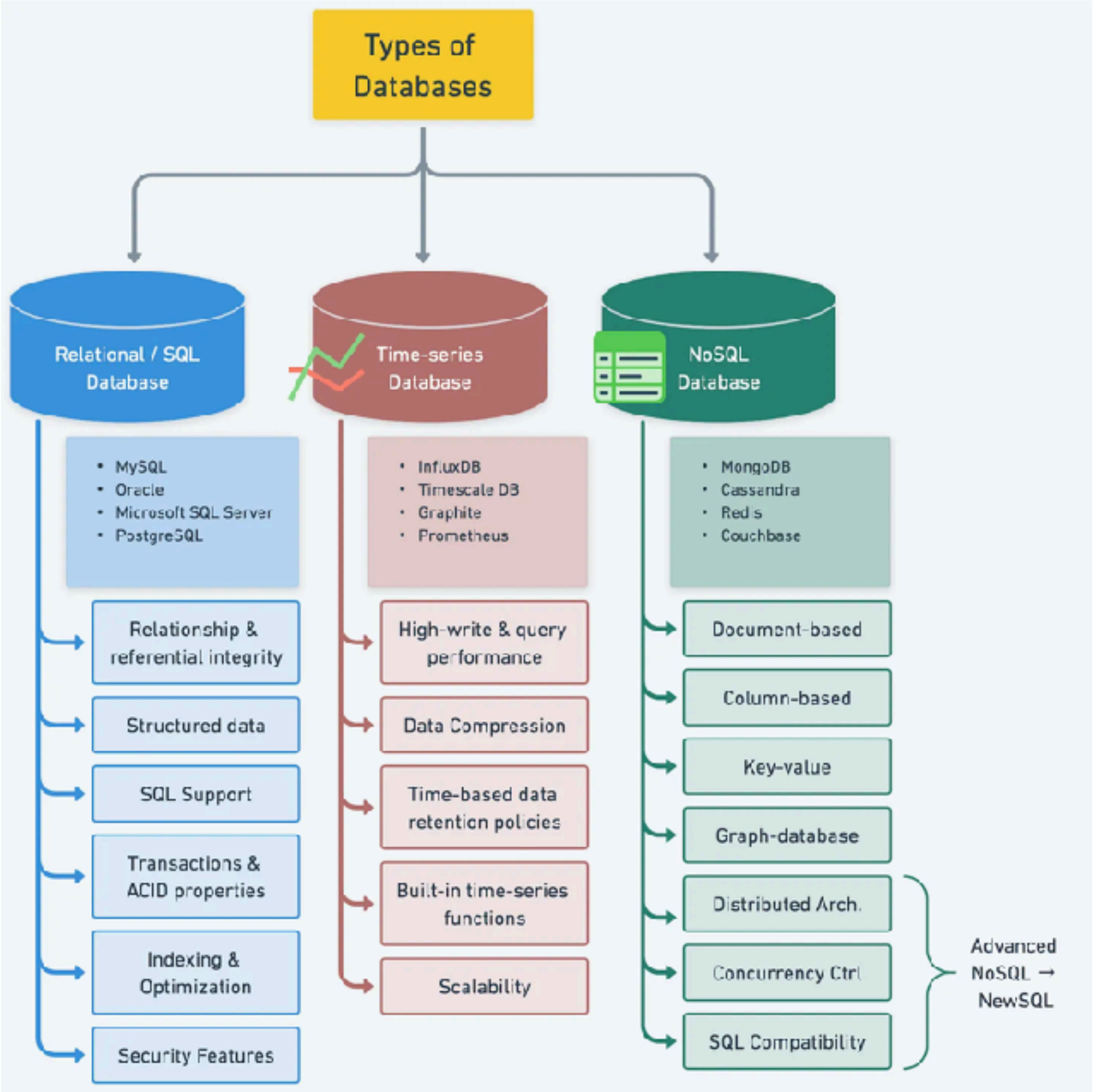
CouchDB relax

mongoDB

HYPERTABLE INC

Neo4j

redis



https://blog.bytebytego.com/p/understanding-database-types

Relational Database ?

Relational Database ?

Relational **D**ata**B**ase **M**anament **S**ystem

Maintain data in **tables**

Pre-define structure

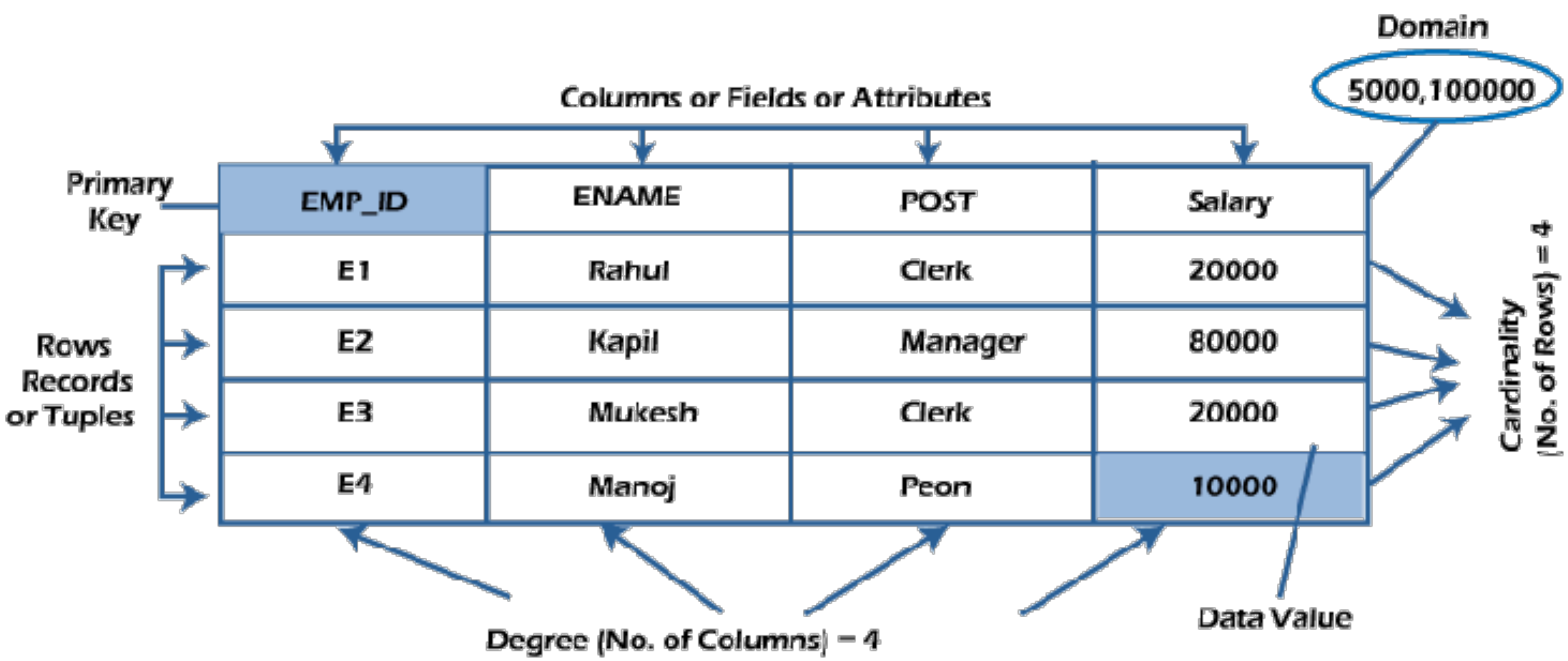
Each table has **Primary key**

Many **columns**

Data is represented in term of **row**

Relational Database ?

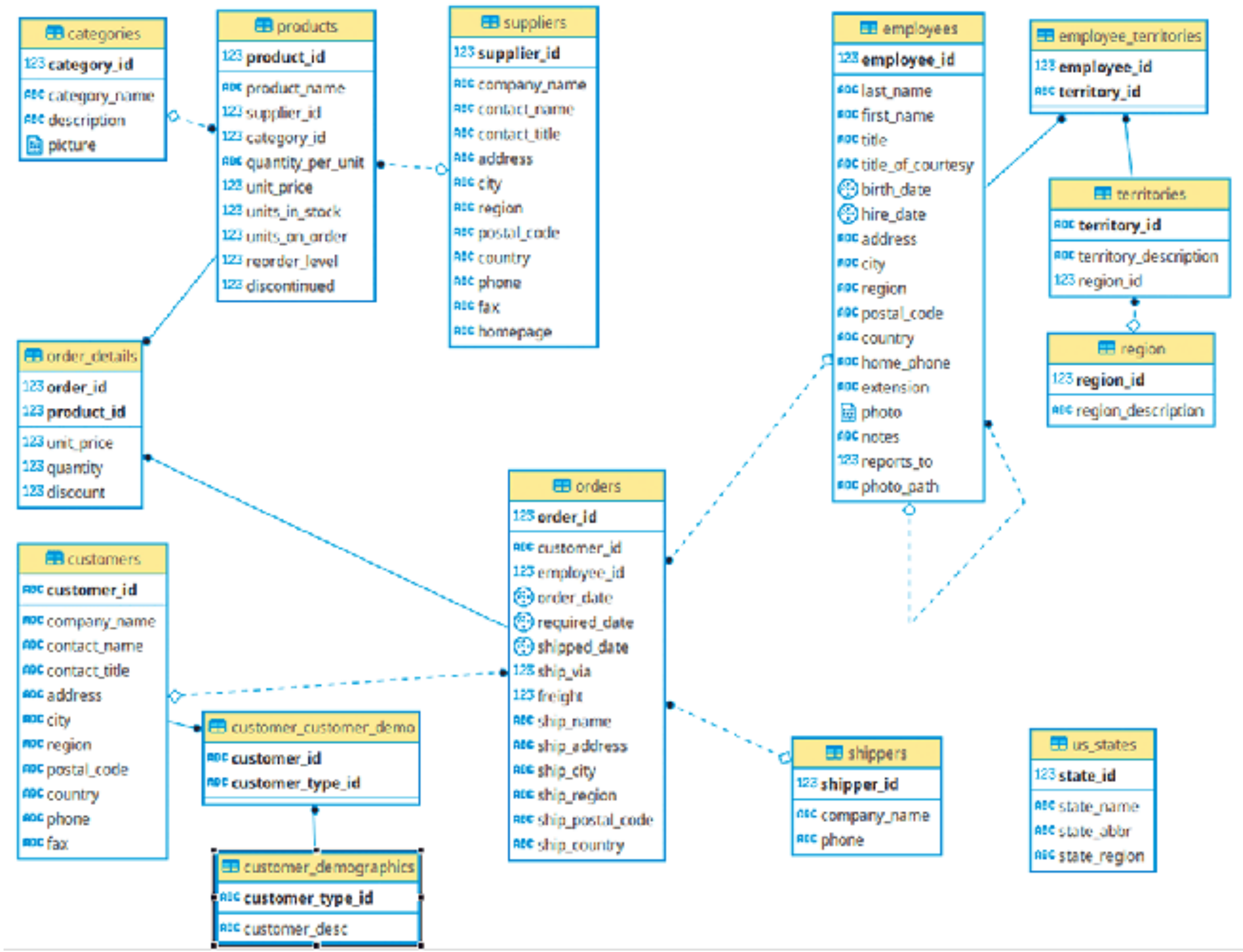
Table = Employee



Relational Database

Database	Ease of use	Support complex query	Community support	Licence
MySQL	High	High	Strong	Open source
PostgreSQL	Medium	Very high	Strong	Open source
MSSQL Server	Medium	Very high	Strong	Proprietary
Oracle	Low	Very high	Moderate	Proprietary

Real Design of RDBMS ?



Questions ?

How to write data ?

How to read data ?

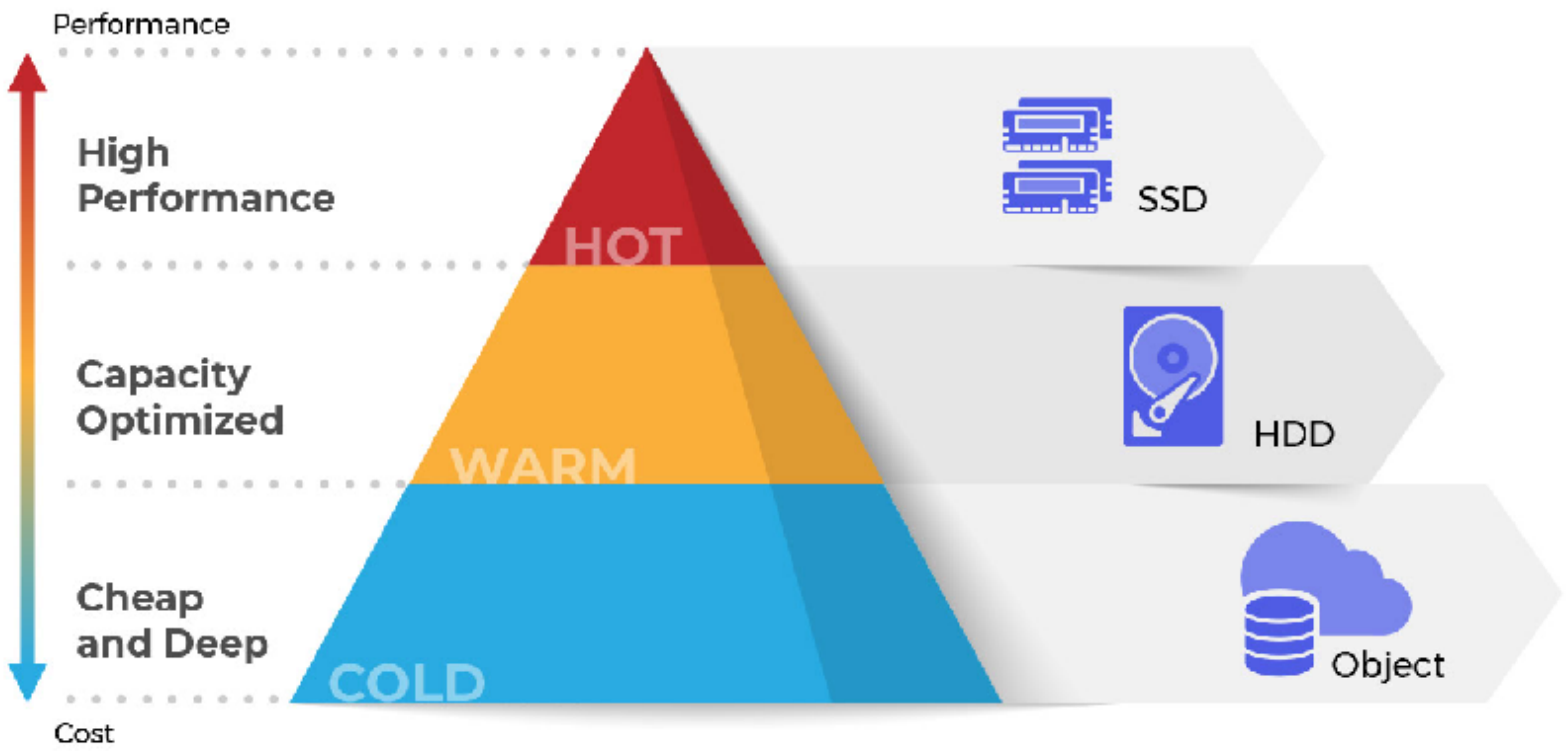
Aggregate data ?

Reporting

Data retention ?

Questions ?

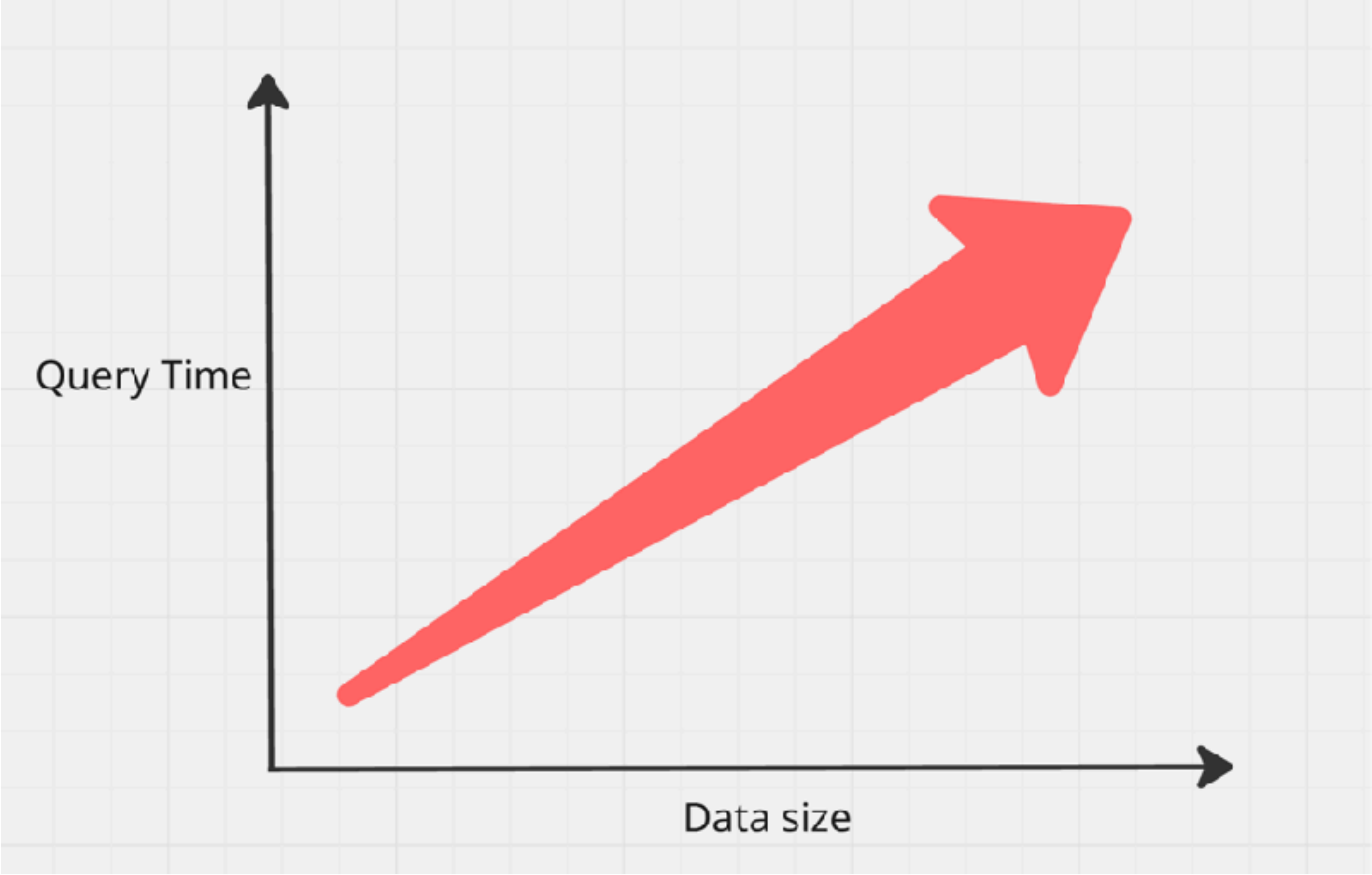
Hot vs Warm vs Cold data ?



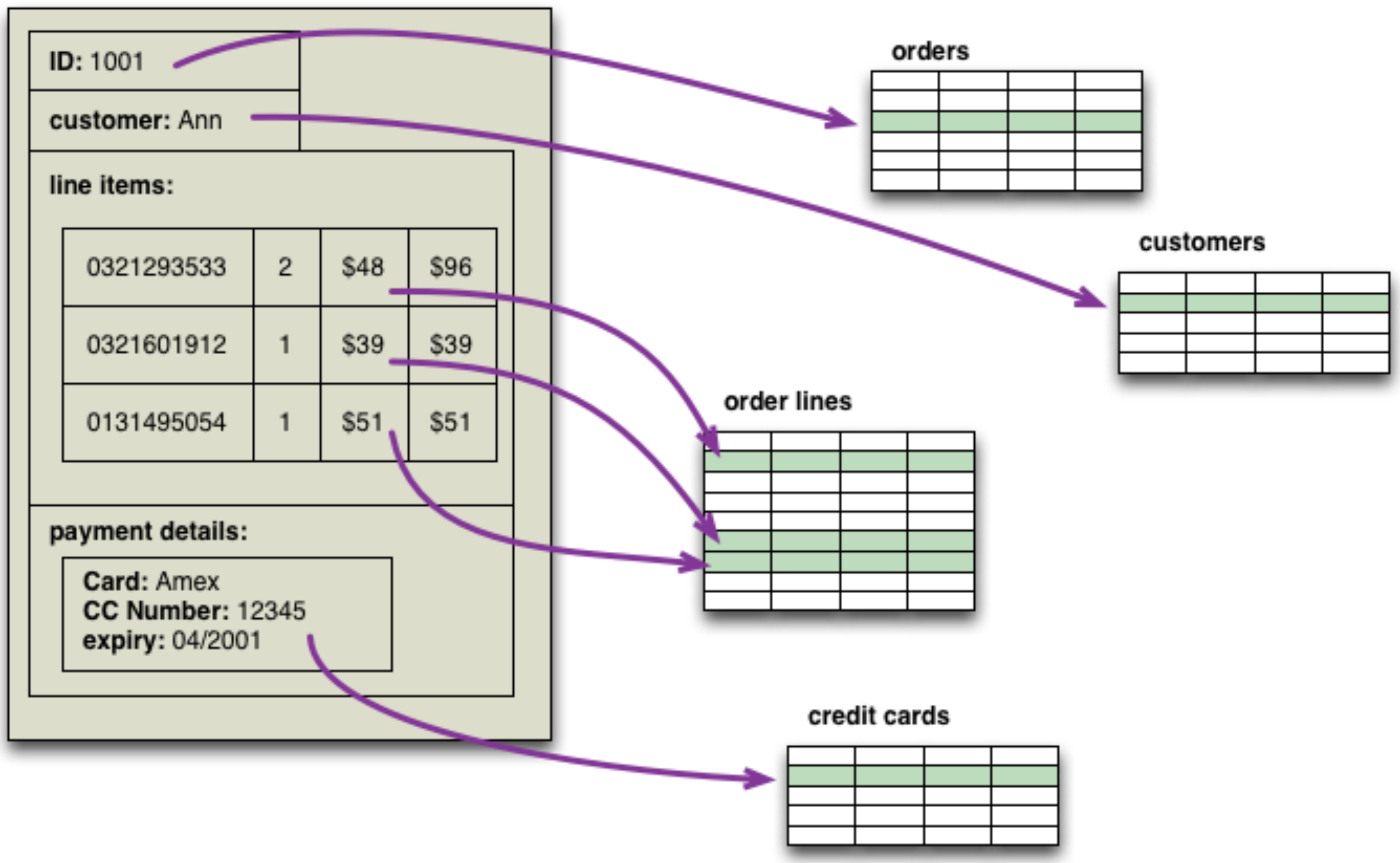
Complex Query ?

```sql
WITH tmp_1 AS
(
SELECT Calc1 =
( (SELECT TOP 1 DataValue
FROM (
SELECT TOP 50 PERCENT DataValue
FROM SOMEDATA
WHERE DataValue IS NOT NULL
ORDER BY DataValue
) AS A
ORDER BY DataValue DESC
) + (SELECT TOP 1 DataValue
FROM (
SELECT TOP 50 PERCENT DataValue
FROM SOMEDATA
WHERE DataValue Is NOT NULL
ORDER BY DataValue DESC
)AS A
ORDER BY DataValue ASC))/2
 ),tmp_2 AS
(SELECT AVG(DataValue) Mean, MAX(DataValue) - MIN(DataValue) AS MaxMinRange
FROM SOMEDATA
),tmp_3 AS
(
SELECT TOP 1 DataValue AS Calc2, COUNT(*) AS Calc2Count
FROM SOMEDATA
GROUP BY DataValue
ORDER BY Calc2Count DESC
)
SELECT Mean, Calc1, Calc2 , MaxMinRange AS [Range]
FROM tmp_1 CROSS JOIN tmp_2 CROSS JOIN tmp_3;
```

Problems ?



NoSQL



https://martinfowler.com/bliki/AggregateOrientedDatabase.html

Polyglot Persistence

Right tool for the right job

https://martinfowler.com/bliki/PolyglotPersistence.html

Data Modeling

Read

Write

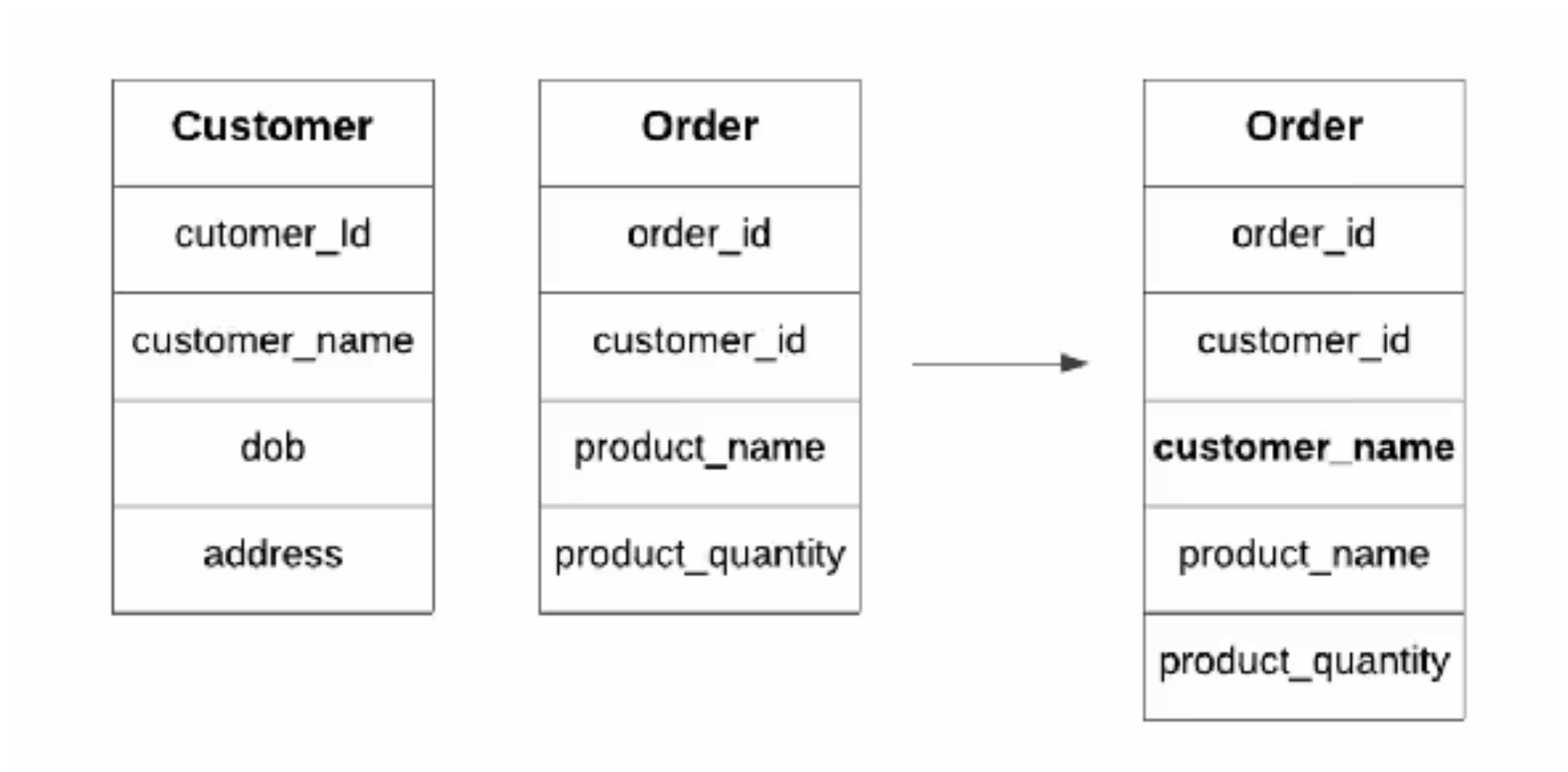
Data Modeling

Normalization

Denormalization

Pre-joins, pre-aggregation

Caching strategies



Normalization ?

Cleansing data
Reorganized data
Remove redundant data
Improve efficient for store data

Organize data in consistent way

Normalization

Divide in to several tables
Link tables with **relationships !!**

Design for write or read ?

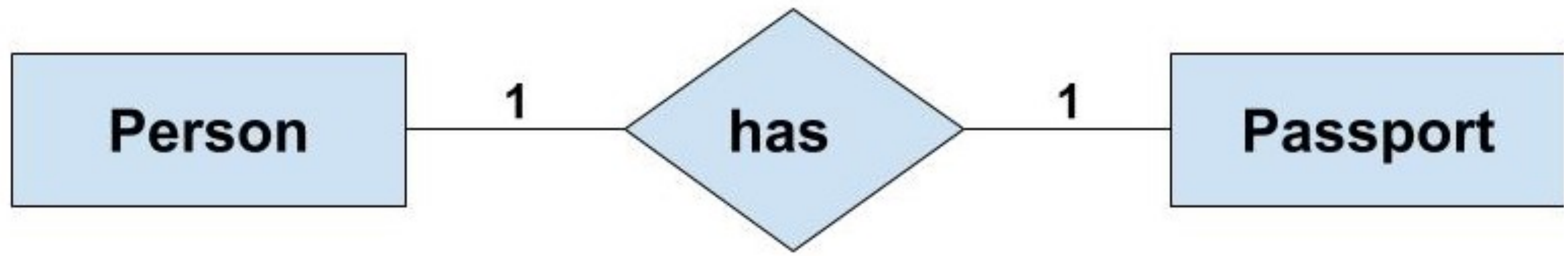
Relationships

Relationship between table

One-to-one

One-to-many

Many-to-many

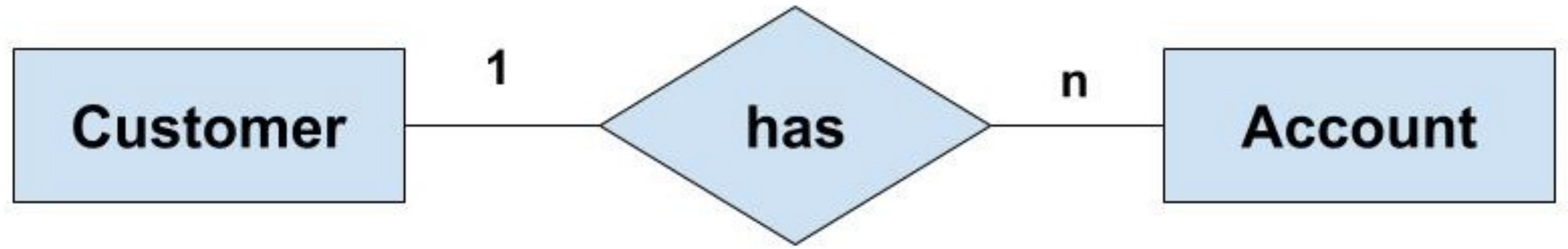


Relationship between table

One-to-one

One-to-many

Many-to-many

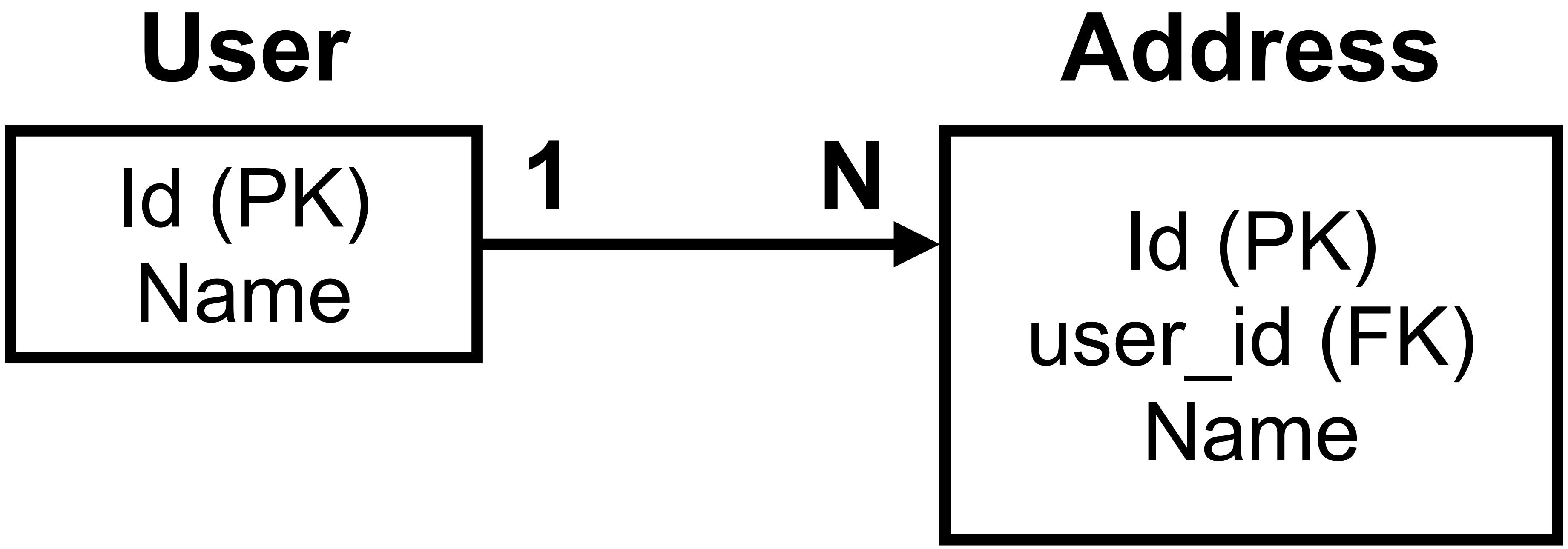


Relationship between table

One-to-one
One-to-many
Many-to-many

Relationships

Primary key (PK)
Composite key
Foreign key (FK)



Normalization forms

First Normal form (1NF)

2NF

3NF

4NF

5NF

6NF

First Normal Form (1NF)

No repeat data in table
Every column should only have single value
Every row should be unique

Sample data in 1NF

Name	Address	Gender	T-shirt Size
User 1	Address 1	Male	Large
User 2	Address 2	Female	Small
User 2	Address 2	Female	Medium
User 3	Address 3	Male	Medium

Second Normal Form (2NF)

Apply 1NF

Have one primary key

Problem in 1NF

Name	Address	Gender	T-shirt Size
User 1	Address 1	Male	Large
User 2	Address 2	Female	Small
User 2	Address 2	Female	Medium
User 3	Address 3	Male	Medium

Sample data in 2NF

User ID	Name	Address	Gender
1	User 1	Address 1	Male
2	User 2	Address 2	Female
3	User 3	Address 3	Male

Sample data in 2NF

User ID	T-shirt Size
1	Large
2	Small
2	Medium
3	Medium

Third Normal Form (3NF)

Apply 2NF

It should only be dependent on the primary key

if any changes to the primary key are made, all data that is impacted must be put into a new table

Problem in 2NF

User ID	Name	Address	Gender
1	User 1	Address 1	Male
2	User 2	Address 2	Female
3	User 3	Address 3	Male

Sample data in 3NF

User ID	Name	Address	Gender
1	User 1	Address 1	1
2	User 2	Address 2	2
3	User 3	Address 3	1

Sample data in 3NF

Gender ID	Gender name
1	Male
2	Female

Sample data in 3NF

User ID	T-shirt Size
1	Large
2	Small
2	Medium
3	Medium

3NF is normal form …

Benefits of data normalization

Freeing up space

Improve query response time

Reduce data inconsistency

Easier to use and analyze

Challenges of data normalization

Slower query response rates
Accurate knowledge is required
Add complexity for teams
Denormalization as an alternative !!

Primary Key (PK)

Foreign Key (FK)

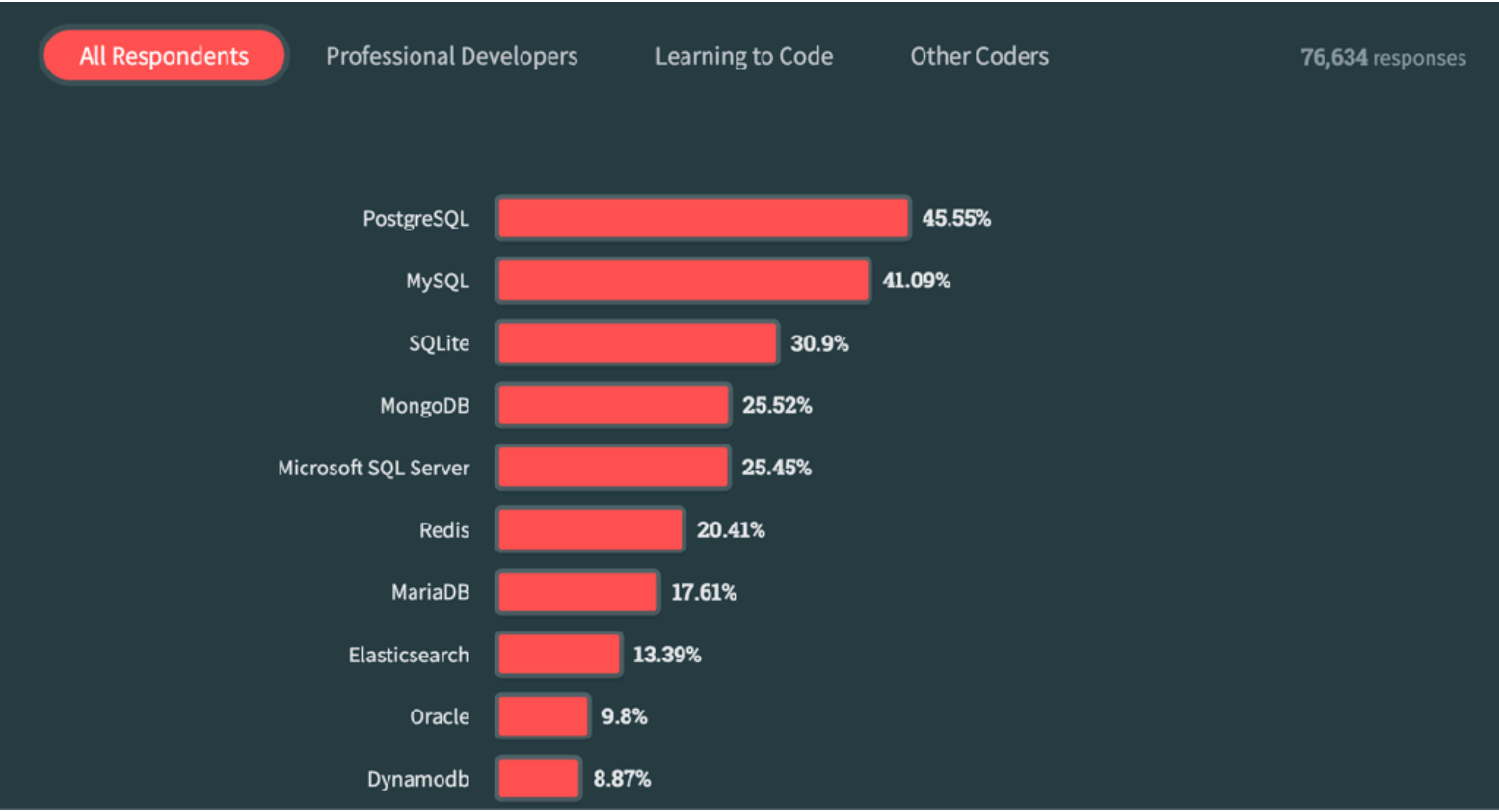
Other **Constraints**

Ensure data quality
Prevent inconsistency at the database level

Not Null	Unique	Check	Default

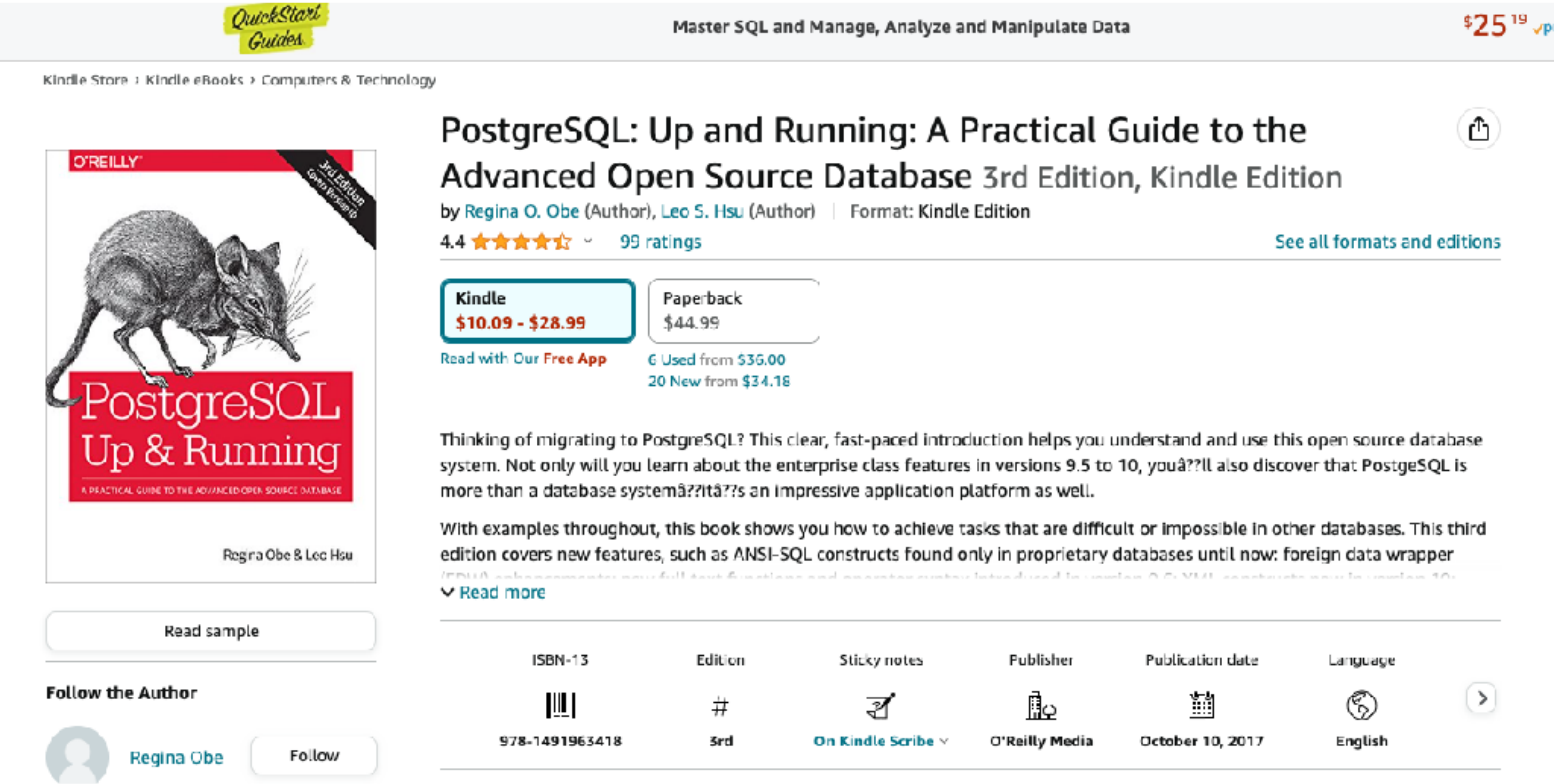
Workshop with PostgreSQL

PostgreSQL Database



https://survey.stackoverflow.co/2023/#most-popular-technologies-database

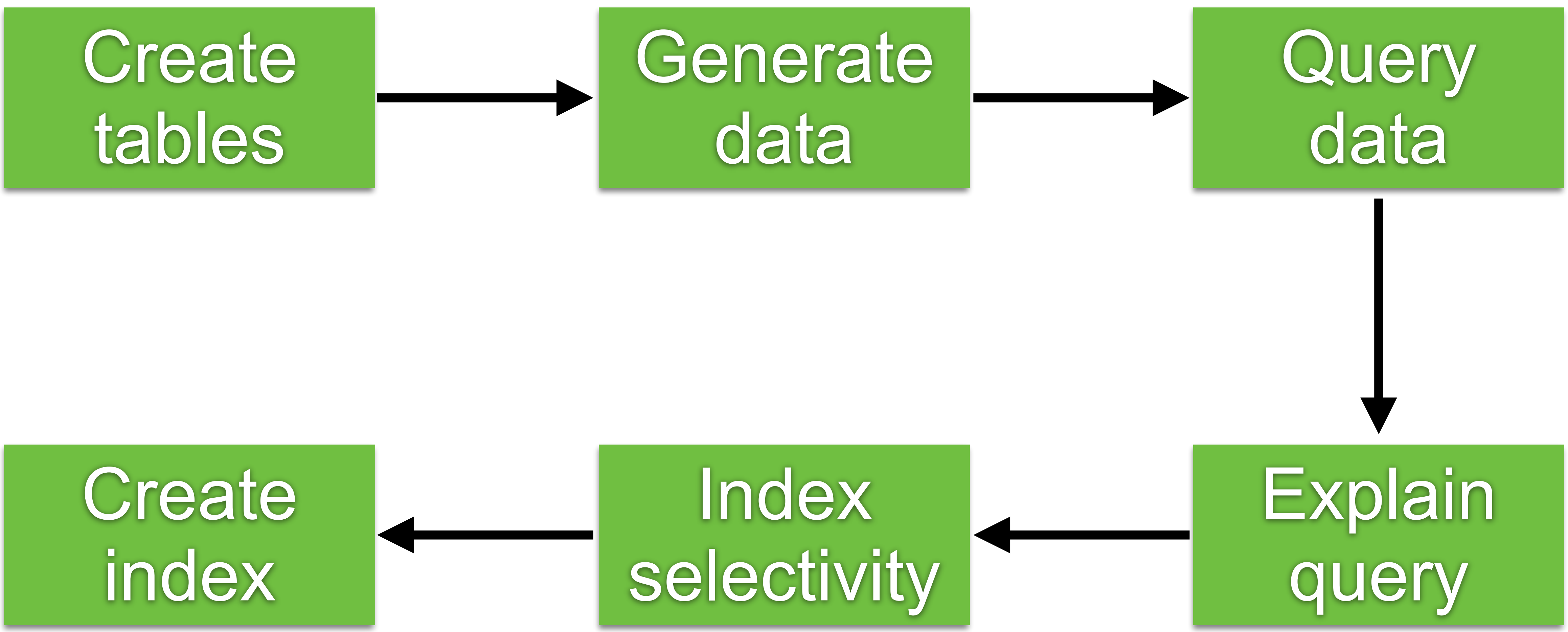
Book Store


QuickStart Guides
Master SQL and Manage, Analyze and Manipulate Data
$25.19
Kindle Store › Kindle eBooks › Computers & Technology
PostgreSQL: Up and Running: A Practical Guide to the Advanced Open Source Database 3rd Edition, Kindle Edition
by Regina O. Obe (Author), Leo S. Hsu (Author) | Format: Kindle Edition
4.4 99 ratings
See all formats and editions
Kindle
$10.09 - $28.99
Paperback
$44.99
Read with Our Free App
6 Used from $36.00
20 New from $34.18
Thinking of migrating to PostgreSQL? This clear, fast-paced introduction helps you understand and use this open source database system. Not only will you learn about the enterprise class features in versions 9.5 to 10, youâ??ll also discover that PostgeSQL is more than a database systemâ??itâ??s an impressive application platform as well.
With examples throughout, this book shows you how to achieve tasks that are difficult or impossible in other databases. This third edition covers new features, such as ANSI-SQL constructs found only in proprietary databases until now: foreign data wrapper
Read more
O'REILLY
3rd Edition
PostgreSQL Up & Running
A PRACTICAL GUIDE TO THE ADVANCED OPEN SOURCE DATABASE
Regina Obe & Leo Hsu
Read sample
Follow the Author
Regina Obe
Follow
ISBN-13
978-1491963418
Edition
3rd
Sticky notes
On Kindle Scribe
Publisher
O'Reilly Media
Publication date
October 10, 2017
Language
English

Book Store


book
book_id UUID
isbn VARCHAR(14)
title VARCHAR(255) NN
publication_date DATE NN
publisher_id UUID
rating NUMERIC(4, 3)
book_category
book_id UUID
category_id UUID
category
category_id UUID
name VARCHAR(255) NN
publisher
publisher_id UUID
name VARCHAR(255) NN
book_author
book_id UUID
author_id UUID
author
author_id UUID
full_name VARCHAR(255) NN

Step in workshop

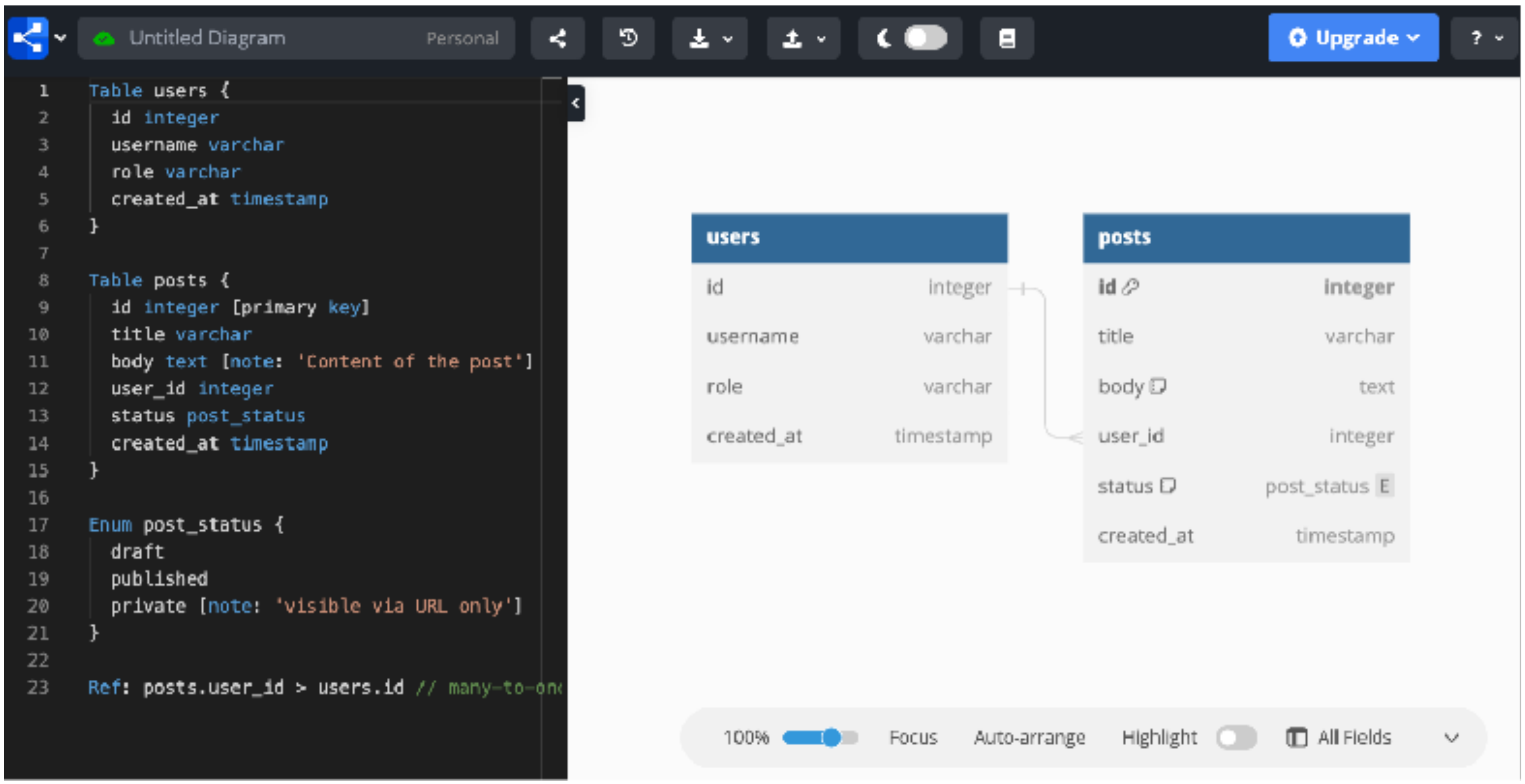


https://github.com/up1/workshop-database/tree/main/workshop/basic-workshop

Design

Design table with DBML

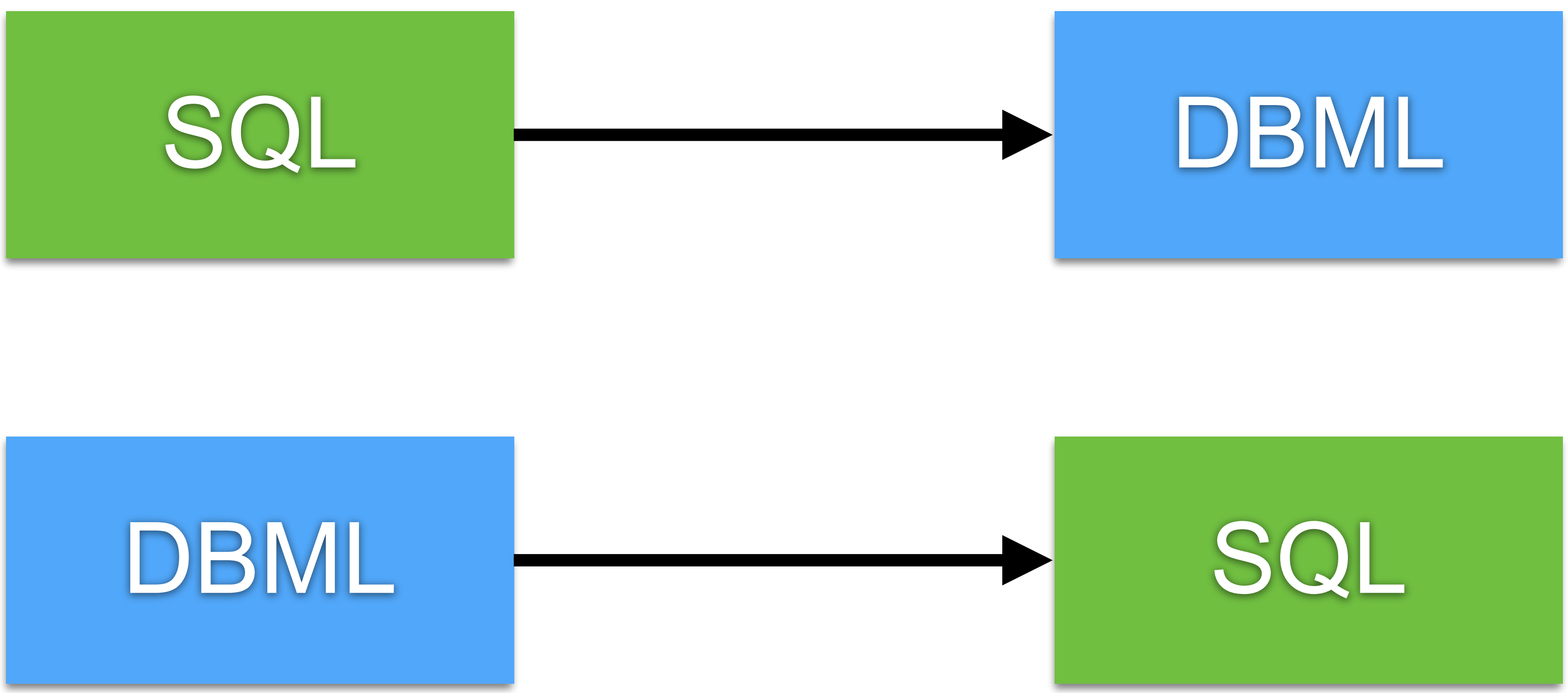
Database Markup Language



https://dbml.dbdiagram.io/home/

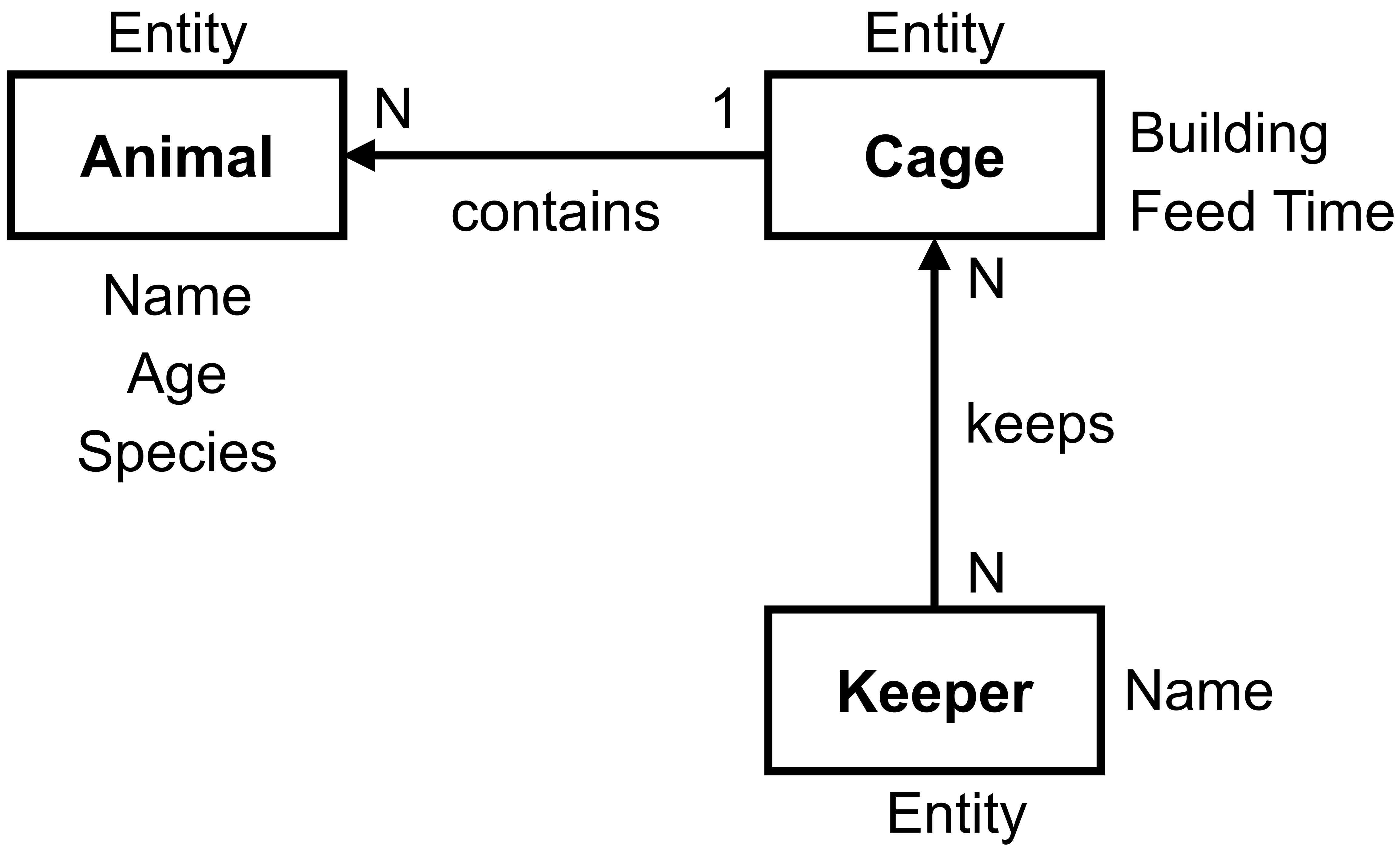
Convert SQL to DBML

$npm install -g @dbml/cli



https://dbml.dbdiagram.io/cli/#convert-a-sql-file-to-dbml

Design with entity relation diagram


Entity
Animal
Name
Age
Species
N
contains
1
Entity
Cage
Building
Feed Time
N
keeps
N
Keeper
Name
Entity

How to manage RDBMS ?

Database Language

Data Definition Language (DDL)
Data Manipulation Language (DML)
Data Control Language (DCL)
Transaction Control Language (TCL)

Data Definition Language (DDL)

Create
Alter
Drop
Truncate

Data Manipulation Language (DML)

Select
Insert
Update
Delete

Delete vs Truncate ?

Delete vs Truncate

Delete

Row-level operation

Mark every row matching the `WHERE`-clause as deleted

Slow operation

Truncate

Table operation

Empty the table

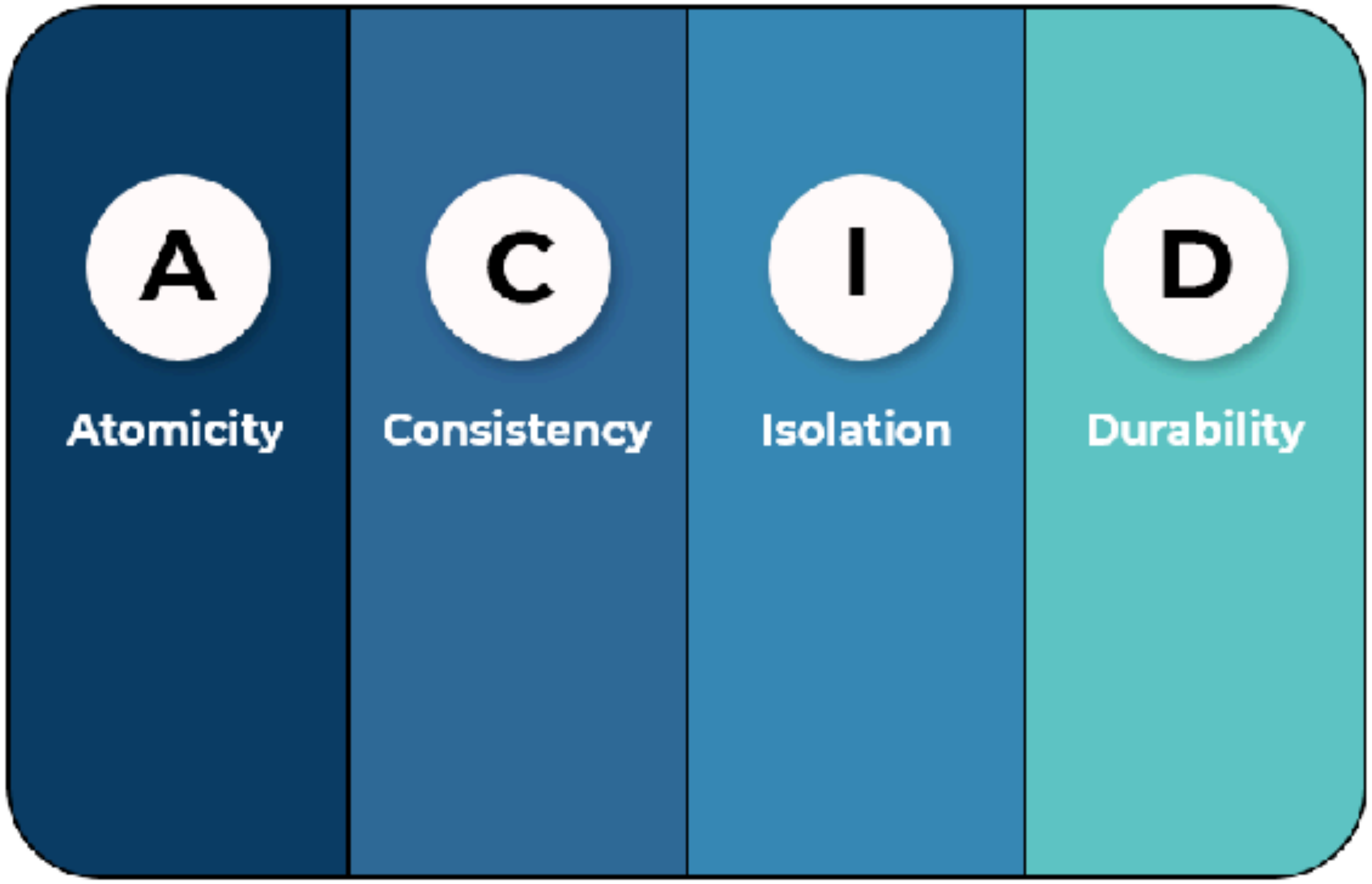
Start a new data file

ACID in RDBMS

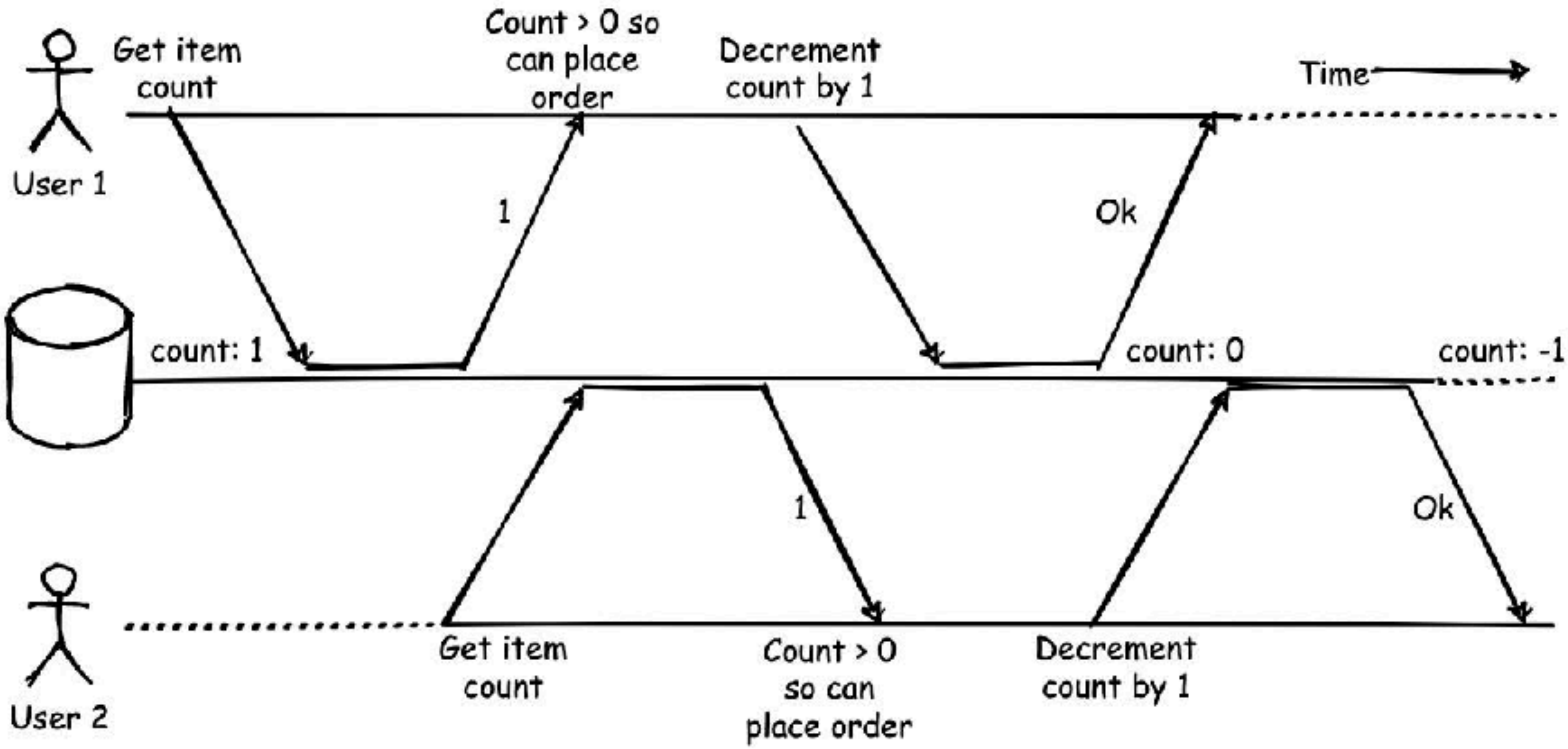
https://en.wikipedia.org/wiki/ACID

ACID in RDBMS

Ensure database in valid state even in the event of unexpected errors

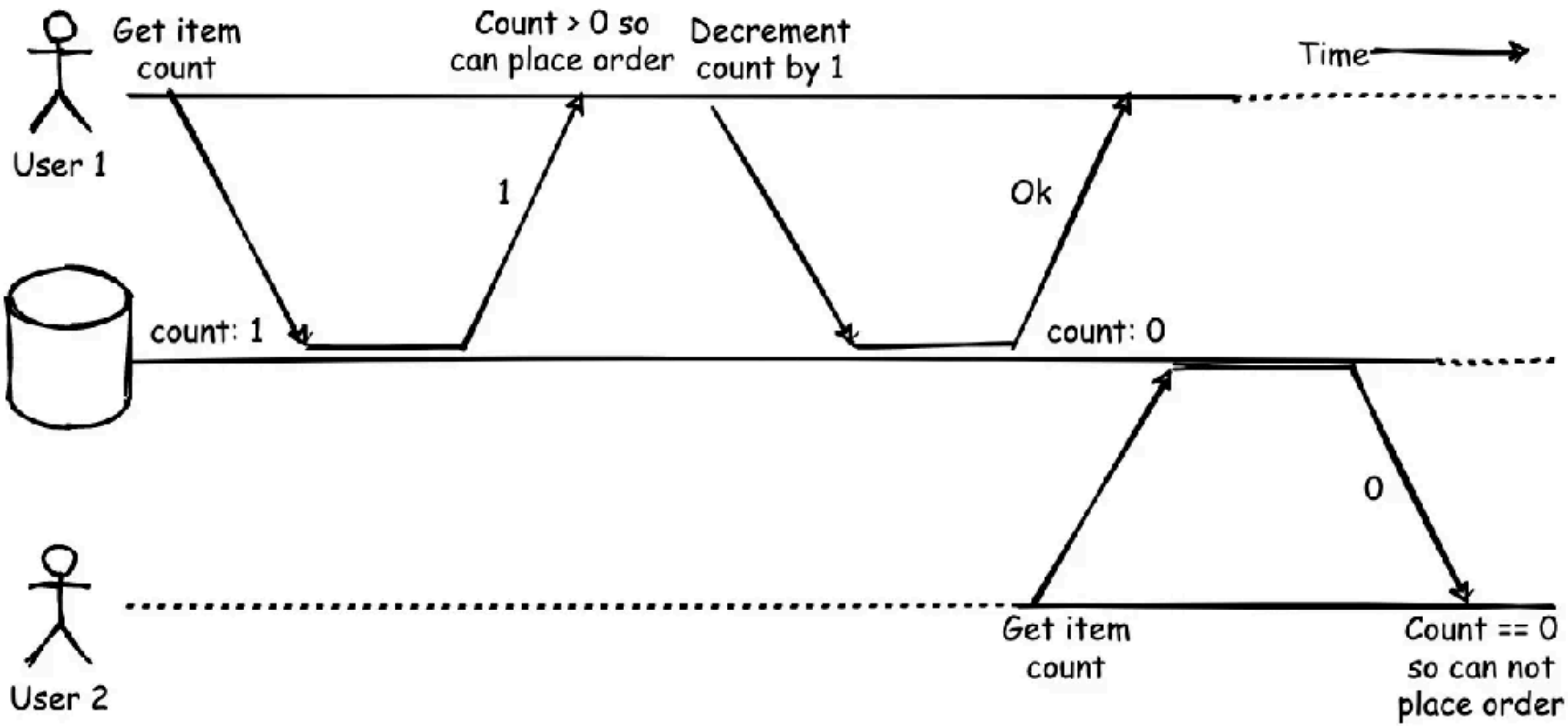


Race condition



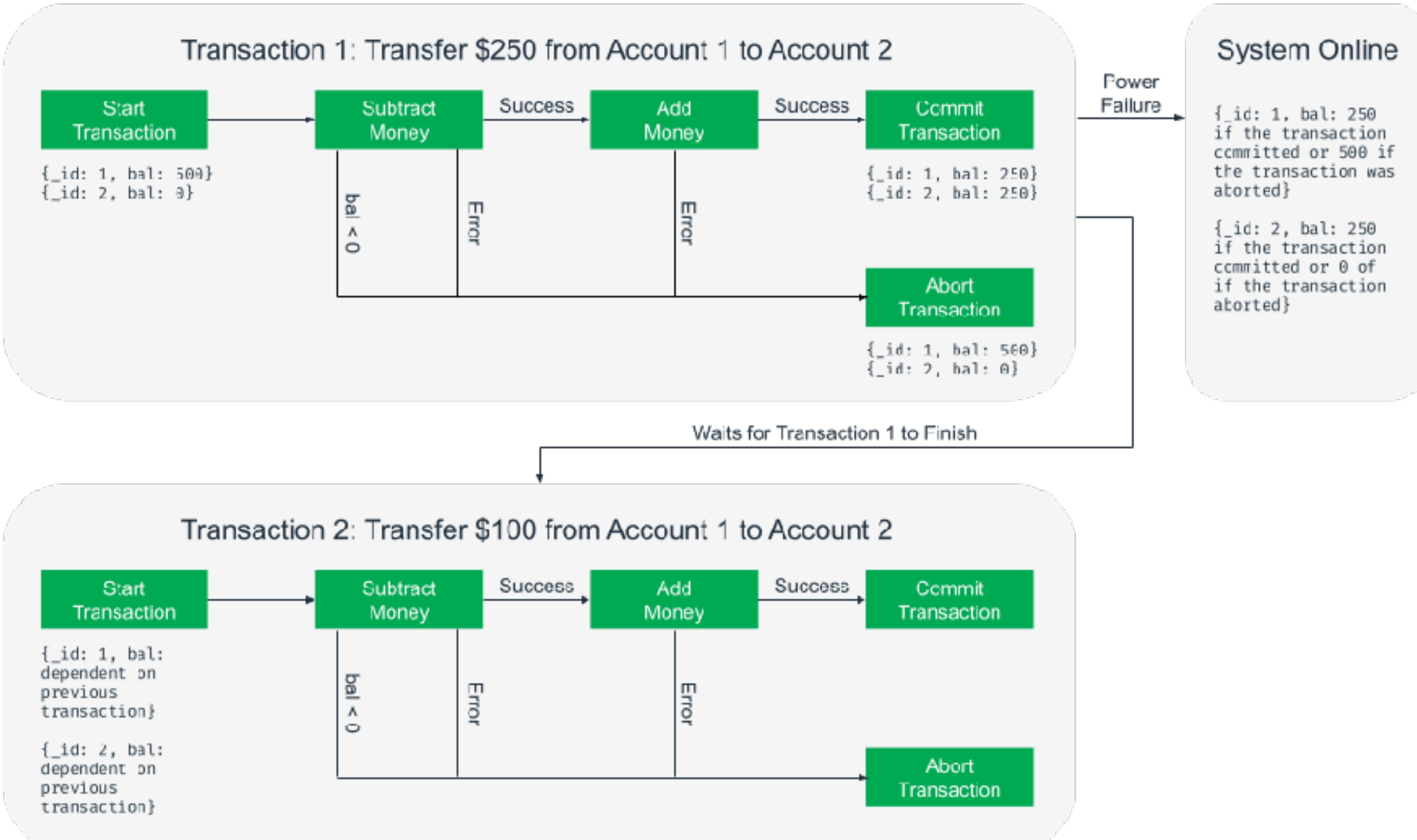
https://en.wikipedia.org/wiki/Race_condition#Example

Sequencial



https://www.mongodb.com/basics/acid-transactions

ACID in RDBMS



https://www.mongodb.com/basics/acid-transactions

Structure **Q**uery **L**anguage

SQL

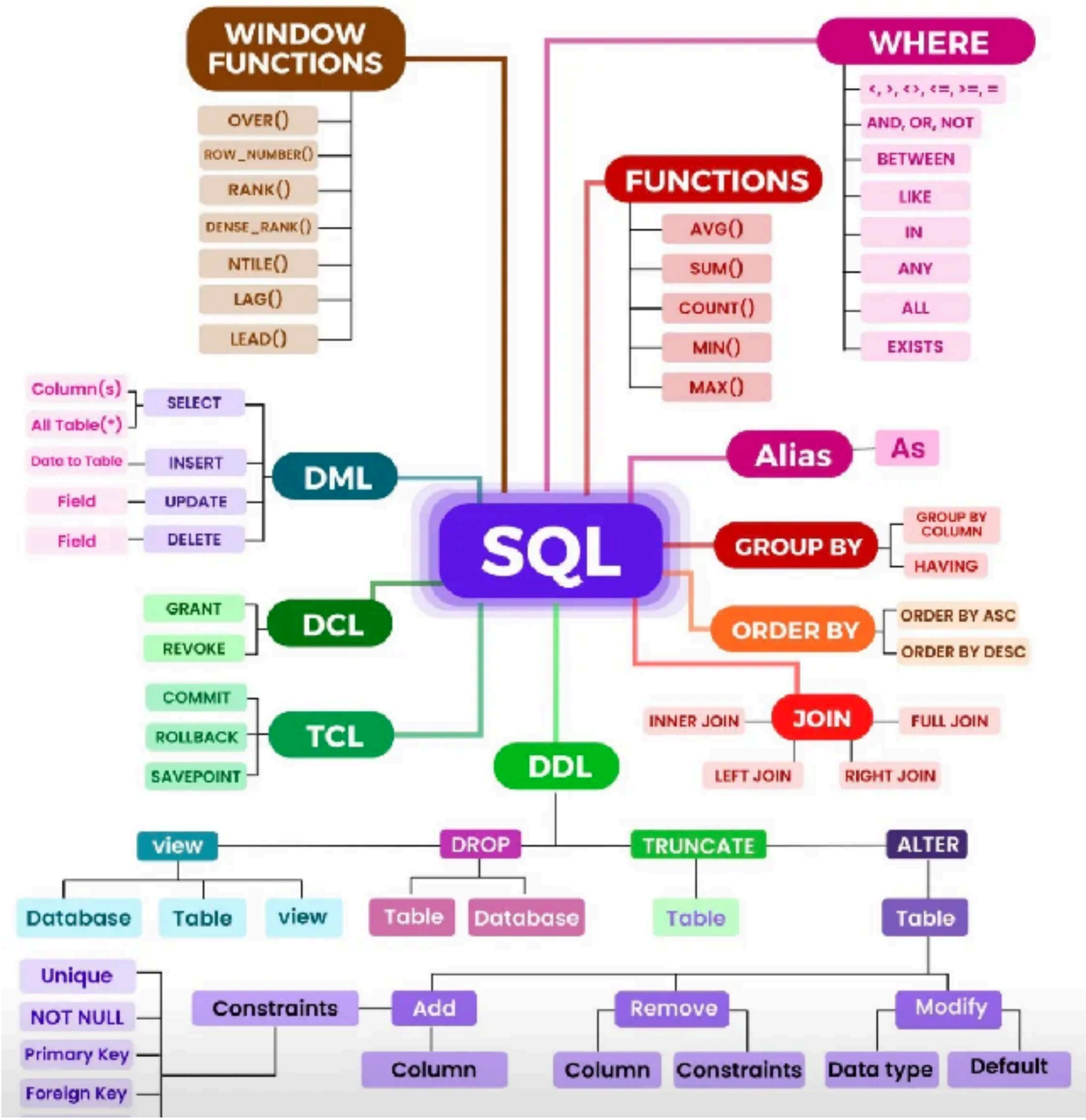
Maintain data in **tables**

Pre-define structure

Each table has **Primary key**

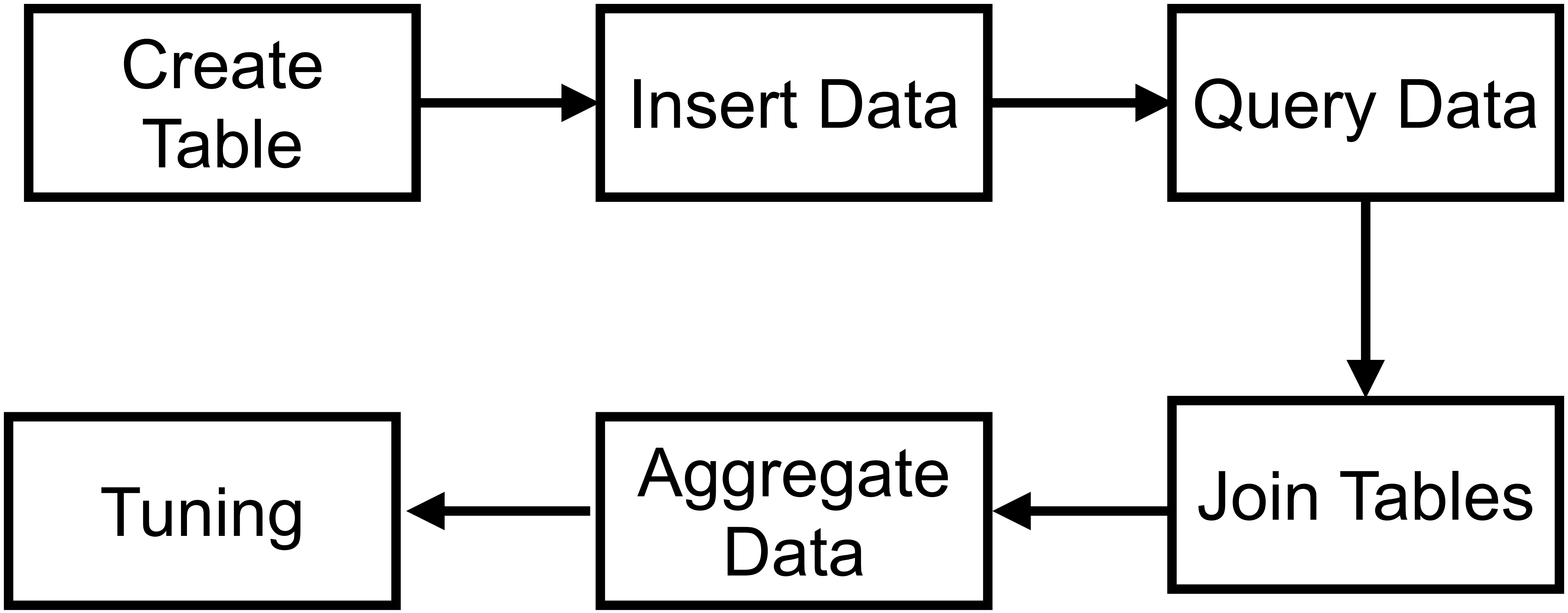
Many **columns**

Data is represented in term of **row**



WINDOW FUNCTIONS
OVER()
ROW_NUMBER()
RANK()
DENSE_RANK()
NTILE()
LAG()
LEAD()
WHERE
<, >, <>, <=, >=, =
AND, OR, NOT
BETWEEN
LIKE
IN
ANY
ALL
EXISTS
FUNCTIONS
AVG()
SUM()
COUNT()
MIN()
MAX()
Column(s)
All Table(*)
SELECT
Data to Table
INSERT
Field
UPDATE
Field
DELETE
DML
SQL
Alias
As
GROUP BY
GROUP BY COLUMN
HAVING
GRANT
REVOKE
DCL
ORDER BY
ORDER BY ASC
ORDER BY DESC
COMMIT
ROLLBACK
SAVEPOINT
TCL
INNER JOIN
JOIN
FULL JOIN
LEFT JOIN
RIGHT JOIN
DDL
view
DROP
TRUNCATE
ALTER
Database
Table
view
Table
Database
Table
Table
Unique
NOT NULL
Primary Key
Foreign Key
Constraints
Add
Column
Remove
Column
Constraints
Modify
Data type
Default

Learning paths



Query Data

SELECT columns
FROM table
WHERE condition A
AND condition B
GROUP BY columns
ORDER BY columns **ASC|DESC**

Select Data process

Table A	Table B	Table C

Select Data process

Table A | Table B | Table C

FROM + JOIN (merge)

Table A, B, C

Select Data process

Table A

Table B

Table C

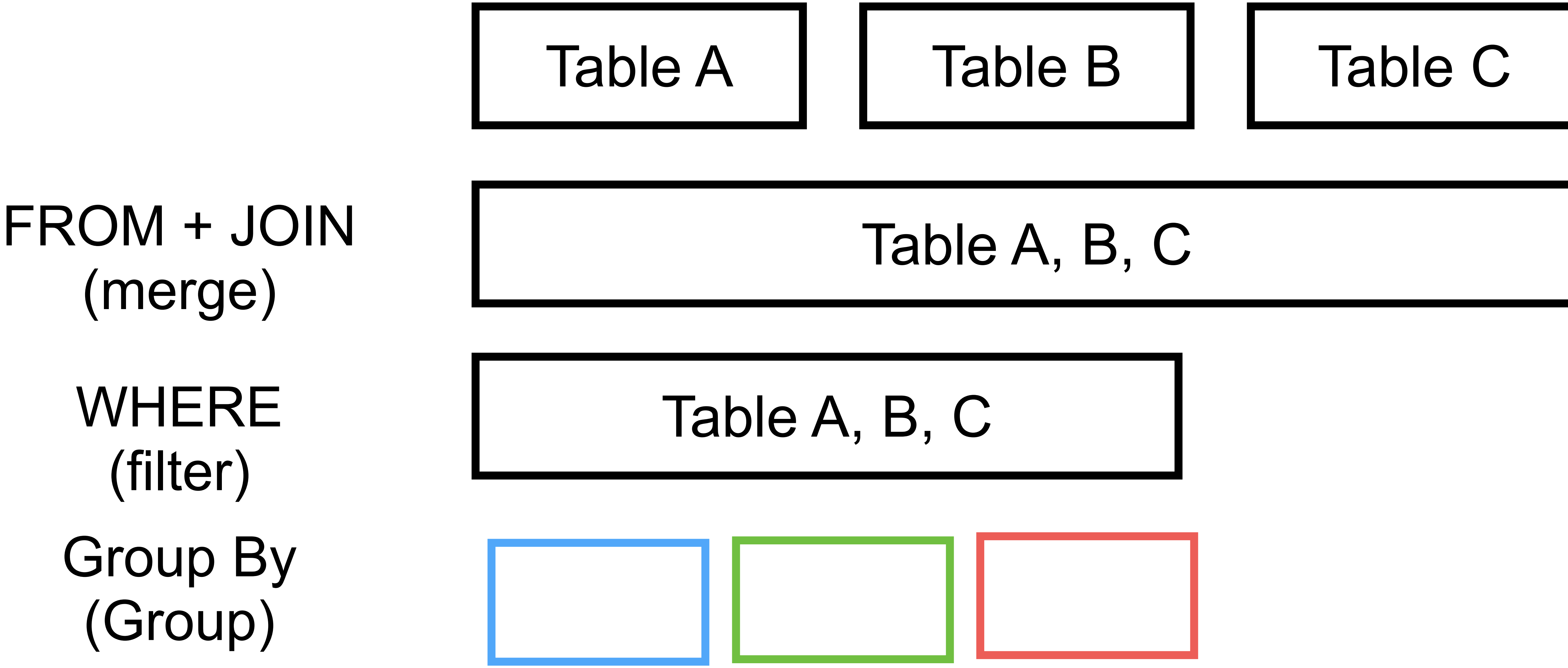
FROM + JOIN (merge)

Table A, B, C

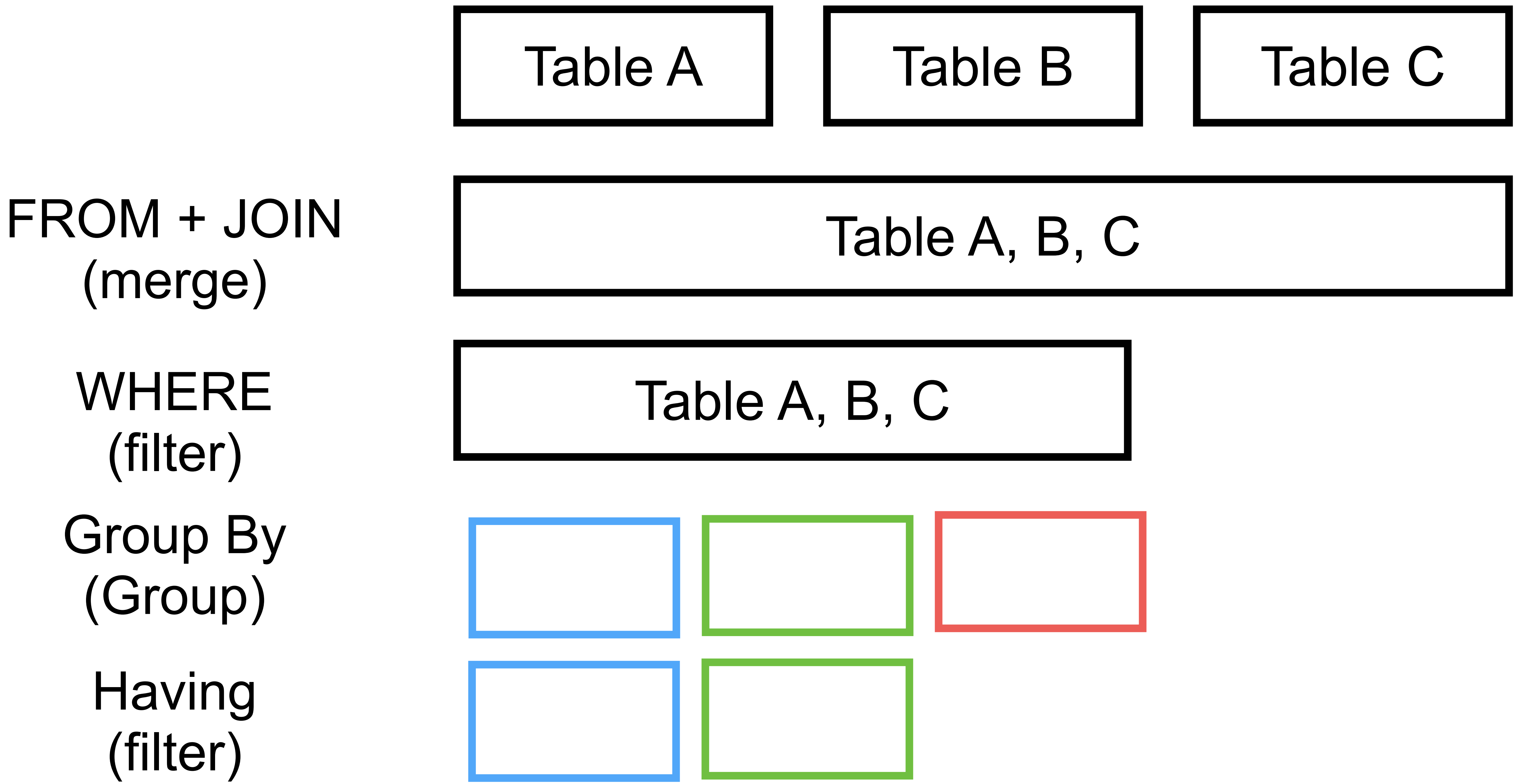
WHERE (filter)

Table A, B, C

Select Data process



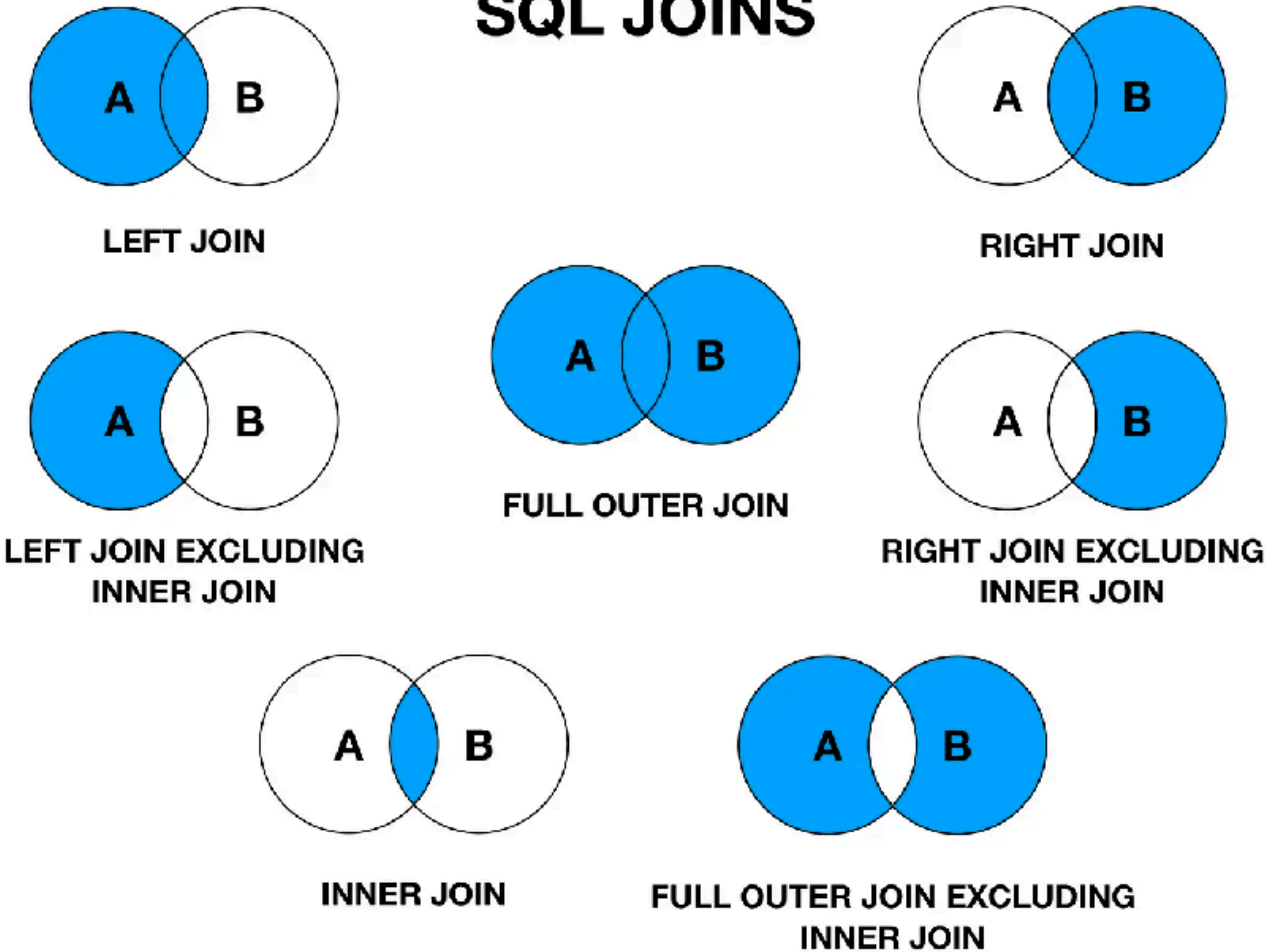
Select Data process



Joining

https://www.postgresql.org/docs/current/tutorial-join.html

SQL JOINS


A
B
LEFT JOIN
RIGHT JOIN
FULL OUTER JOIN
LEFT JOIN EXCLUDING INNER JOIN
RIGHT JOIN EXCLUDING INNER JOIN
INNER JOIN
FULL OUTER JOIN EXCLUDING INNER JOIN

Self-join

Join in PostgreSQL

Left join

Inner join

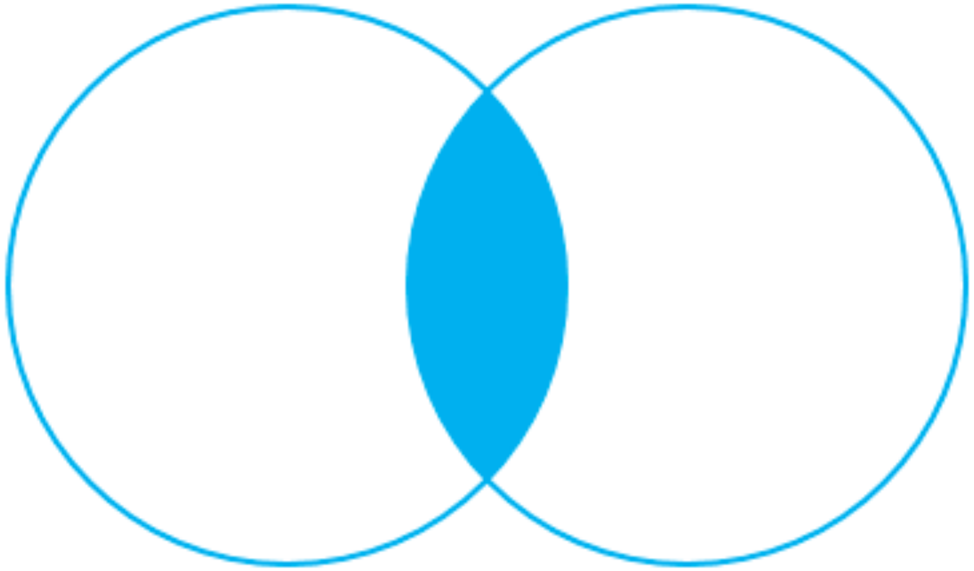
Right join

Full outer join

Workshop

Basket A	Basket B
Apple	Orange
Orange	Apple
Banana	Watermelon
Cucumber	Pear

Inner join

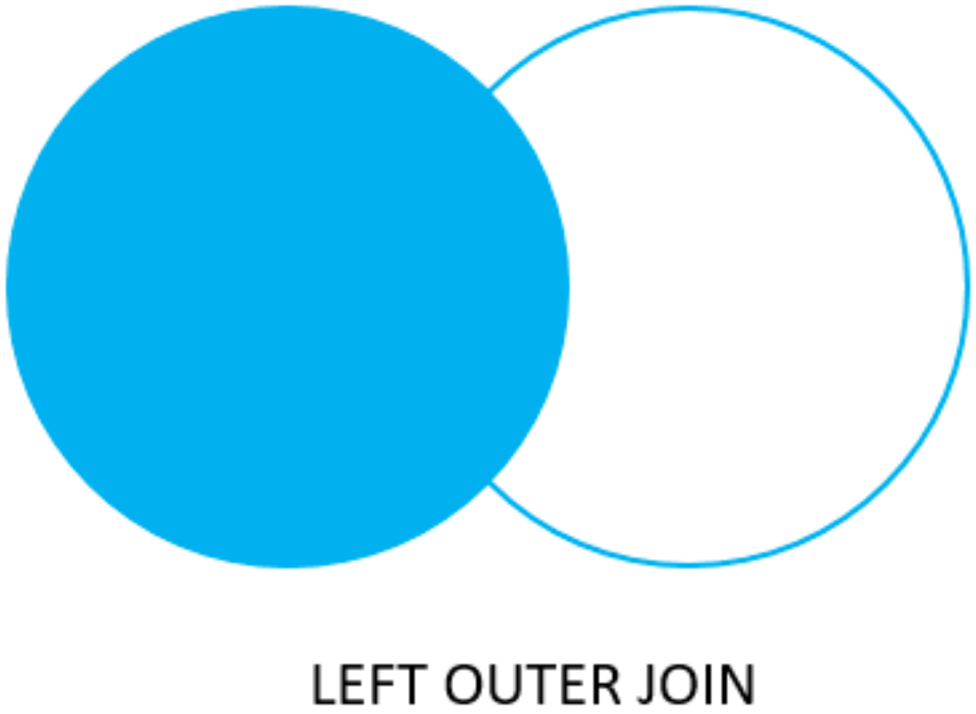


INNER JOIN

Inner join

Basket A	Basket B
Apple	**Orange**
Orange	**Apple**
Banana	Watermelon
Cucumber	Pear

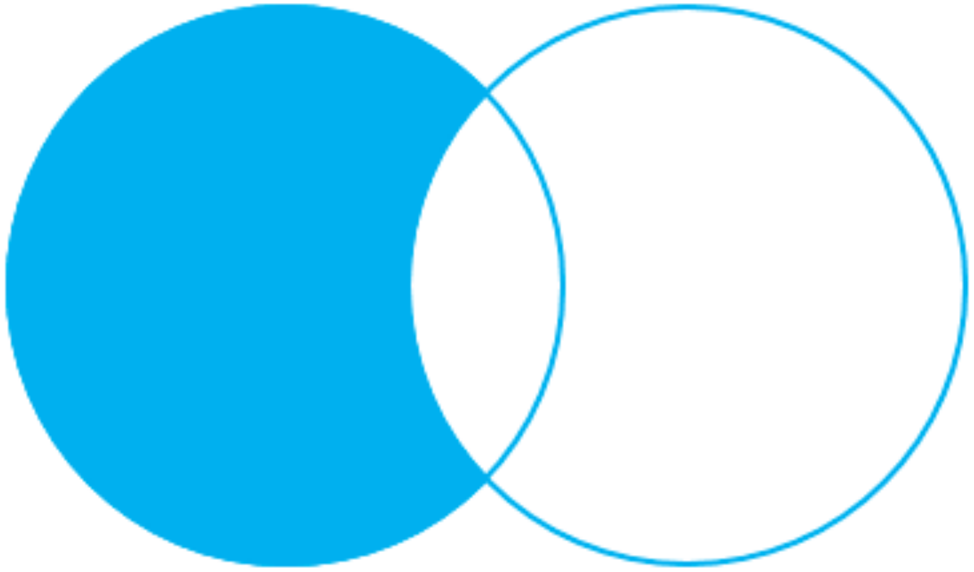
Left join (1)



Left join (1)

Basket A	Basket B
Apple	**Orange**
Orange	**Apple**
Banana	Watermelon
Cucumber	Pear

Left join (2)

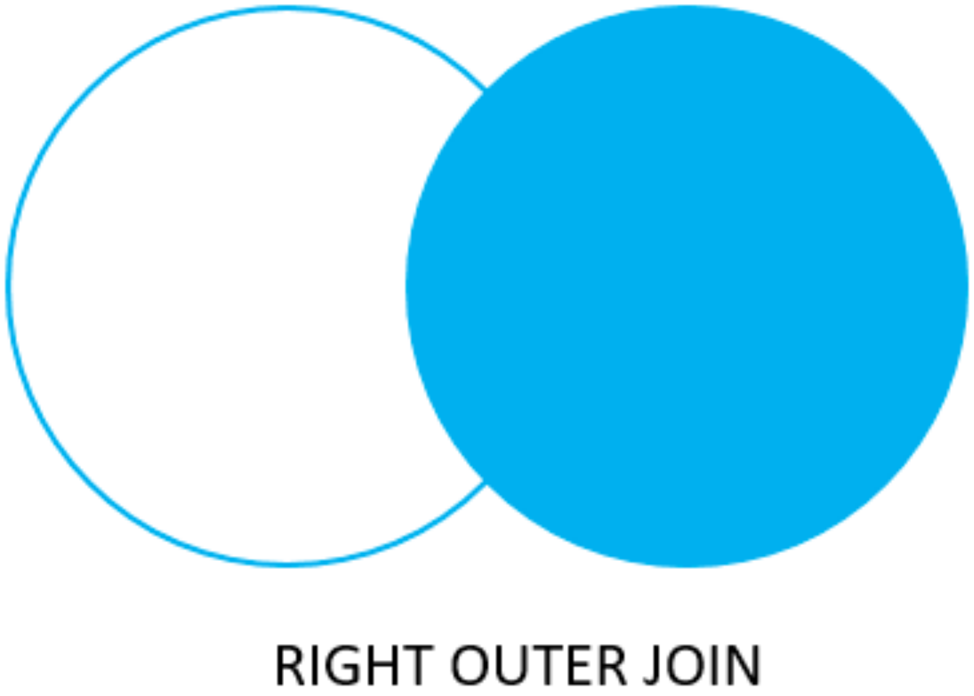


LEFT OUTER JOIN – only
rows from the left table

Left join (2)

Basket A	Basket B
Apple	Orange
Orange	Apple
Banana	Watermelon
Cucumber	Pear

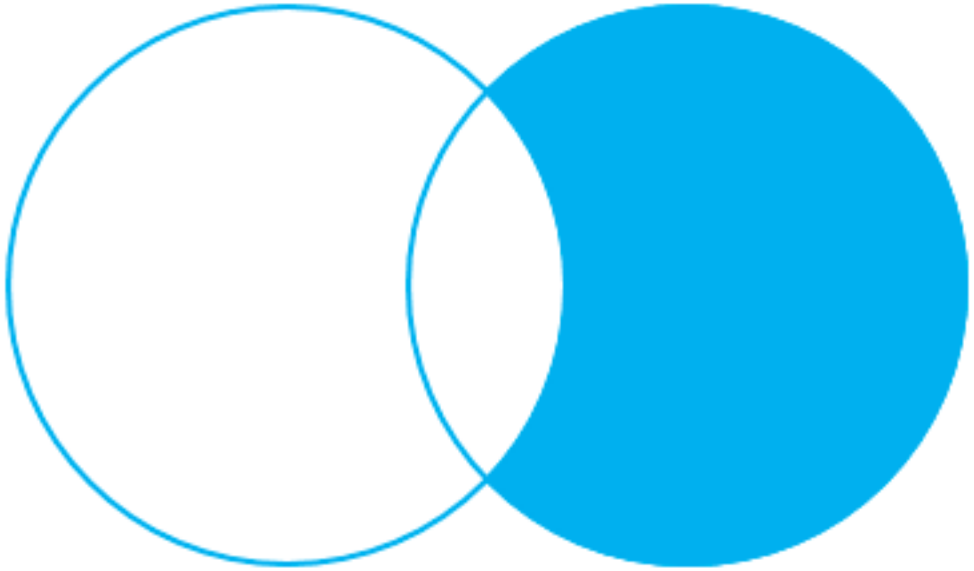
Right join (1)



Right join (1)

Basket A	Basket B
Apple	**Orange**
Orange	**Apple**
Banana	**Watermelon**
Cucumber	**Pear**

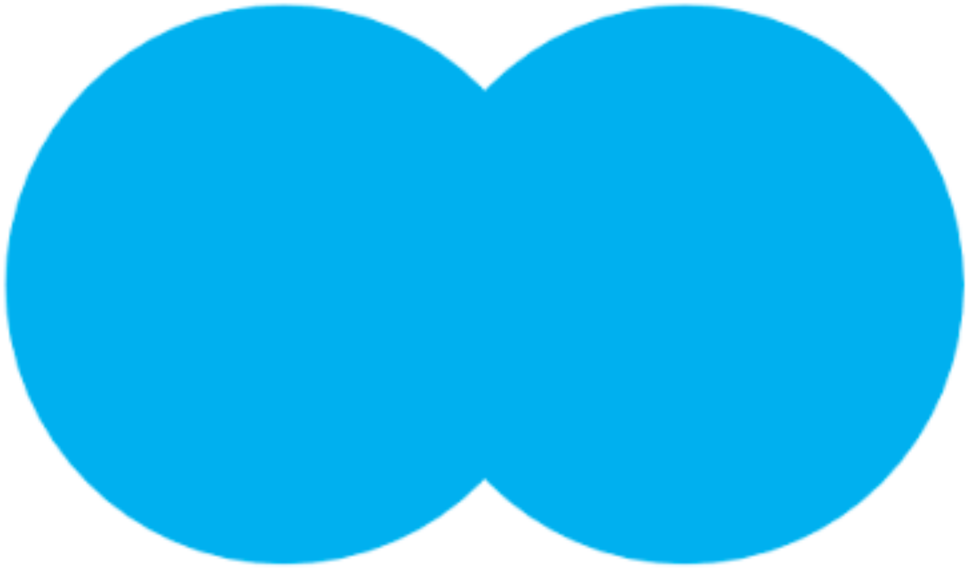
Right join (2)



Right join (2)

Basket A	Basket B
Apple	Orange
Orange	Apple
Banana	**Watermelon**
Cucumber	**Pear**

Full outer join (1)



FULL OUTER JOIN

Full outer join (1)

Basket A	Basket B
Apple	Orange
Orange	Apple
Banana	Watermelon
Cucumber	Pear

2

Full outer join (1)

Basket A	Basket B
Apple	Orange
Orange	Apple
Banana	Watermelon
Cucumber	Pear

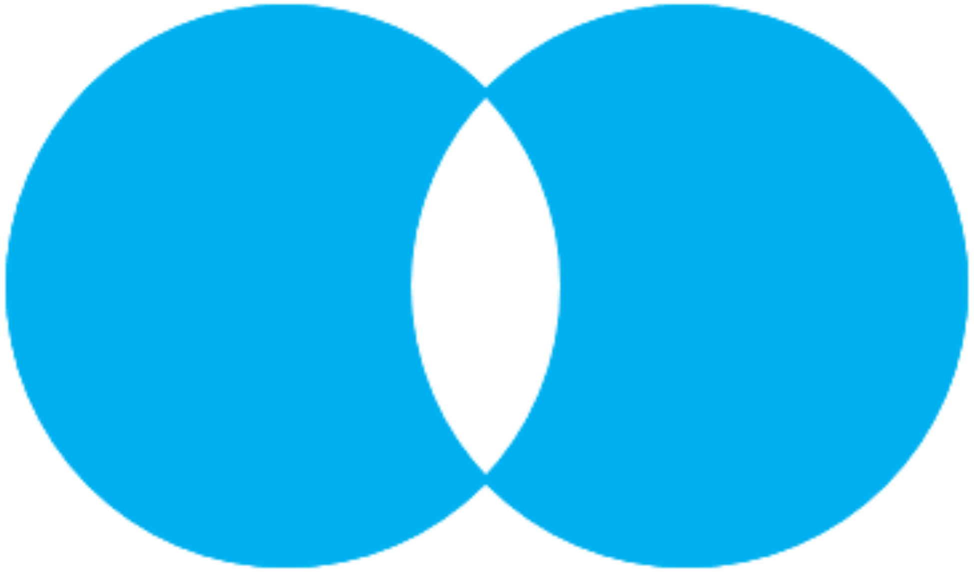
2

Full outer join (1)

Basket A	Basket B
Apple	Orange
Orange	Apple
Banana	Watermelon
Cucumber	Pear

2

Full outer join (2)



FULL OUTER JOIN – only
rows unique to both tables

Full outer join (2)

Basket A	Basket B
Apple	Orange
Orange	Apple
Banana	Watermelon
Cucumber	Pear

2

Full outer join (2)

Basket A

Apple
Orange
Banana
Cucumber

Basket B

Orange
Apple
Watermelon
Pear

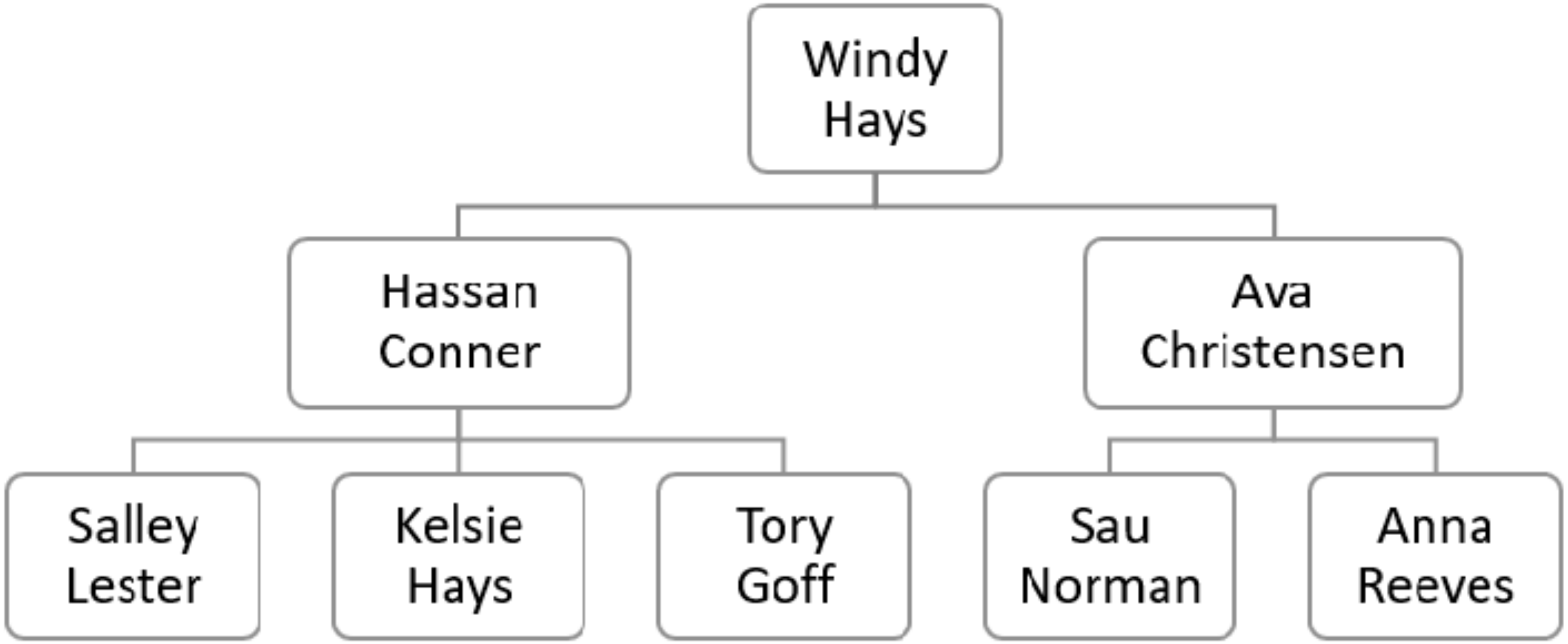
2

Workshop with Joins

https://github.com/up1/workshop-database/blob/main/workshop/joins.md

Self-join !!!

Regular join that joins a table to itself
Query hierarchical data



Workshop with Self-Join

https://github.com/up1/workshop-database/blob/main/workshop/self-join.md

Problems ?

Problems

Slow query

Accurate knowledge is required

Add complexity

Need denormalization ?

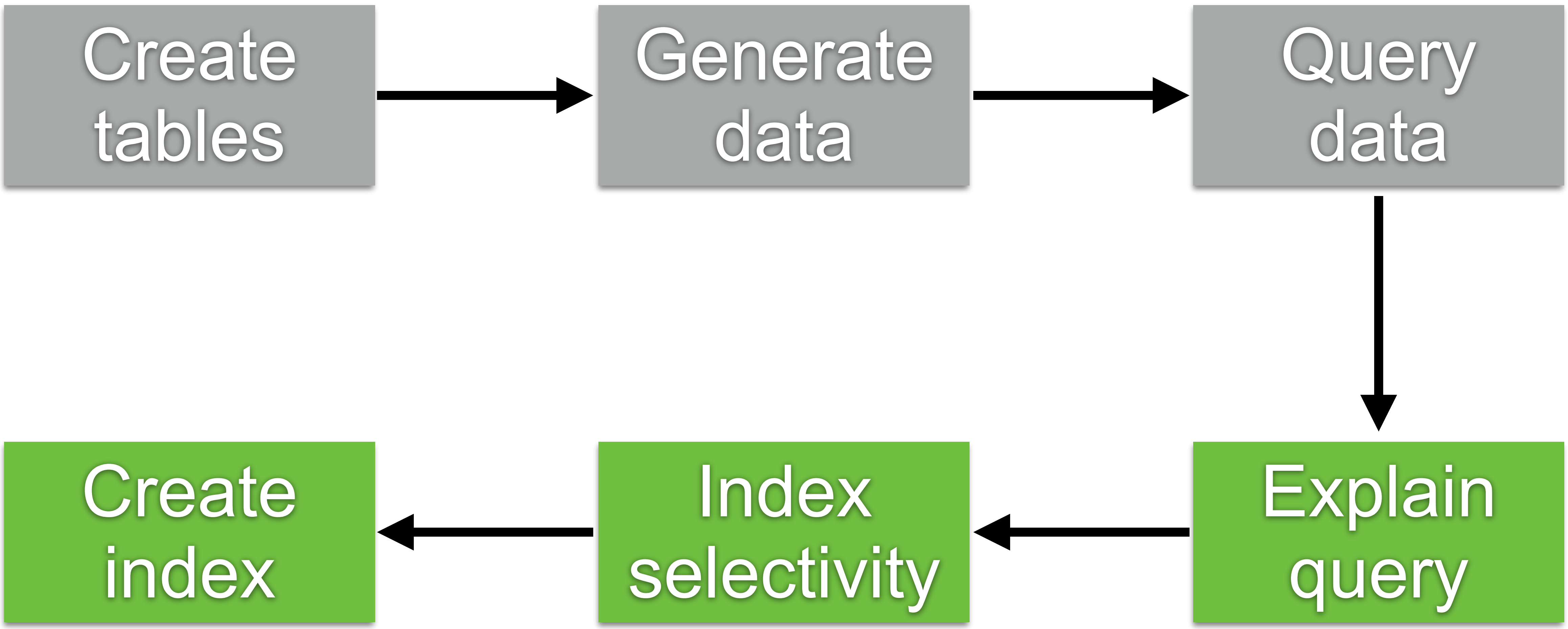
Rewrite query ?

Database vs application caching ?

Improvement for query data

When ? Why ? How ? Who ?

Step in workshop



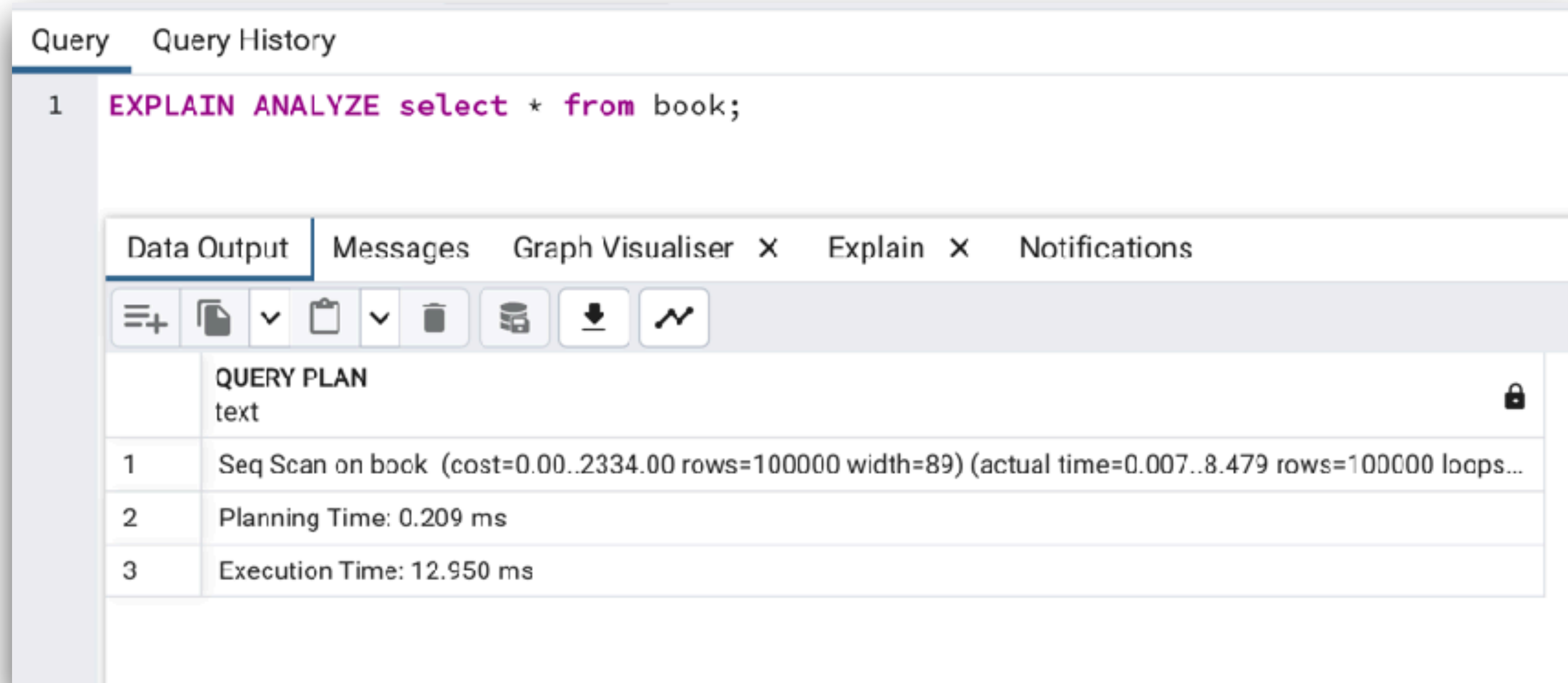
Tips for SQL Optimization

Bad	Good
SELECT *	SELECT column1, column2
SELECT ALL	SELECT with Limit, Offset, Top
Many joins !!	Use appropriate joins
SELECT distinct !! For large datasets	Use group by, union , exists, row_number
%column%	Use full text search feature
count(*) > 0 or IN	exists (select 1 from table WHERE …)
Don’t use subquery	Use derived table or pre-joined tables

Always
Explain Analyze Query !!

Query plan in PostgreSQL

EXPLAIN ANALYZE <SQL>



https://github.com/up1/workshop-database/tree/main/workshop#query-plan

Indexing Strategies

https://www.postgresql.org/docs/current/indexes-types.html

Index

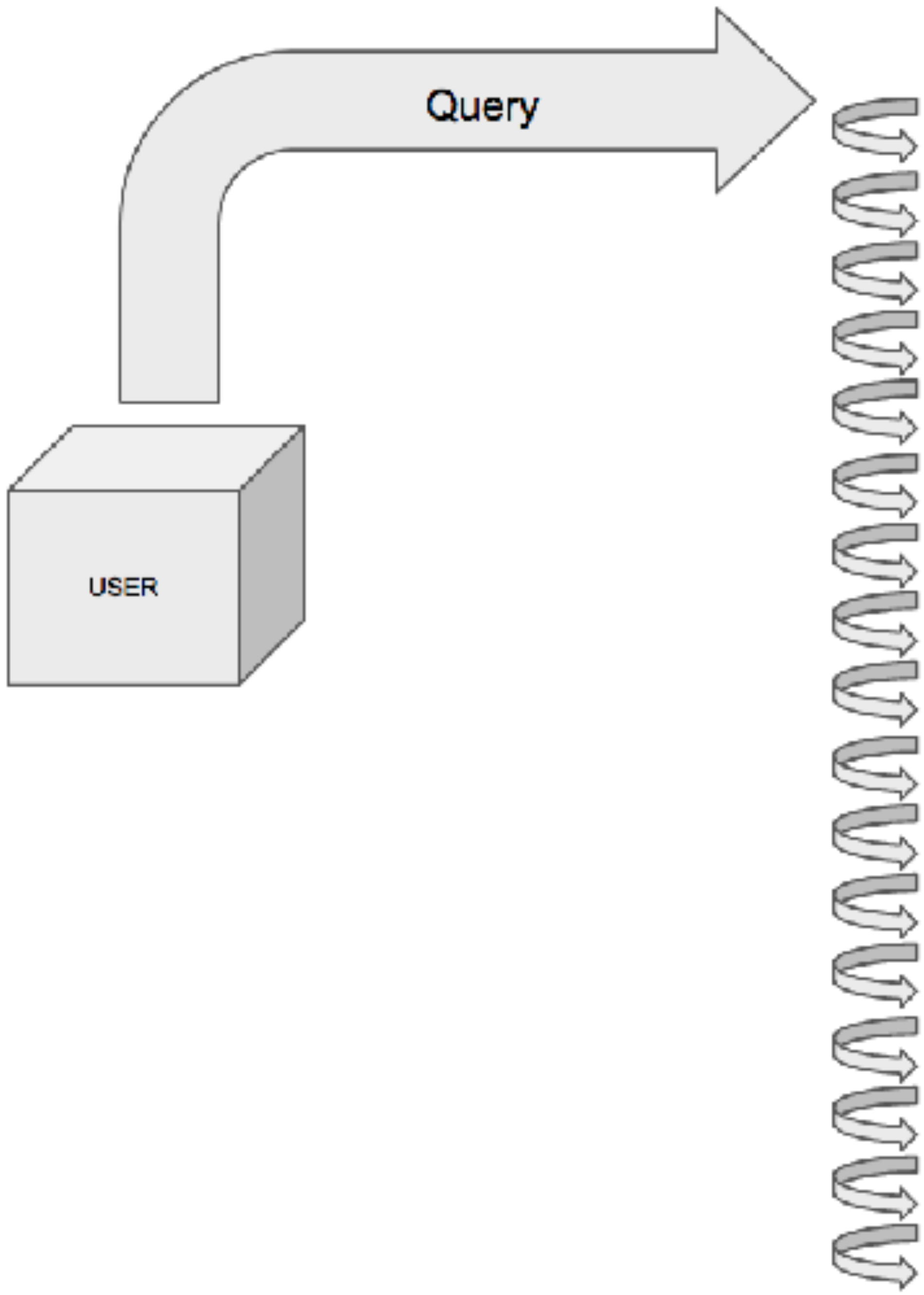
Symbols

A

B

C

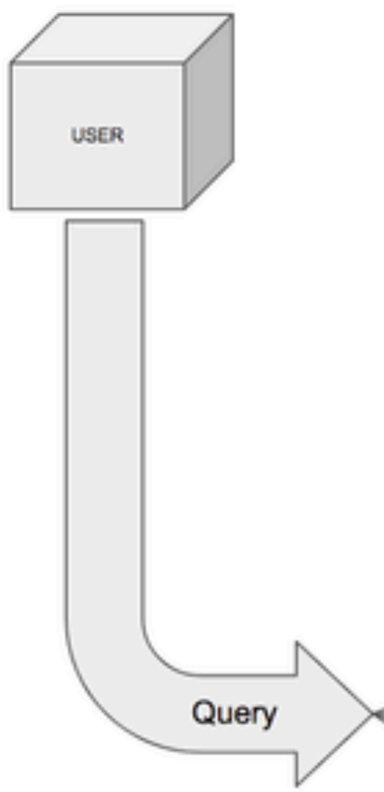
Without Index (full table scan)



Table

company_id	units	unit_cost
10	12	1.15
12	12	1.05
14	18	1.31
18	18	1.34
11	24	1.15
16	12	1.31
10	12	1.15
12	24	1.3
18	6	1.34
18	12	1.35
14	12	1.95
21	18	1.36
12	12	1.05
20	6	1.31
18	18	1.34
11	24	1.15
14	24	1.05

With Index



Index

company_id	pointer
10	_123
10	_129
11	_127
11	_138
12	_124
12	_130
12	_135
14	_125
14	_131
14	_133
16	_128
18	_126
18	_131
18	_132
18	_137
20	_136
21	_134

Table

pointer	company_id	units	unit_cost
_123	10	12	1.15
_124	12	12	1.05
_125	14	18	1.31
_126	18	18	1.34
_127	11	24	1.15
_128	16	12	1.31
_129	10	12	1.15
_130	12	24	1.3
_131	18	6	1.34
_132	18	12	1.35
_133	14	12	1.95
_134	21	18	1.36
_135	12	12	1.05
_136	20	6	1.31
_137	18	18	1.34
_138	11	24	1.15
_139	14	24	1.05

With Index

Table

	Column A	Column B
RowId 1	Some value 1	Some another value 1
RowId 2	Some value 2	Some another value 2
RowId 3	Some value 3	Some another value 3
RowId 4	Some value 1	Some another value 4
RowId 5	Some value 5	Some another value 5
RowId 6	Some value 6	Some another value 6
RowId 7	Some value 1	Some another value 7
RowId 8	Some value 8	Some another value 8

Index

Some value 1	"RowId 1" "RowId 4" "RowId 7"
Some value 2	RowId 2
Some value 3	RowId 3
Some value 5	RowId 5
Some value 6	RowId 6
Some value 8	RowId 8

Index by column "Column A"

Main factors for good index ?

How much space does the index use ?
How fast does it read ?
How much overhead does it add to writes ?
How well does it handle concurrency ?

Quick lookup, fast insert, reliable delete

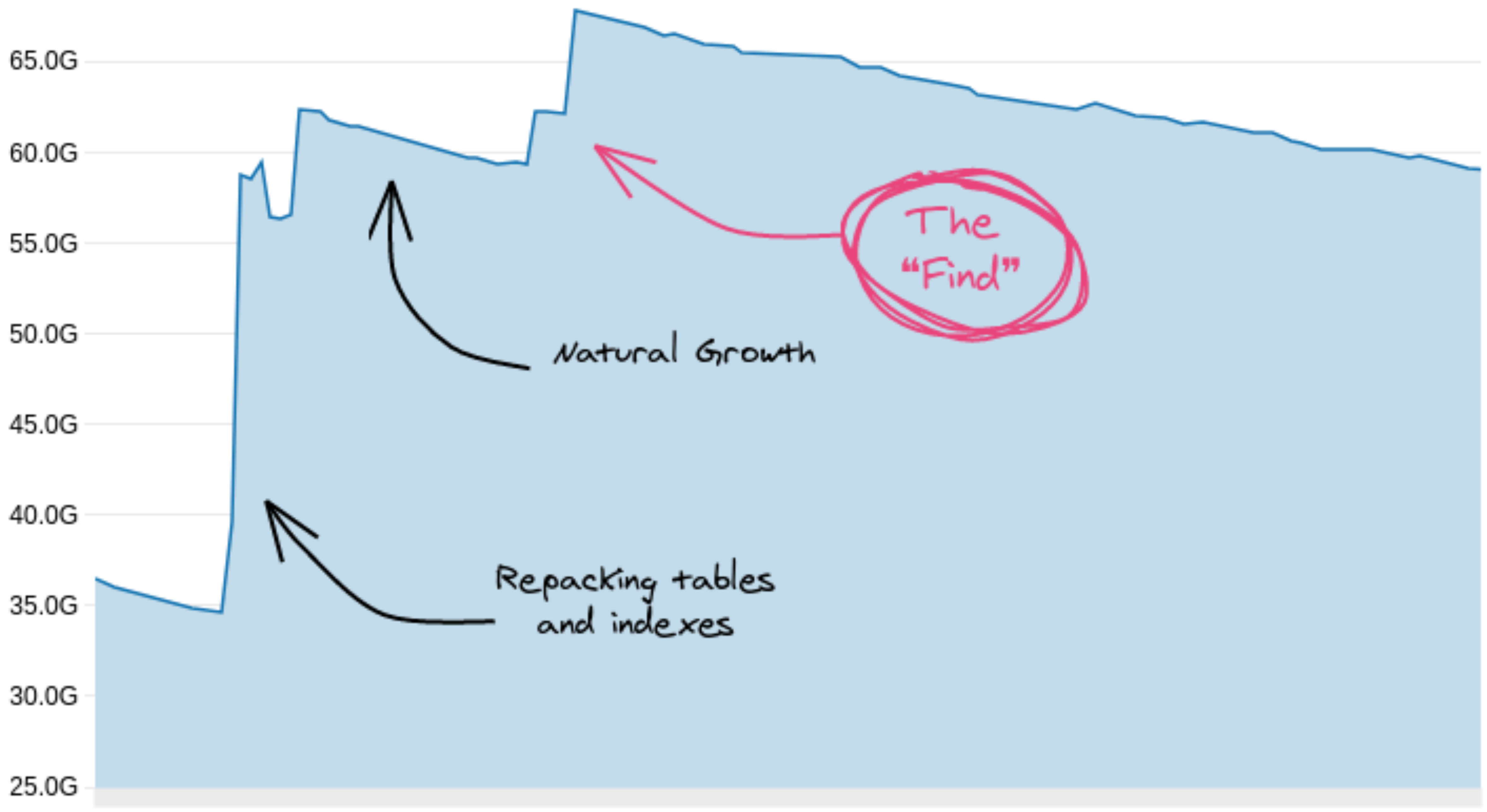
Index selectivity

Factor to select a column to create an index

The number of **distinct values** in the indexed column divided by the number of records in the table is called a selectivity of an index

Prefer indexing columns with selectivity greater than > 0.85 (use Index scan)

Unused Index ?



https://hakibenita.com/postgresql-unused-index-size

Types Index in PostgreSQL

B-Tree

Hash

Block range indexes (BRIN)

Generalized inverted index (GIN)

Generalized inverted search tree index (GiST)

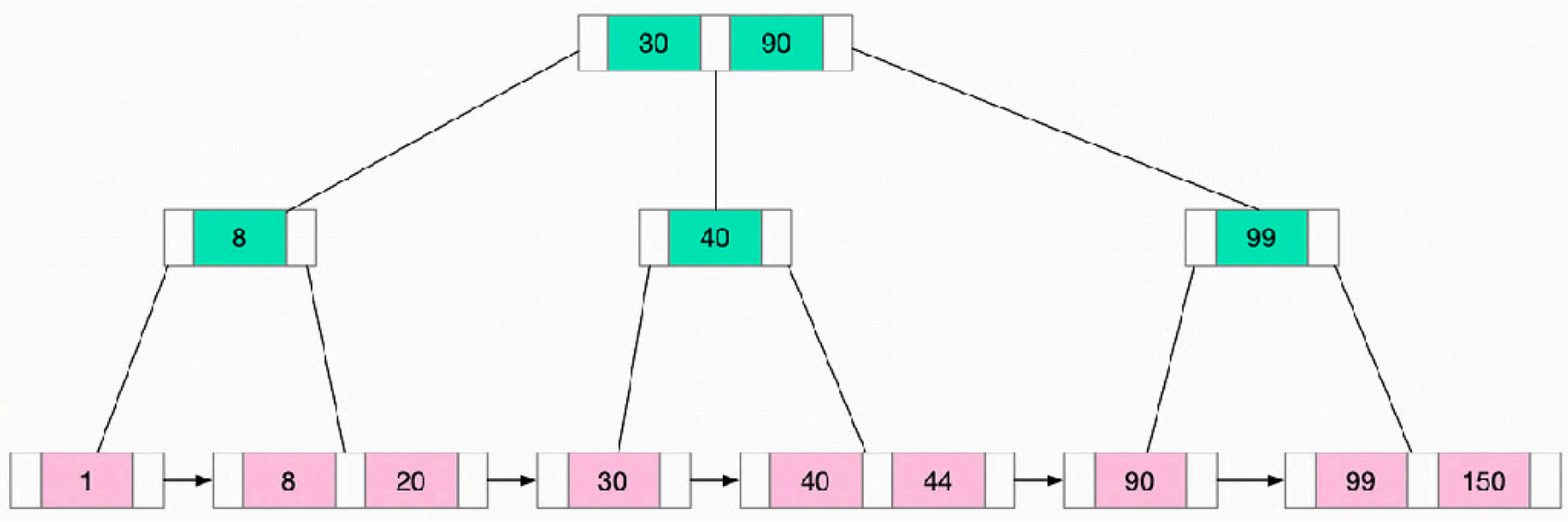
Space partitioned GiST (SP-GiST)

https://www.postgresql.org/docs/current/indexes-types.html

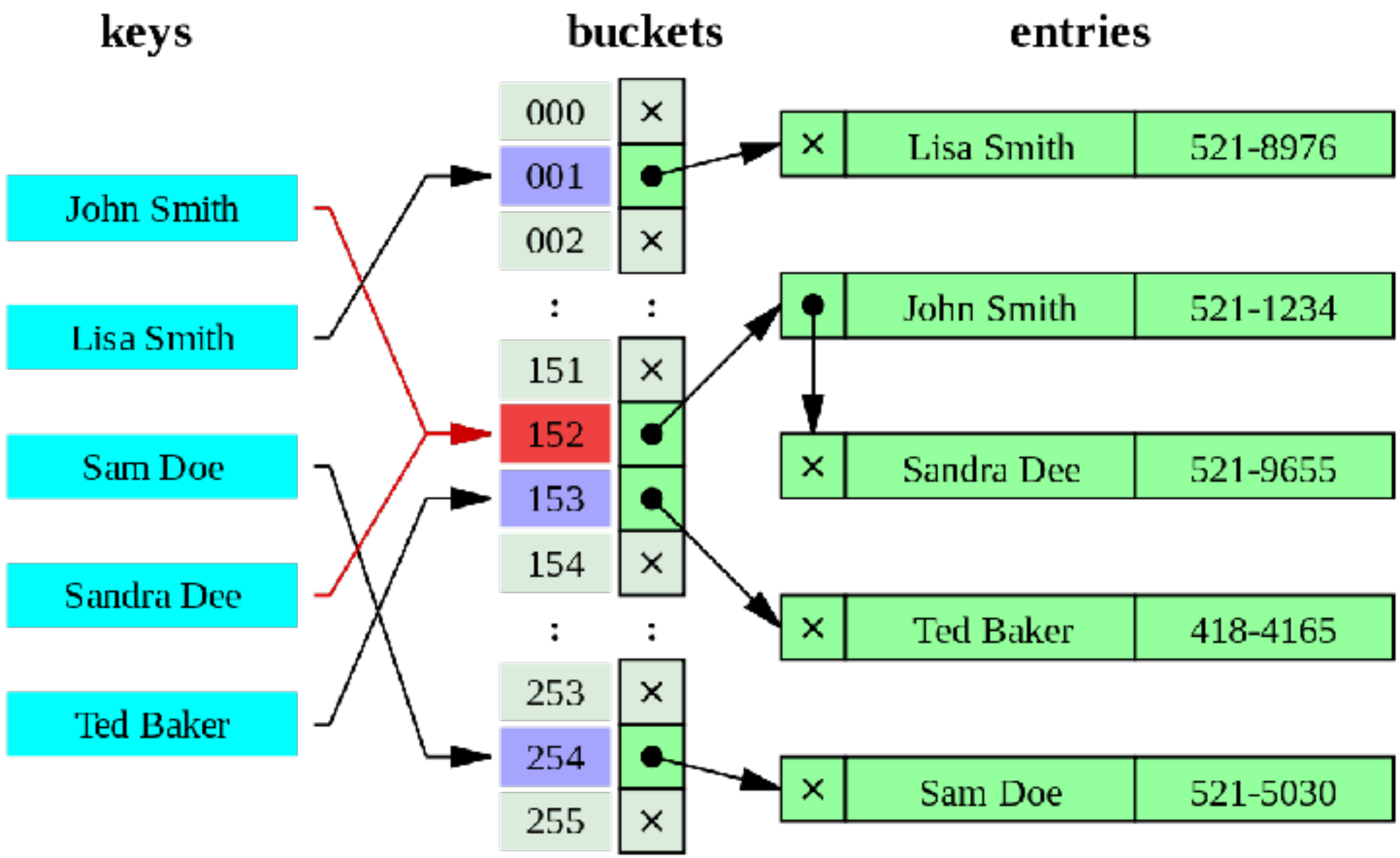
PostgreSQL Index Types

Index type	Data	Target	Use case
B-Tree	High-cardinality scalar columns	Fast equality, range, between operation	PK, FK, timestamp/range scan, sorting
GIN	Composite values (arrays, JSONB, tsvector)	Efficient contains, full-text, array membership	Full text search, JSONB containment, array lookup
GiST	Complex or non-scalar types	Flexible tree, Range and text	PostGIS spatial queries, trigram similarity, range types
BRIN	Very large data, ordered table	Lightweight index for append-only and range scan	Time-series data Log data
Bloom	Many low-cardinality columns	Compact multi-column equality filters	Filter data, check data existing

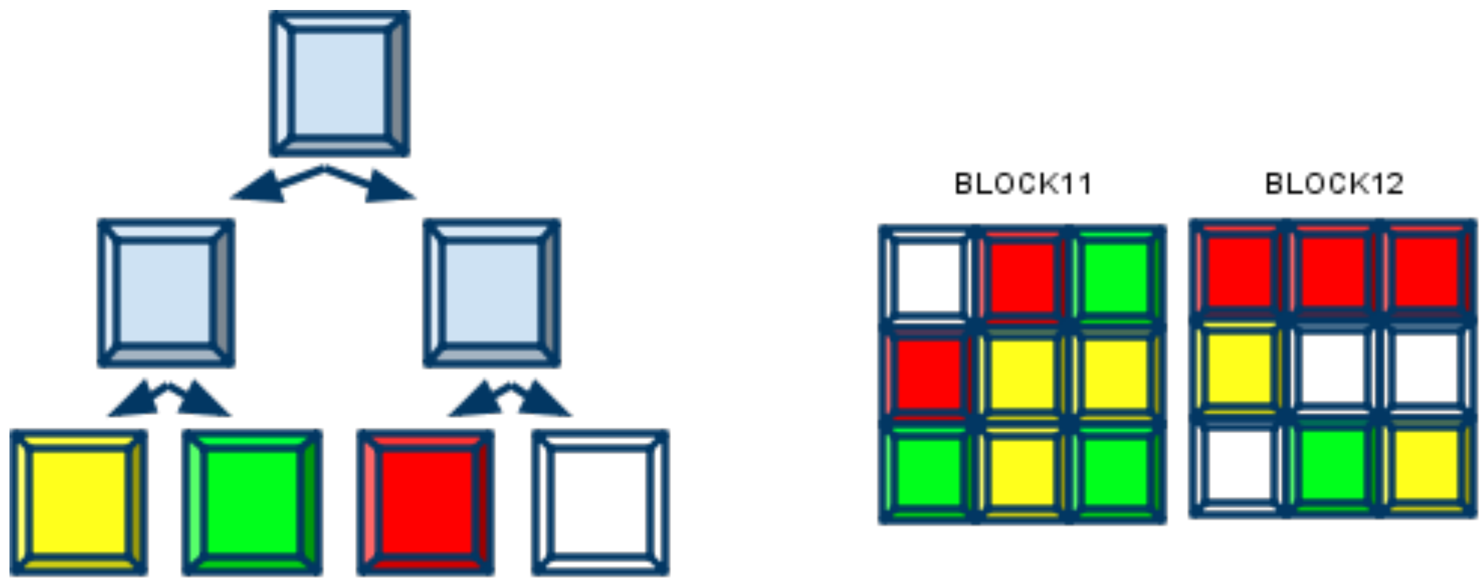
B-Tree +



Hash

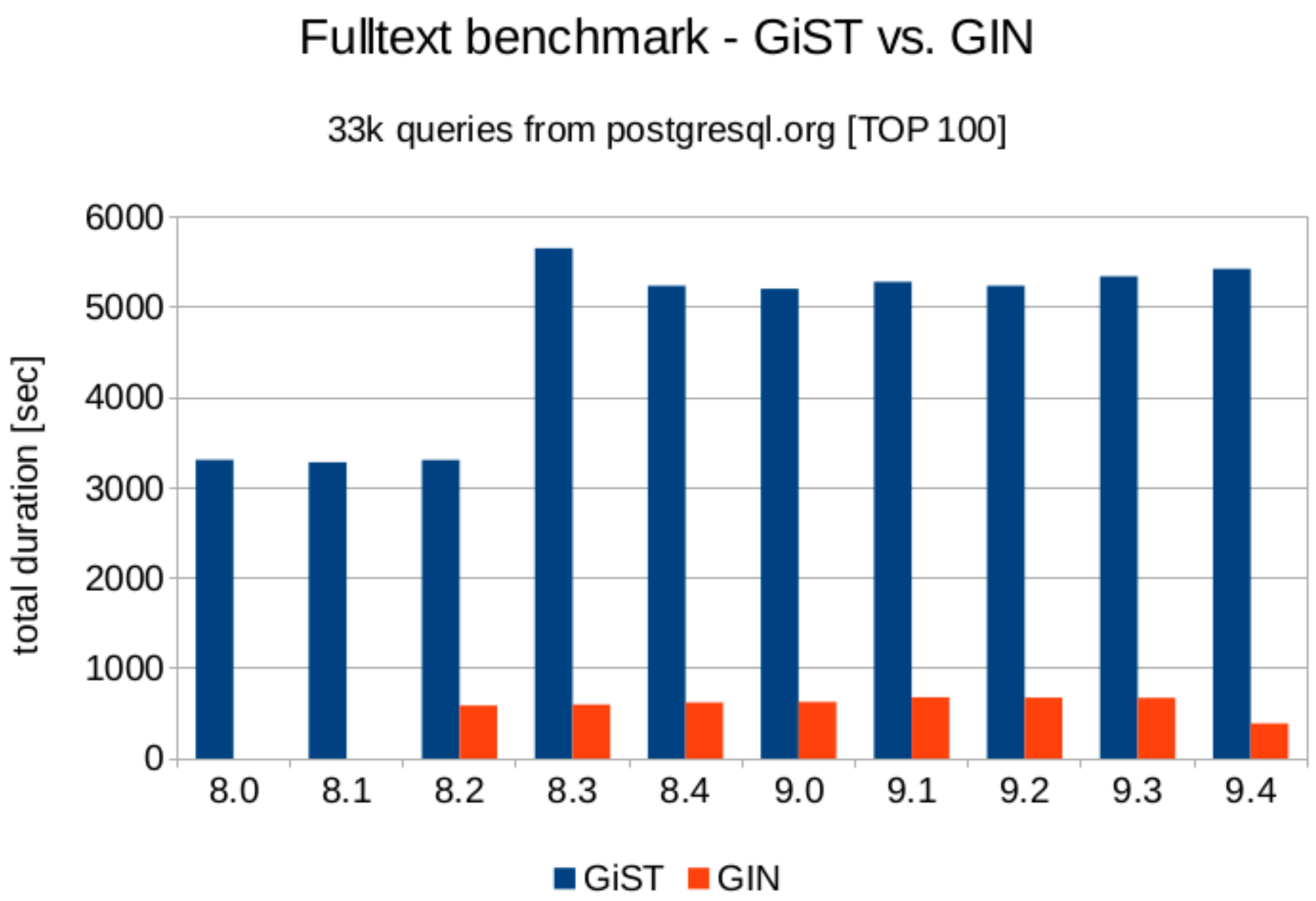

keys
buckets
entries
John Smith
Lisa Smith
Sam Doe
Sandra Dee
Ted Baker
000
001
002
:
:
151
152
153
154
:
:
253
254
255
Lisa Smith
521-8976
John Smith
521-1234
Sandra Dee
521-9655
Ted Baker
418-4165
Sam Doe
521-5030

Bitmap



KEY	Start Row ID	End Row ID	BATMAP
YELLOW	11.0	12.9	10000
GREEN	11.0	12.9	01000
RED	11.0	12.9	00100
null	11.0	12.9	00010

GIN vs GiST


Fulltext benchmark - GiST vs. GIN
33k queries from postgresql.org [TOP 100]
total duration [sec]
6000
5000
4000
3000
2000
1000
0
8.0
8.1
8.2
8.3
8.4
9.0
9.1
9.2
9.3
9.4
GiST
GIN

Tuning and Use Index ?

Size vs Speed

Extension setup

Combine indexes

Measure impact with **EXPLAIN** (before and after)

Don’t over-index
Unused index

Size of index ?

Workshop with Indexing

https://github.com/up1/workshop-database/tree/main/workshop/basic-workshop

Tuning Performance

Tuning performance

Normalization vs Denormalization

Partition

Sharding

Database architecture

Database parameteres

Partition

https://hakibenita.com/postgresql-unused-index-size

Partition

Table partitioning refers to **splitting** what is logically one large table into **smaller physical pieces**

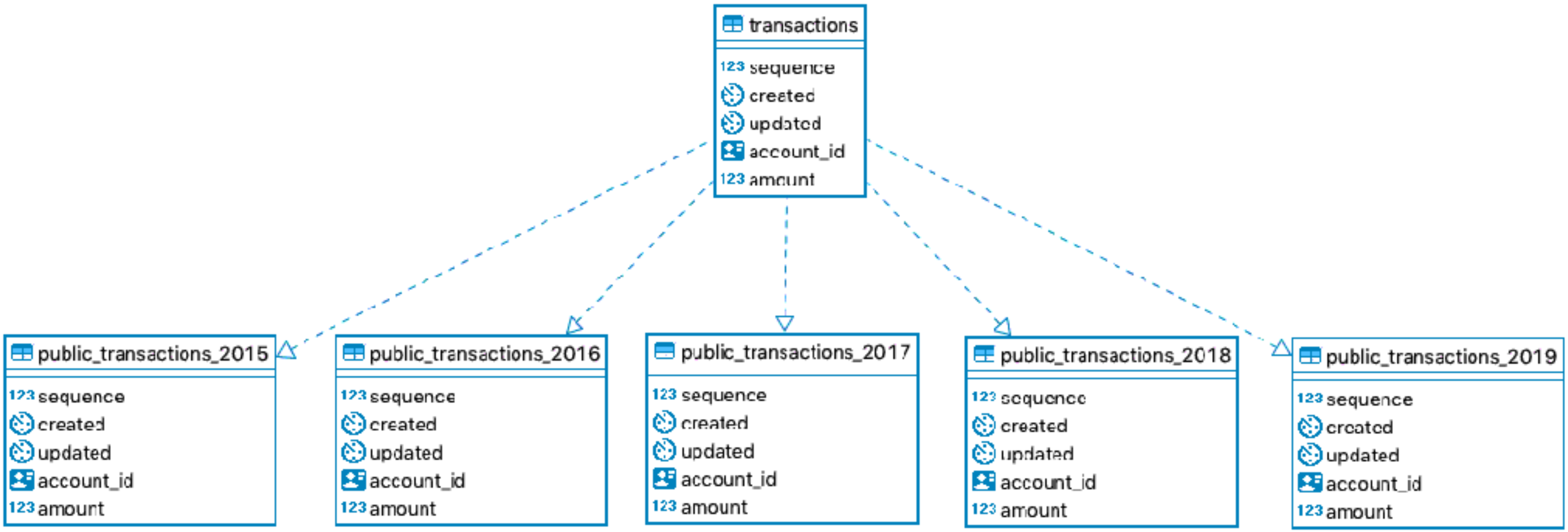
Reduce index size

Partition

Horizontal partitioning
Vertical partitioning

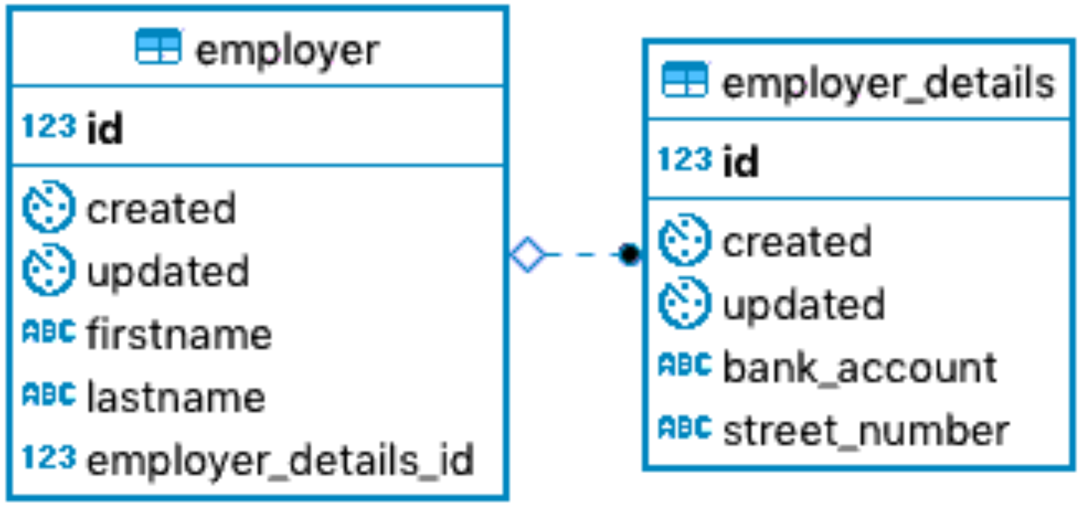
Horizontal partitioning

Putting different rows into different table



Vertical partitioning

Horizontal partitioning

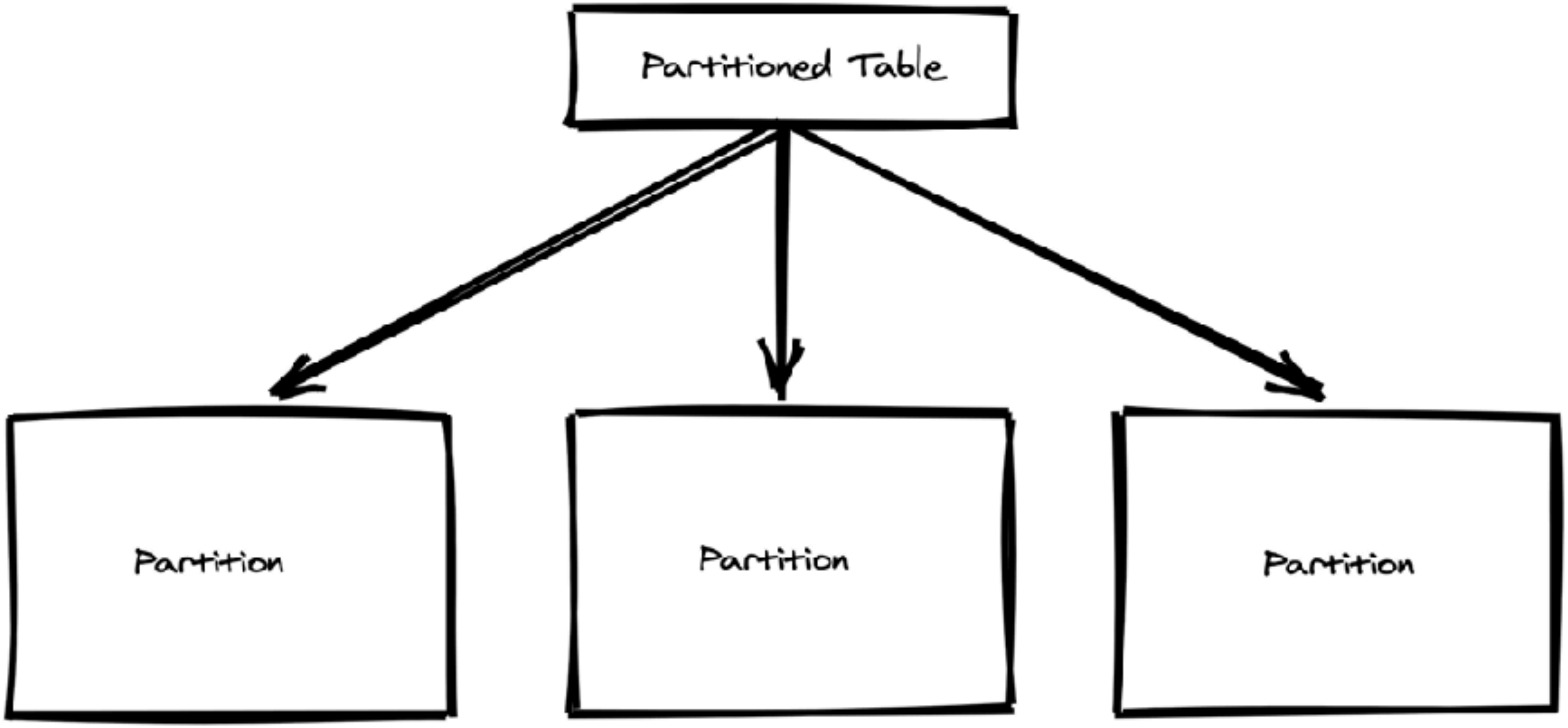


Types Horizontal partitioning

Range
List

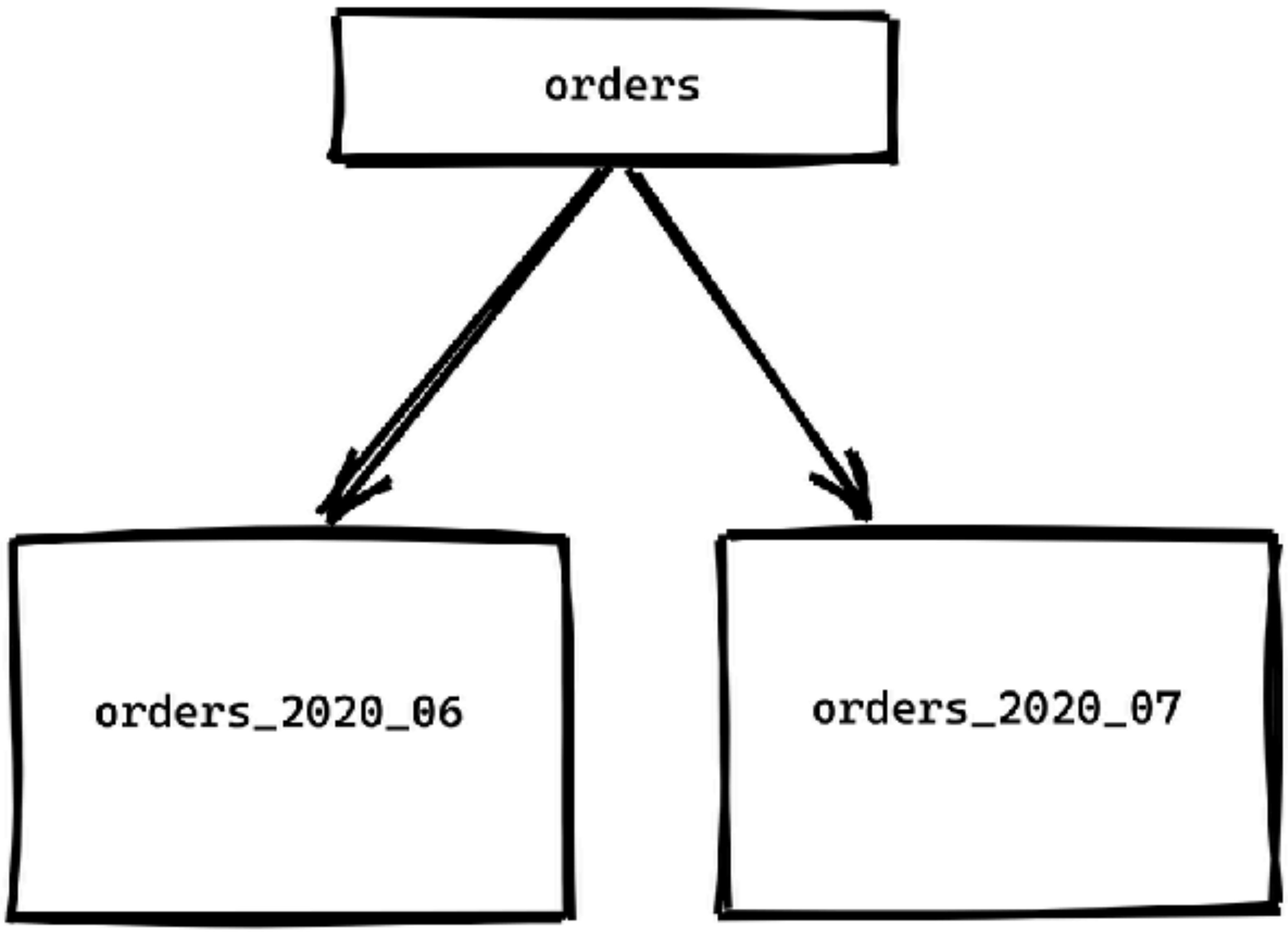
Example

Partition tables



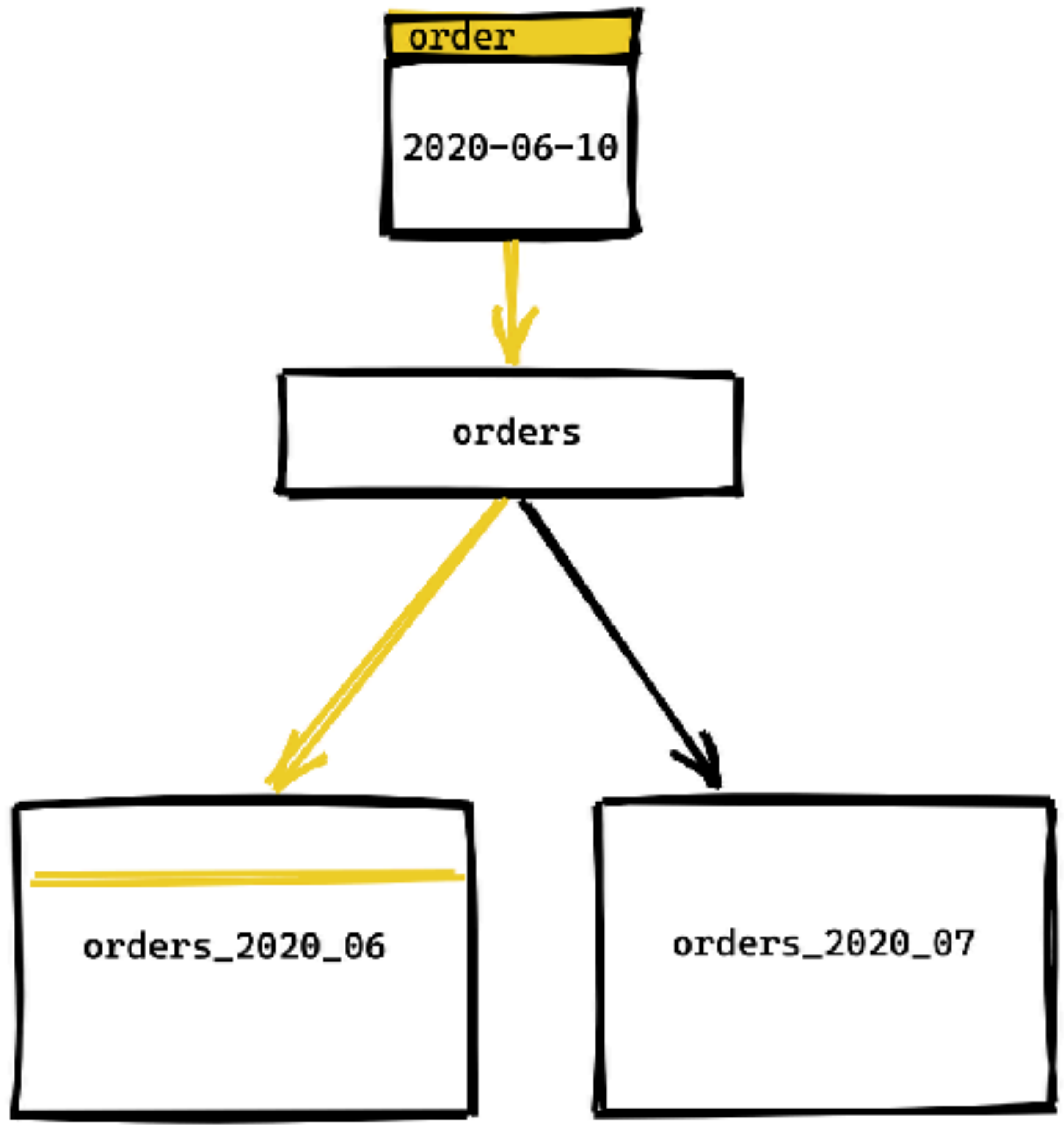
https://chriserwin.com/table-partitioning/

Orders table



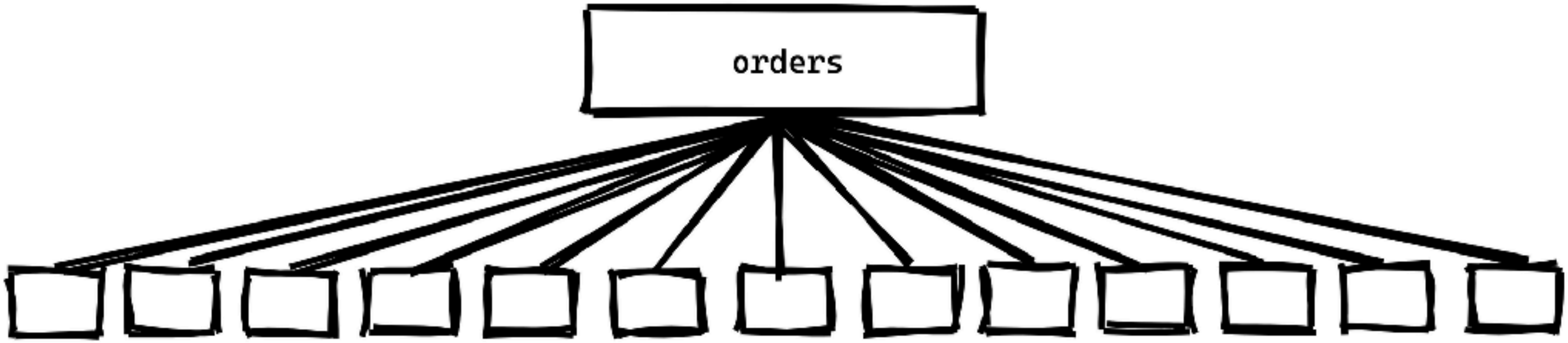
https://chriserwin.com/table-partitioning/

Insert data into orders



https://chriserwin.com/table-partitioning/

...



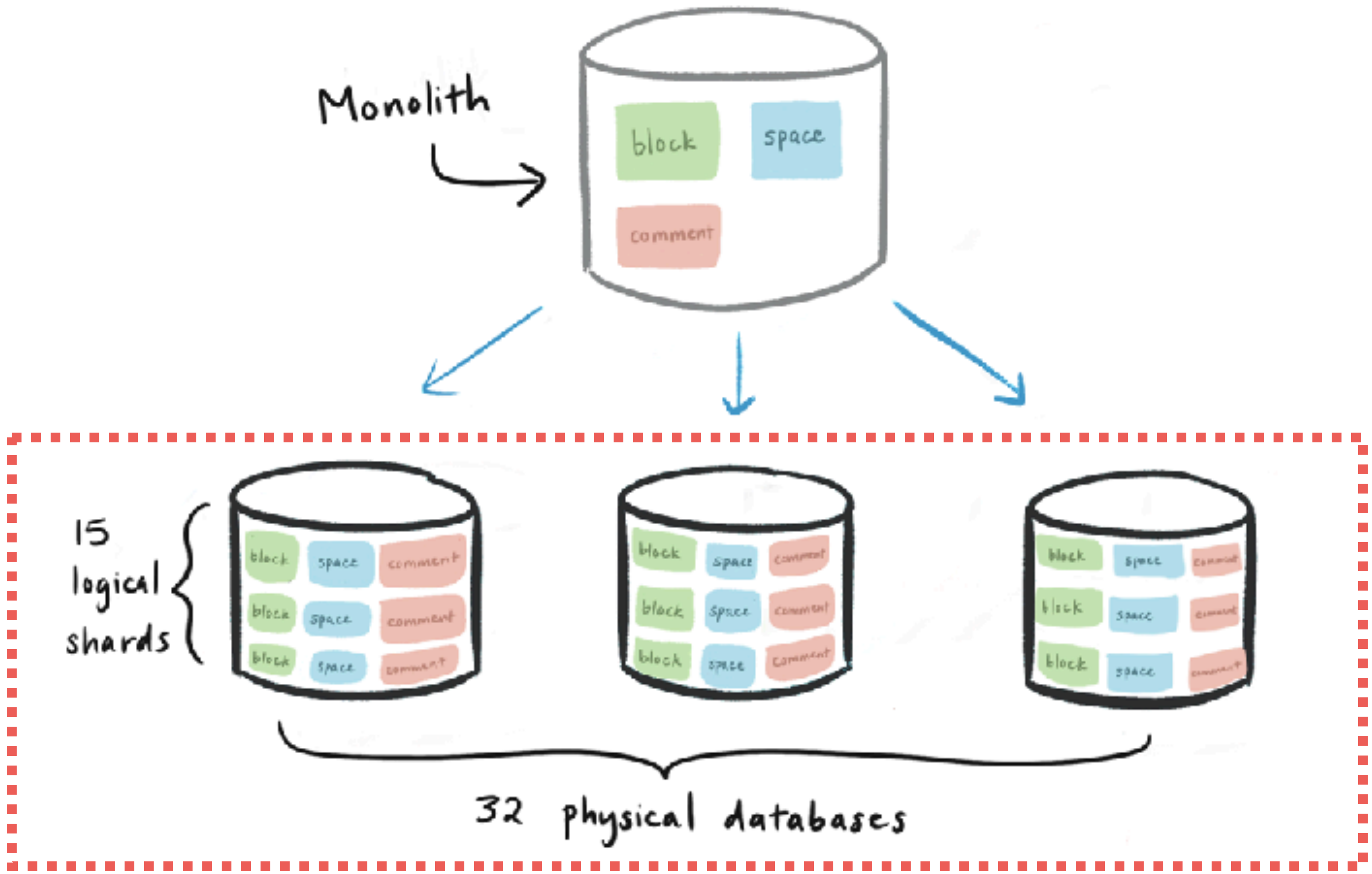
https://chriserwin.com/table-partitioning/

Workshop with Partition

https://github.com/up1/workshop-database/blob/main/workshop/basic-workshop/partition.md

Sharding

Sharding (Physical server)



https://www.notion.so/blog/sharding-postgres-at-notion

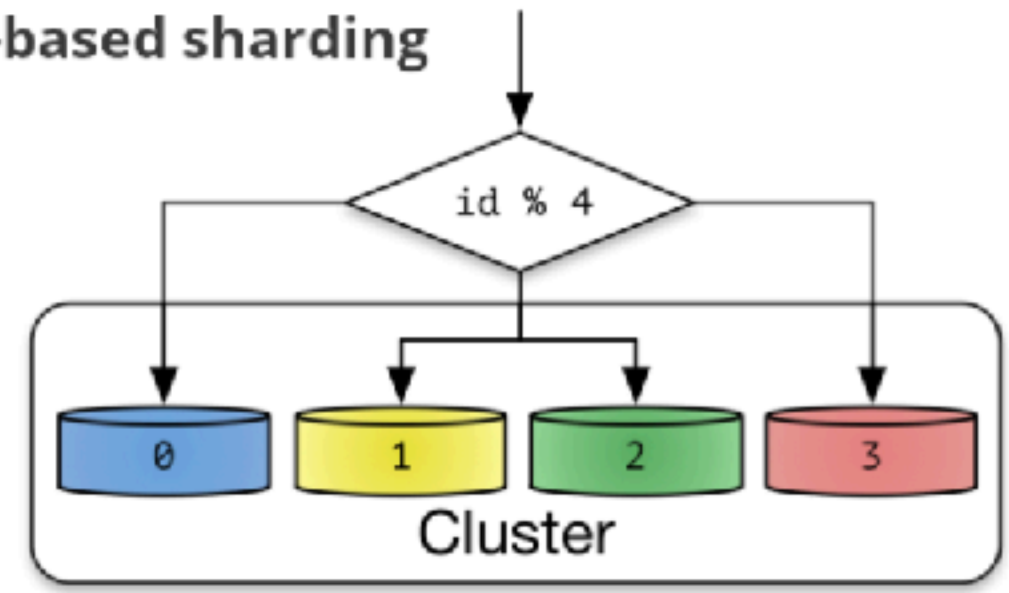
Sharding techniques

Hash
Range
Location
Key/Directory

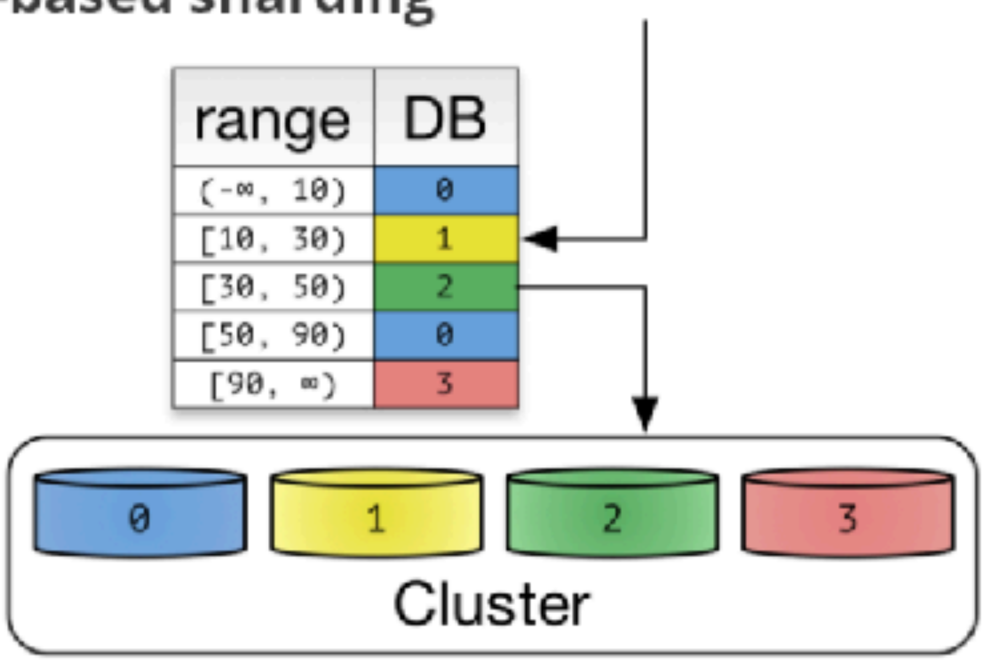
https://www.somkiat.cc/introduction-database-sharding/

Sharding

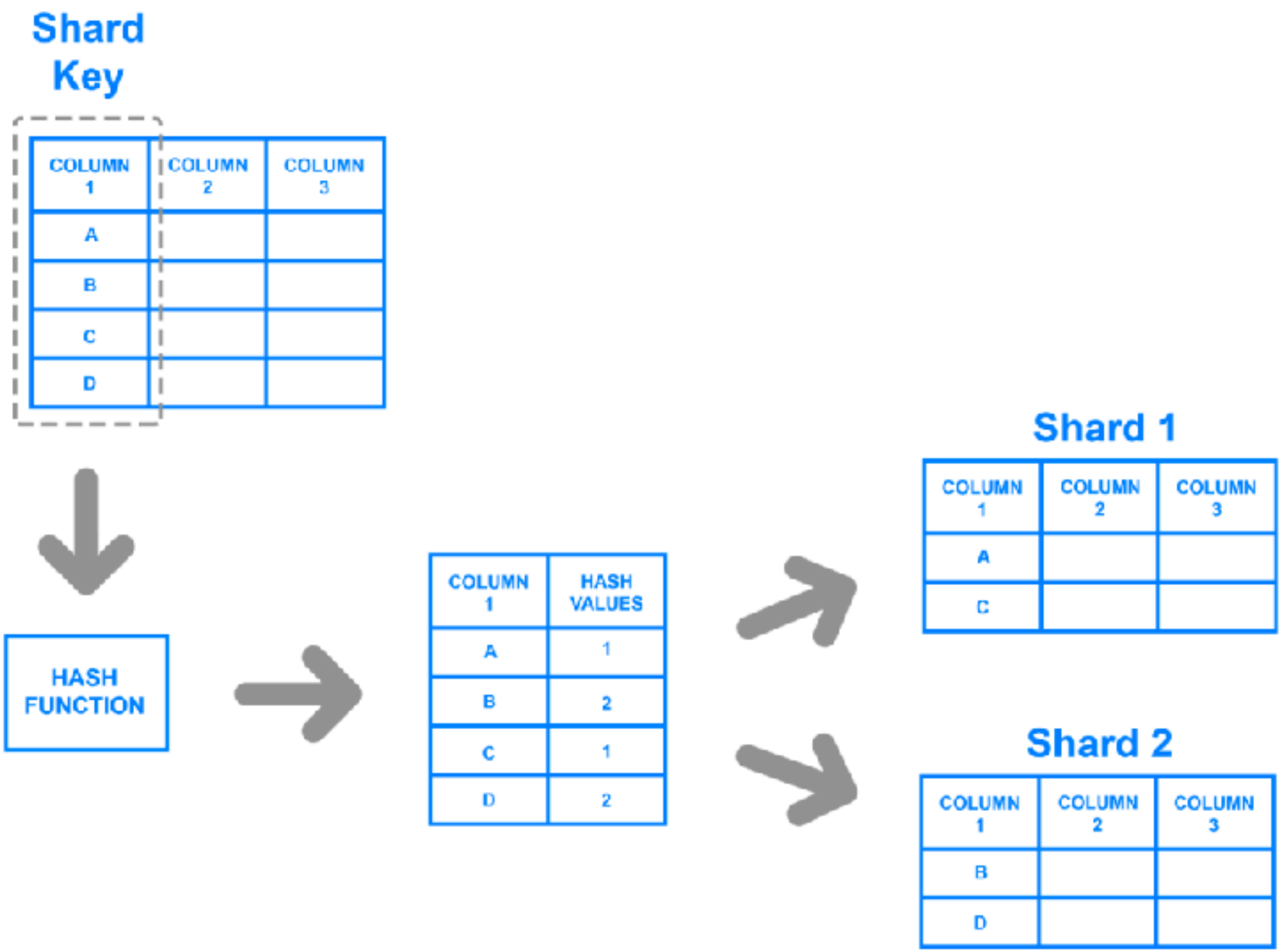
(a) Hash-based sharding



(b) Range-based sharding



Shard Key



Shard Key
COLUMN 1
COLUMN 2
COLUMN 3
A
B
C
D
HASH FUNCTION
COLUMN 1
HASH VALUES
A
1
B
2
C
1
D
2
Shard 1
COLUMN 1
COLUMN 2
COLUMN 3
A
C
Shard 2
COLUMN 1
COLUMN 2
COLUMN 3
B
D

Range

PRODUCT	PRICE
WIDGET	$118
GIZMO	$88
TRINKET	$37
THINGAMAJIG	$18
DOODAD	$60
TCHOTCHKE	$999

($0-$49.99)

PRODUCT	PRICE
TRINKET	$37
THINGAMAJIG	$18

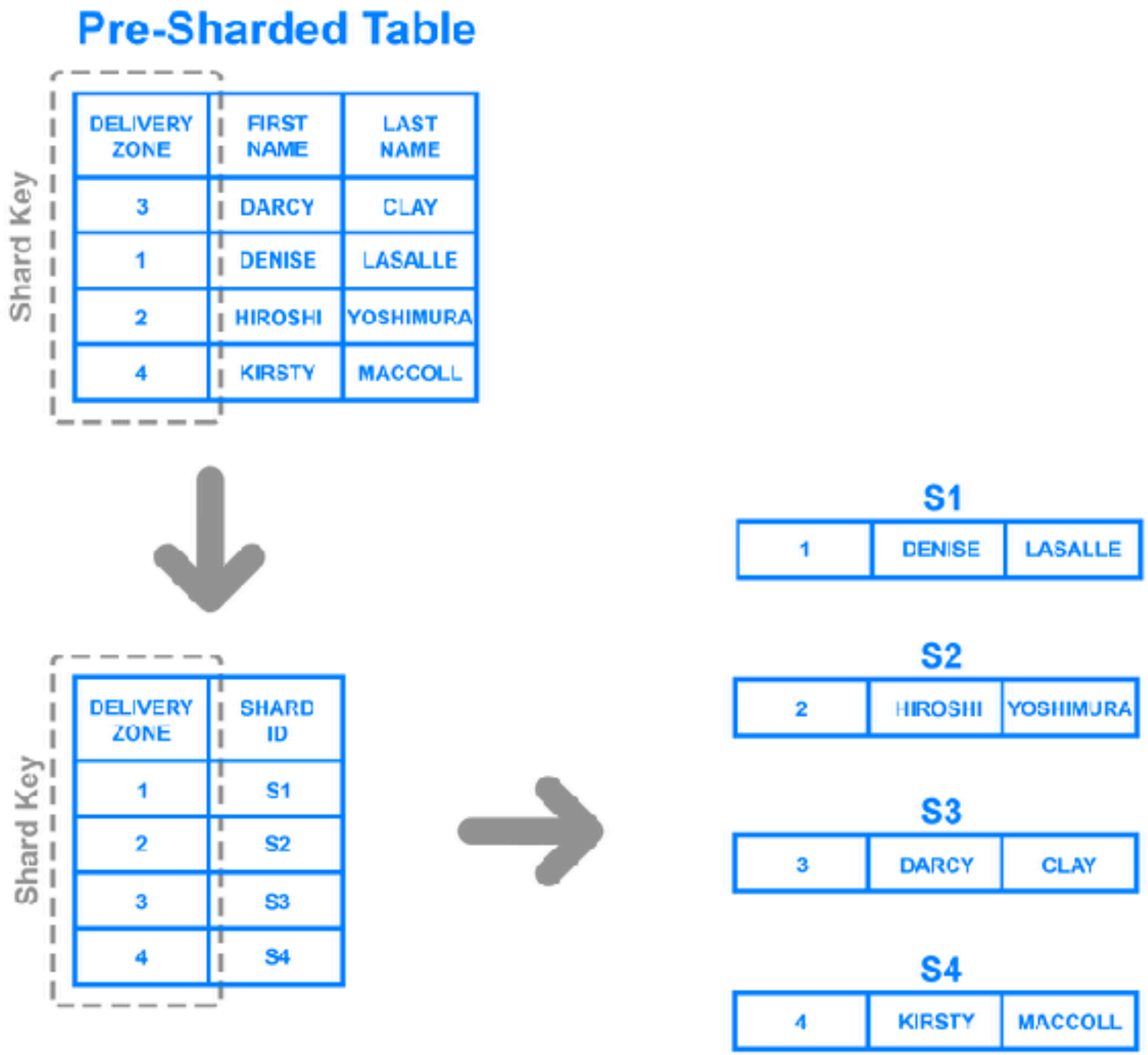
($50-$99.99)

PRODUCT	PRICE
GIZMO	$88
DOODAD	$60

($100+)

PRODUCT	PRICE
WIDGET	$118
TCHOTCHKE	$999

Directory


Pre-Sharded Table
Shard Key
DELIVERY ZONE
FIRST NAME
LAST NAME
3
DARCY
CLAY
1
DENISE
LASALLE
2
HIROSHI
YOSHIMURA
4
KIRSTY
MACCOLL
Shard Key
DELIVERY ZONE
SHARD ID
1
S1
2
S2
3
S3
4
S4
S1
1
DENISE
LASALLE
S2
2
HIROSHI
YOSHIMURA
S3
3
DARCY
CLAY
S4
4
KIRSTY
MACCOLL

Sharding

Shard 1

Stores

```
id  |    name
----+---------------
1   | my book store
5   | my other store
```

Products

```
id  |  name   | store_id
----+---------+------------
1   | foo     |      1
2   | bar     |      1
3   | baz     |      1
```

Purchases

```
id  |  product_id | store_id | price
----+-------------+----------+------
1   |      2      |    1     | 1000
2   |      1      |    1     | 1200
3   |      3      |    1     | 1199
```

Shard 2

Stores

```
id  |    name
----+--------------
2   | my sock store
6   | old things
```

Products

```
id  |    name    | store_id
----+------------+------------
33  | new socks  |      2
34  | old socks  |      6
35  | old tie    |      6
```

Purchases

```
id  |  product_id | store_id | price
----+-------------+----------+------
102 |      35     |    6     | 600
43  |      33     |    2     | 800
```

https://www.citusdata.com/blog/2016/08/10/sharding-for-a-multi-tenant-app-with-postgres/

Challenge of Large datasets

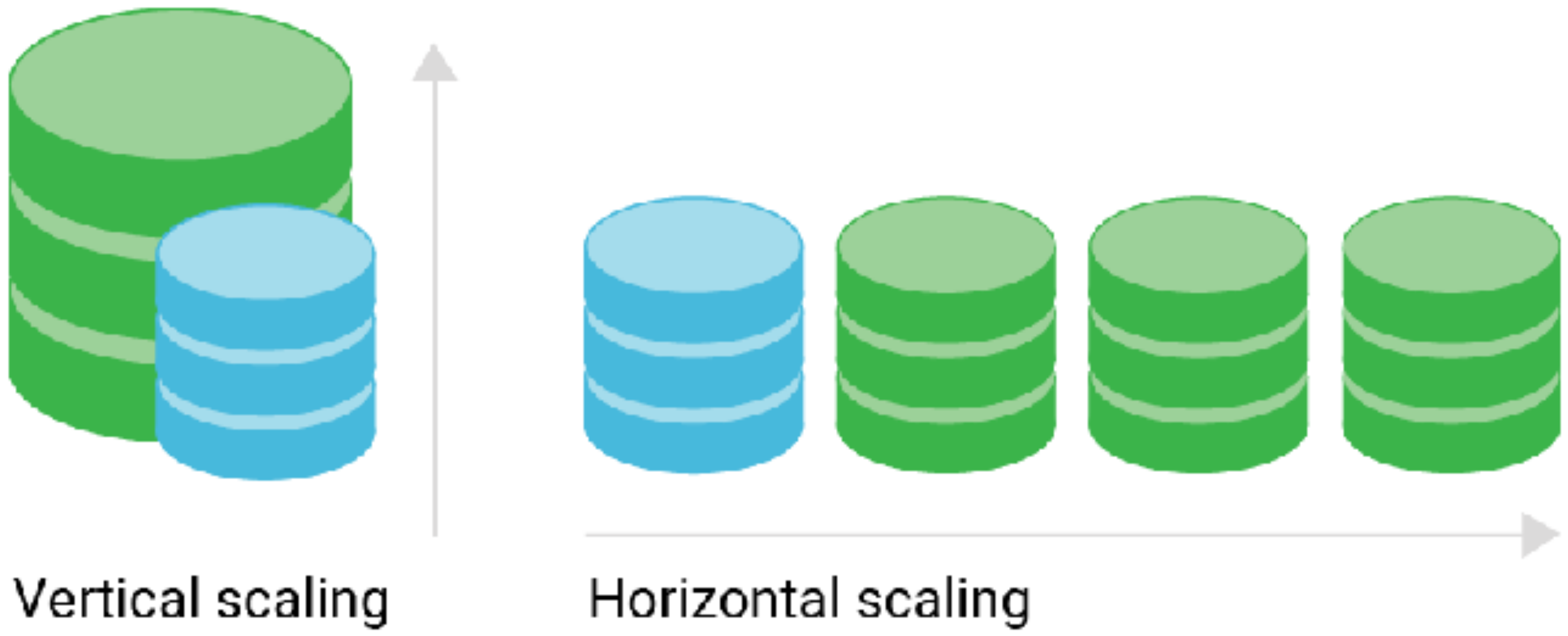
Challenge of Large datasets

Disk I/O, CPU, Memory pressure
Index size and maintenance
Locking and concurrency

Why database slow down with more data ?

Scaling Database ?

Vertical vs Horizontal

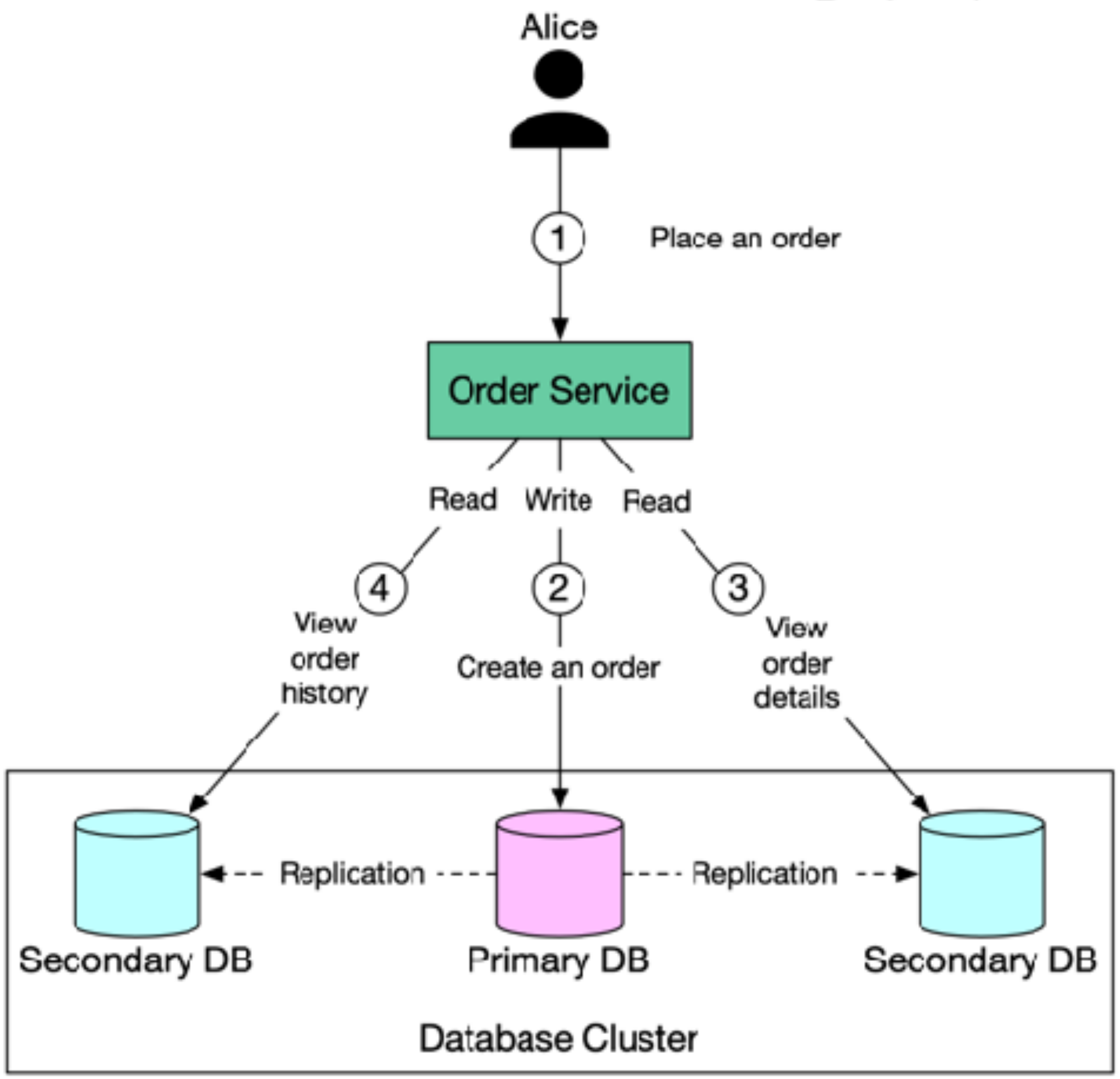


Scalability metrics

Throughput (transactions per second)
Latency (response time)
Availability (uptime)
Durability (data safety)

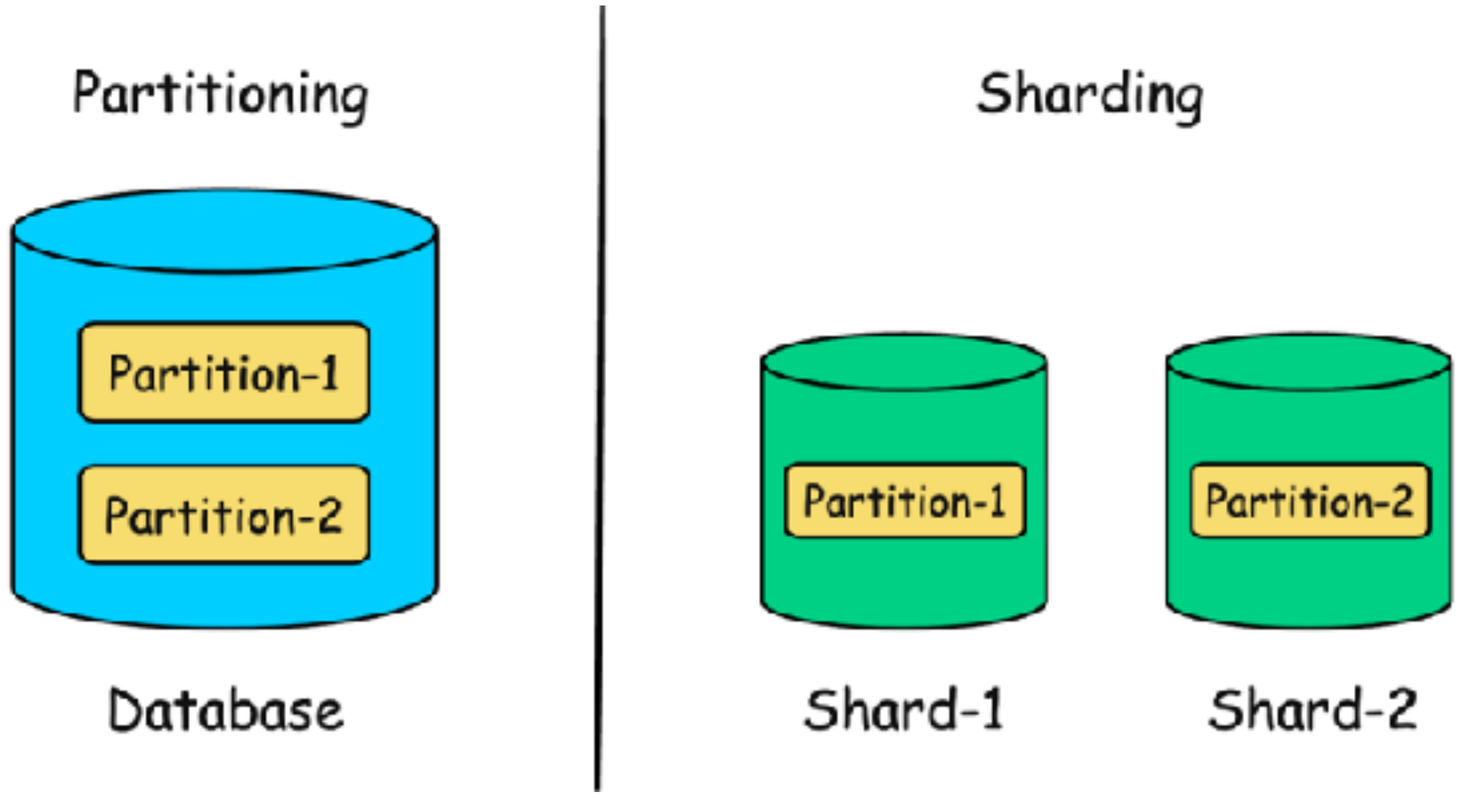
Read replicas

Read-scaling solution



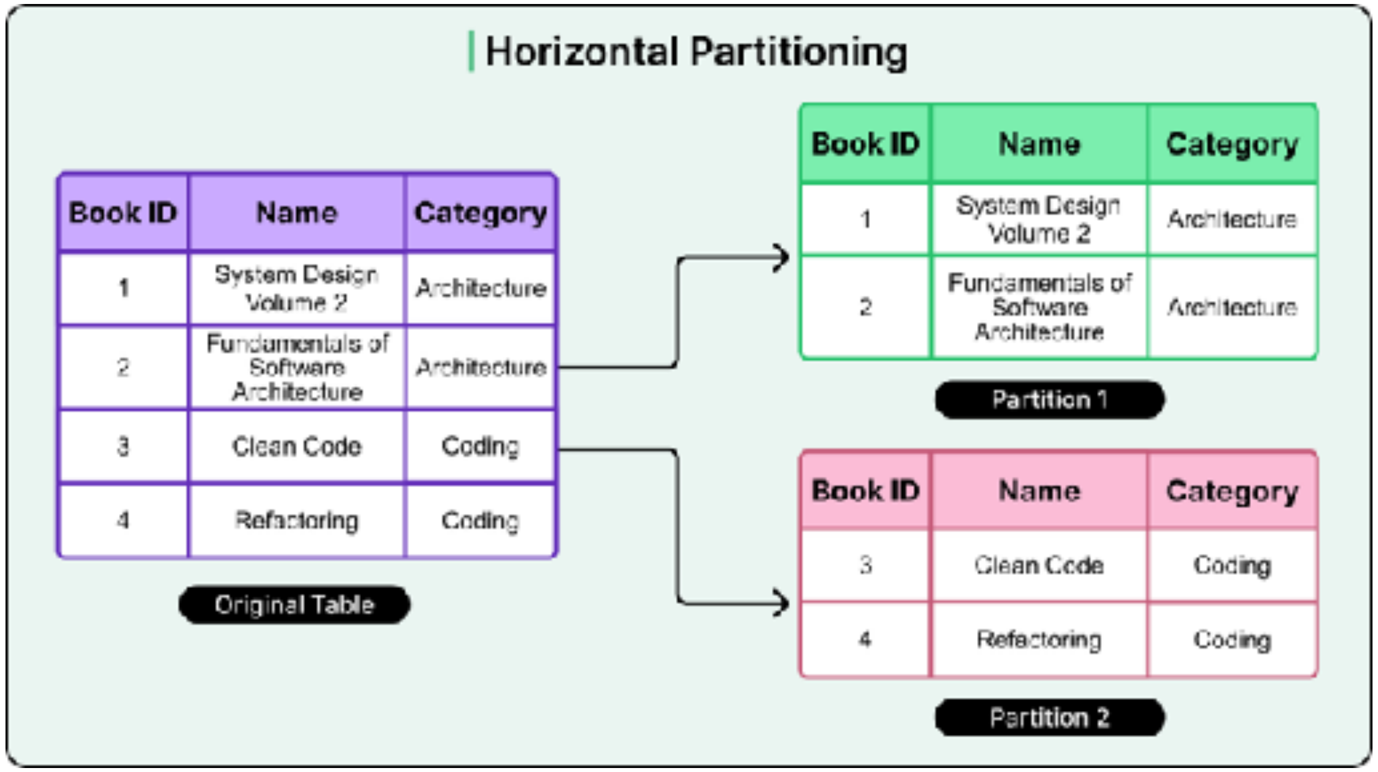
Data distribution strategies

Partitioning (within a single instance)
Sharding (across multiple instances)



Database sharding

Design for large dataset by split into smaller
Divided across multiple instance
Based-on horizontal partitioning
Use when to overcome single instance limitations

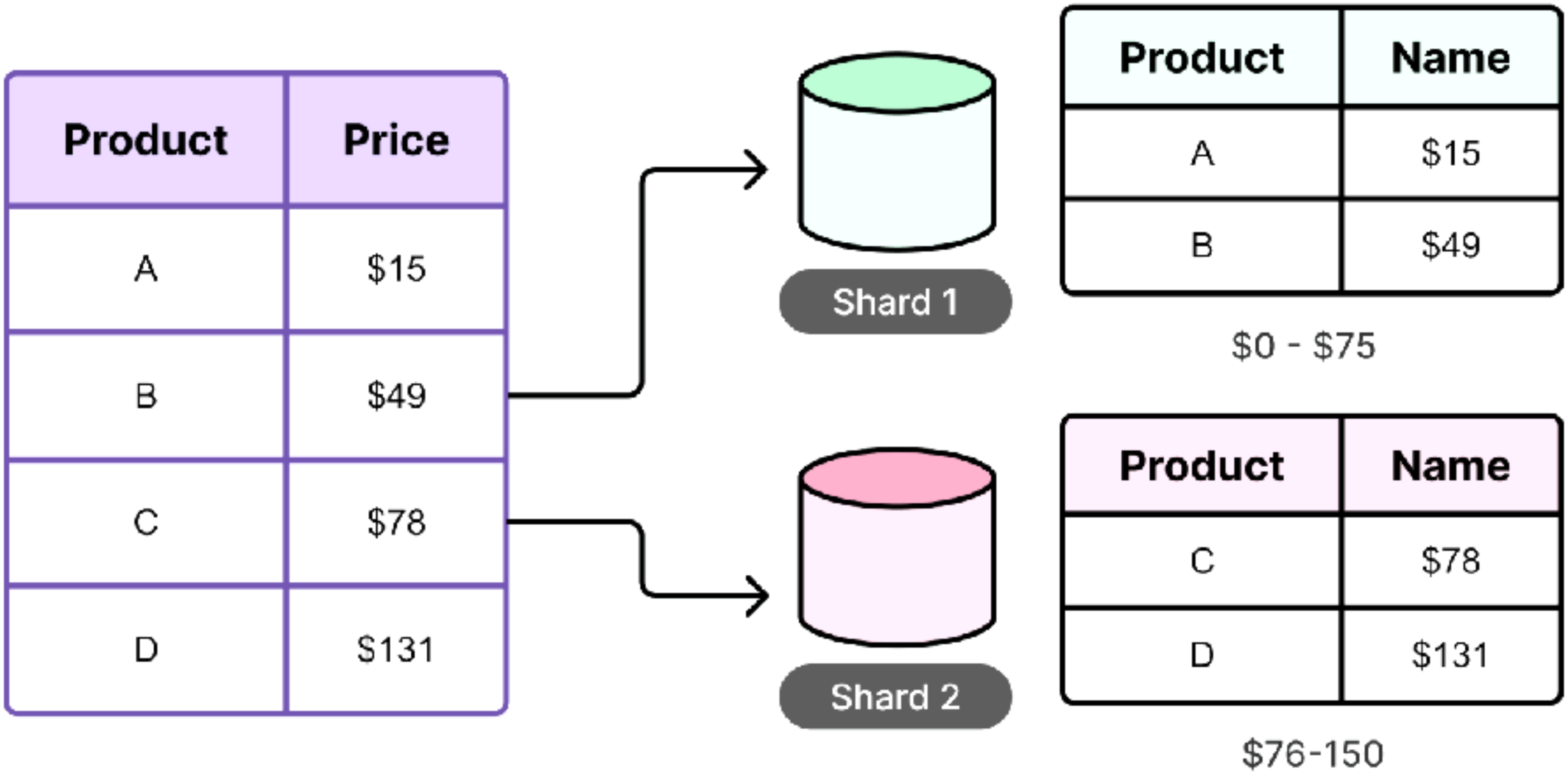


Sharding techniques

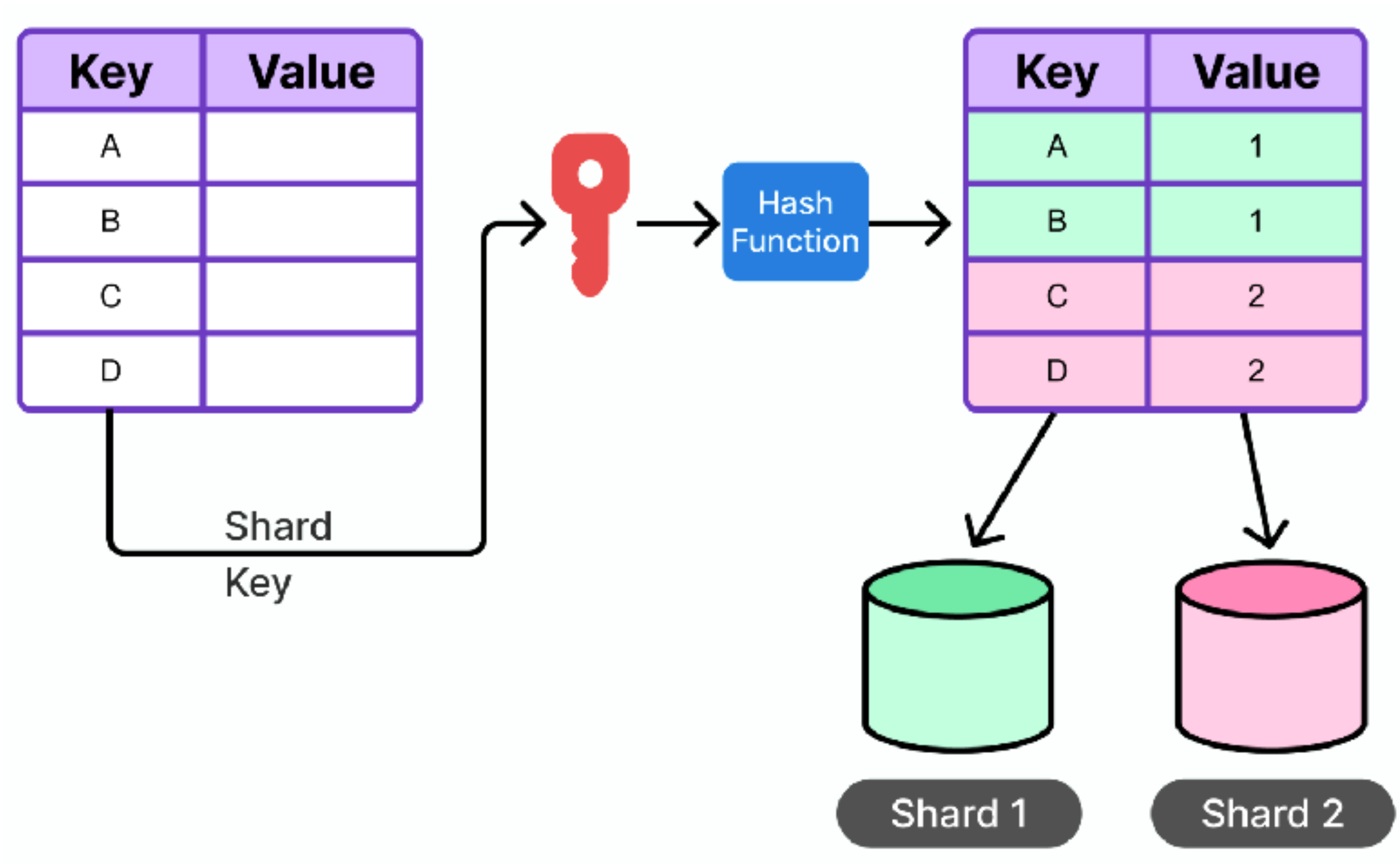
Range
Hash or key-based
Directory-based
Request routing

https://blog.bytebytego.com/p/a-guide-to-database-sharding-key

Range-based sharding



Key-based sharding

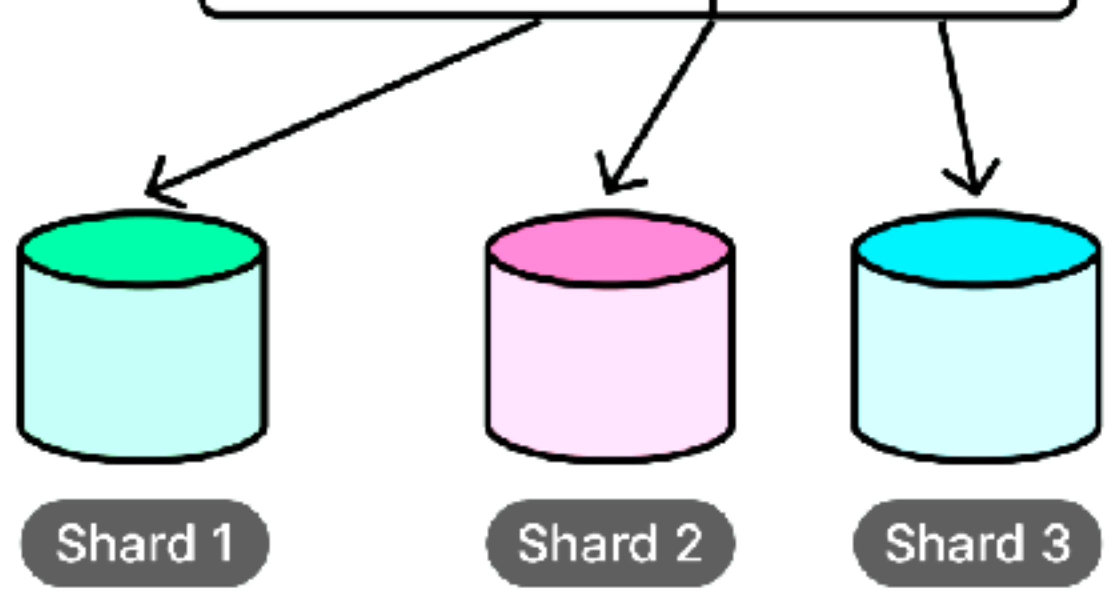


Directory-based sharding

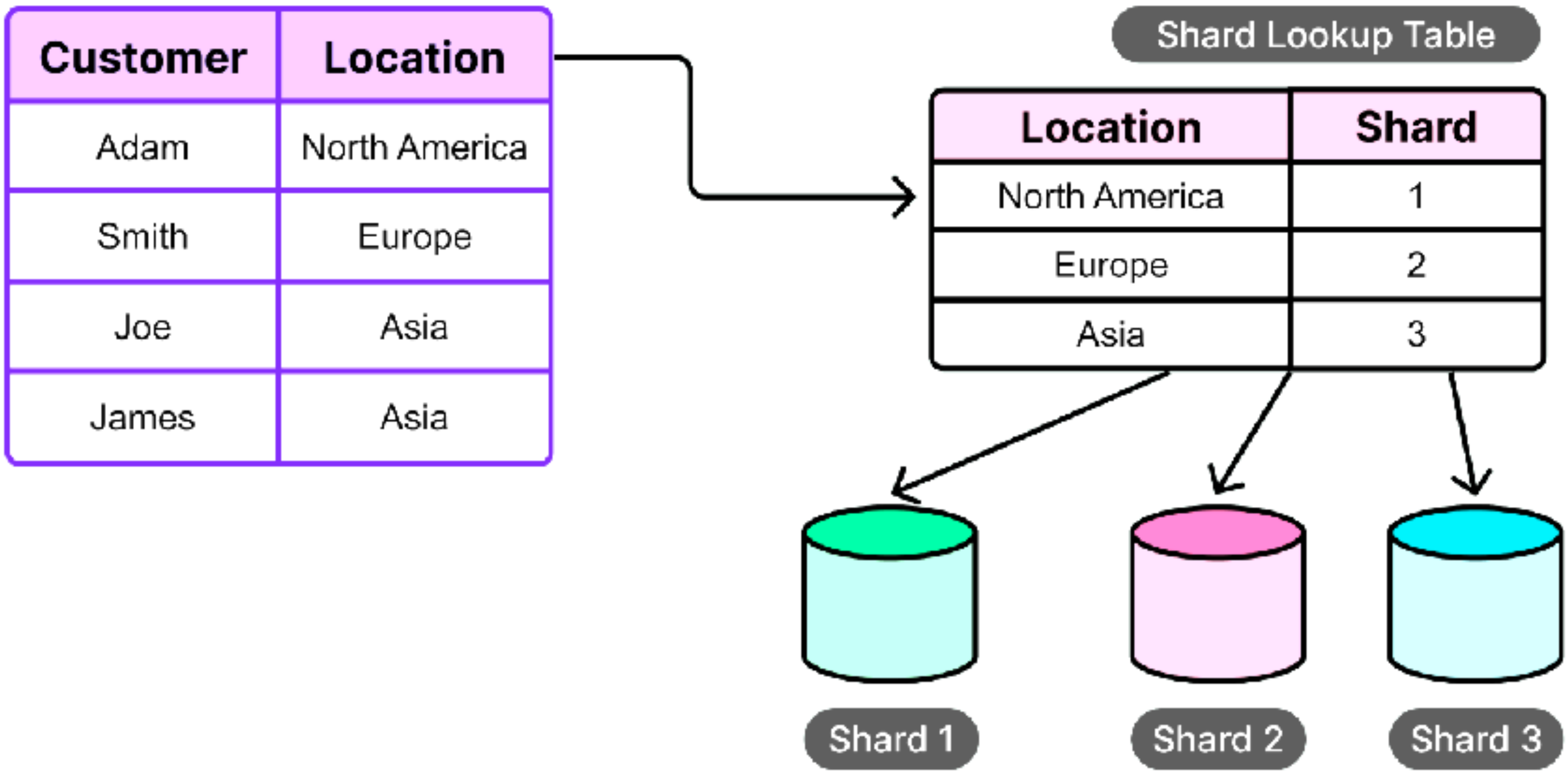
Customer	Location
Adam	North America
Smith	Europe
Joe	Asia
James	Asia

Shard Lookup Table

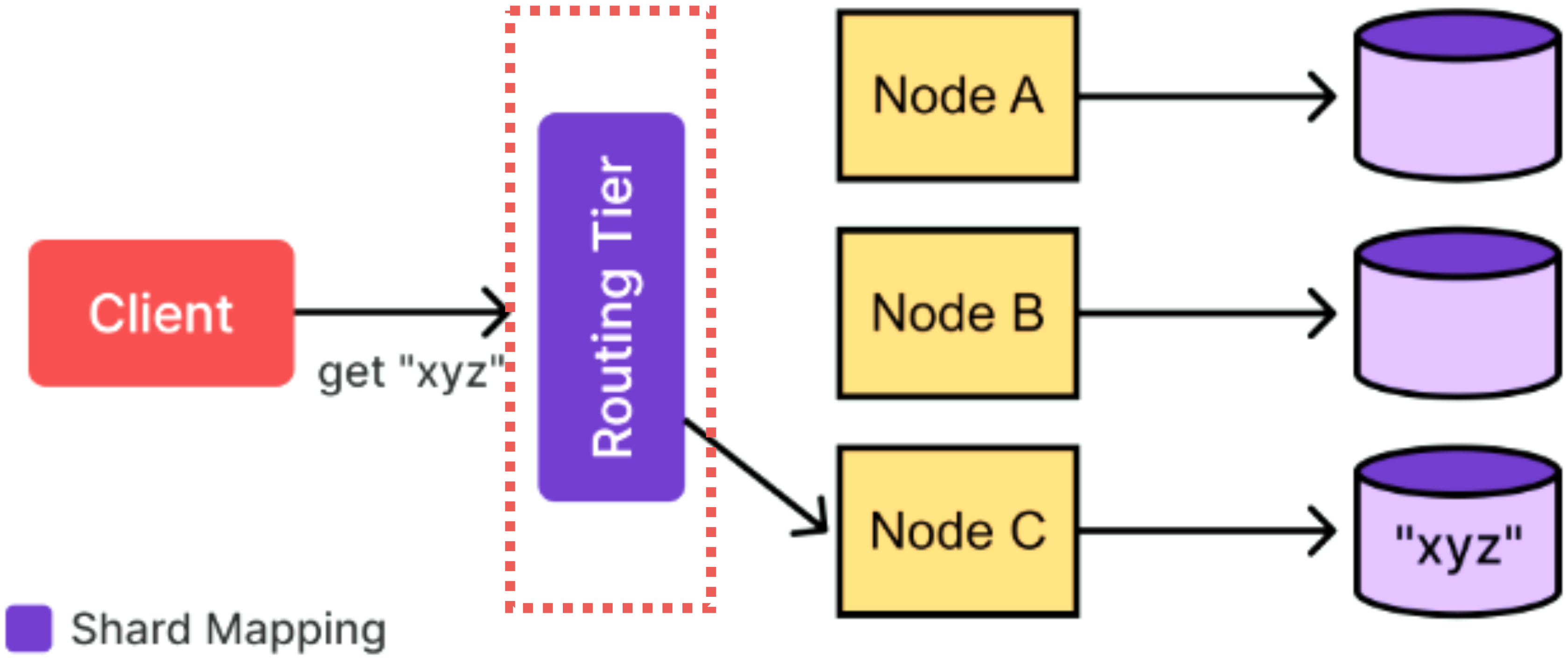
Location	Shard
North America	1
Europe	2
Asia	3



Directory-based sharding



Request routing



Workshop with Sharding

https://github.com/up1/workshop-database/blob/main/workshop/basic-workshop/partition.md

Denormalization

Denormalization techniques

Redundant column/Pre-joining tables

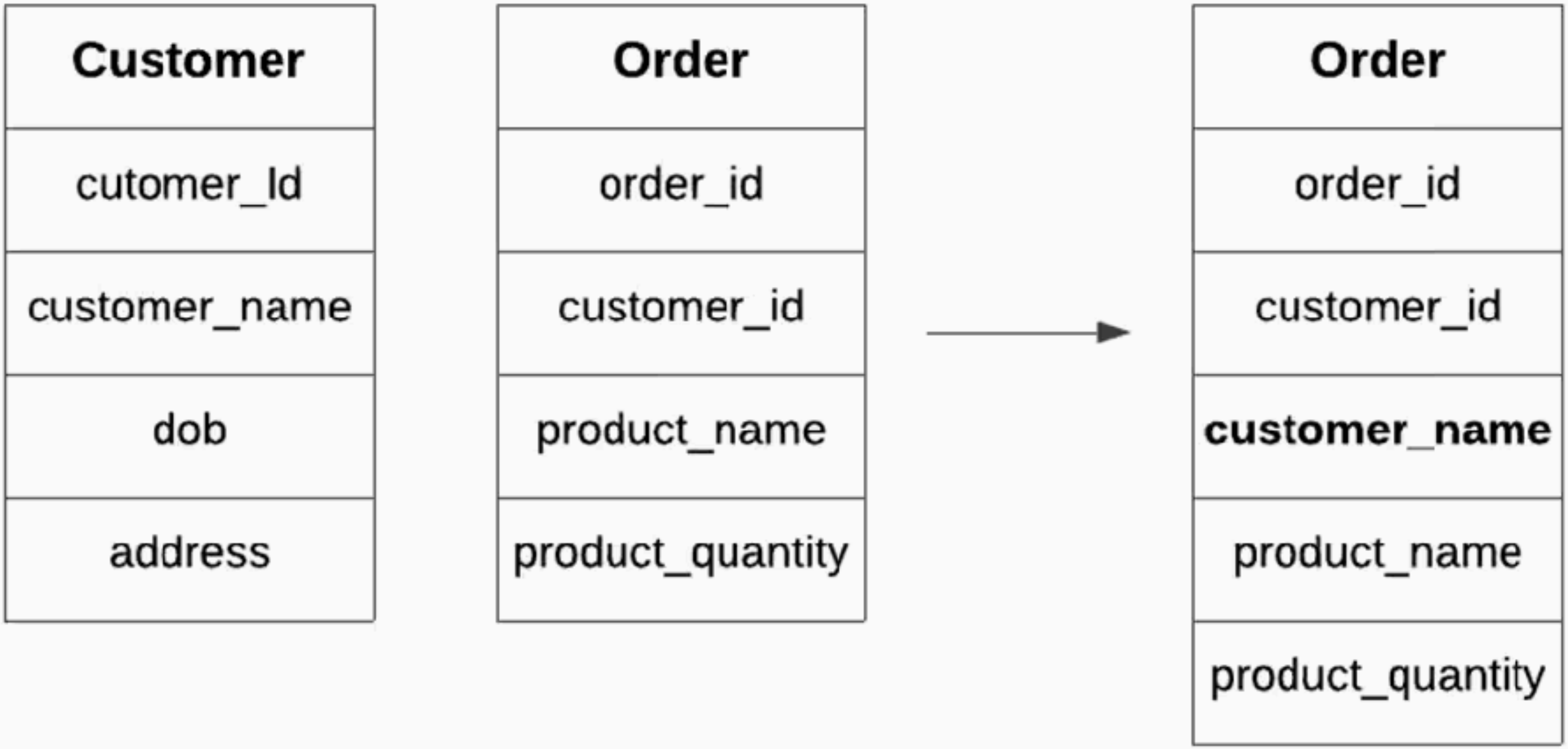
Table splitting (horizontal/vertical)

Add derived columns

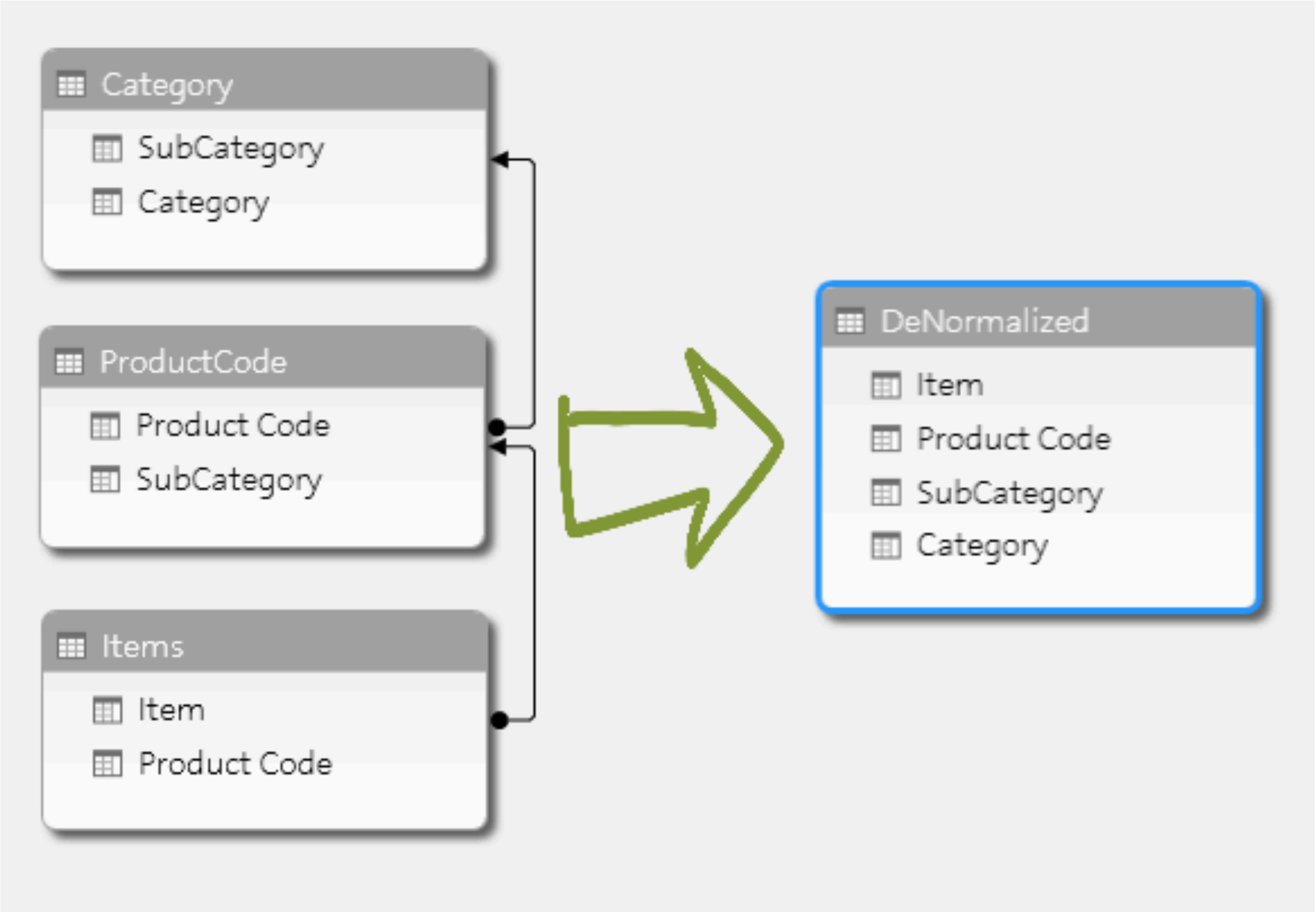
Mirrored tables

Materialized views

Redundant column/Pre-joining tables



Redundant column/Pre-joining tables



Horizontal splitting

Student_ID	Department_ID	Student_Name
1	1	Alex
2	2	Marie
3	1	Sam
4	3	Sara

Computer Science

Student_ID	Department_ID	Student_Name
1	1	Alex
3	1	Sam

Chemistry

Student_ID	Department_ID	Student_Name
2	2	Marie

Botany

Student_ID	Department_ID	Student_Name
4	3	Sara

Vertical splitting

Patient_ID	Patient_Name	Address	Medical_History

Patient_Details

Patient_ID	Patient_Name	Adress

Patient_Medical_History

Patient_ID	Medical_History

Add derived columns

Student_ID	Name
1	Alex
2	Marie

Student_ID	Assignment_ID	Mark
1	1	20
1	2	35.50
2	1	45
2	2	45

Student_ID	Name	Total_Marks
1	Alex	55.5
2	Marie	90

Pros of data denormalization

Improve query performance (UX)

Reduce complexity of data model

Improve application scalability

Generate data report faster

Cons of data denormalization

Increase data redundancy

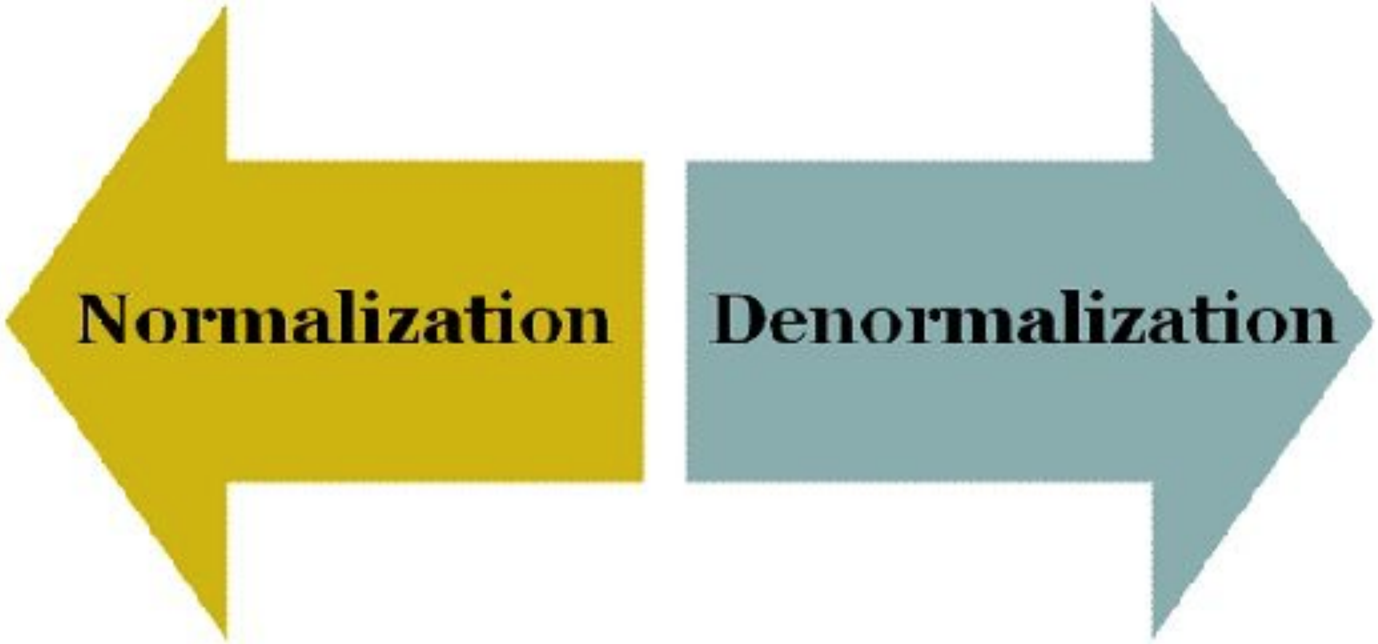
Inconsistency between data sets

Require more disk space (split tables)

More data model (hard to maintain)

Difficult to insert and update data

Maintenance costs


Normalization
Denormalization

Setup Postgres

Database Topologies

Standalone

Master-slave (scale-read)

Cluster and sharding

Essential configuration parameters

Max connections

Purpose

Maximum number of concurrent connections the server will accept

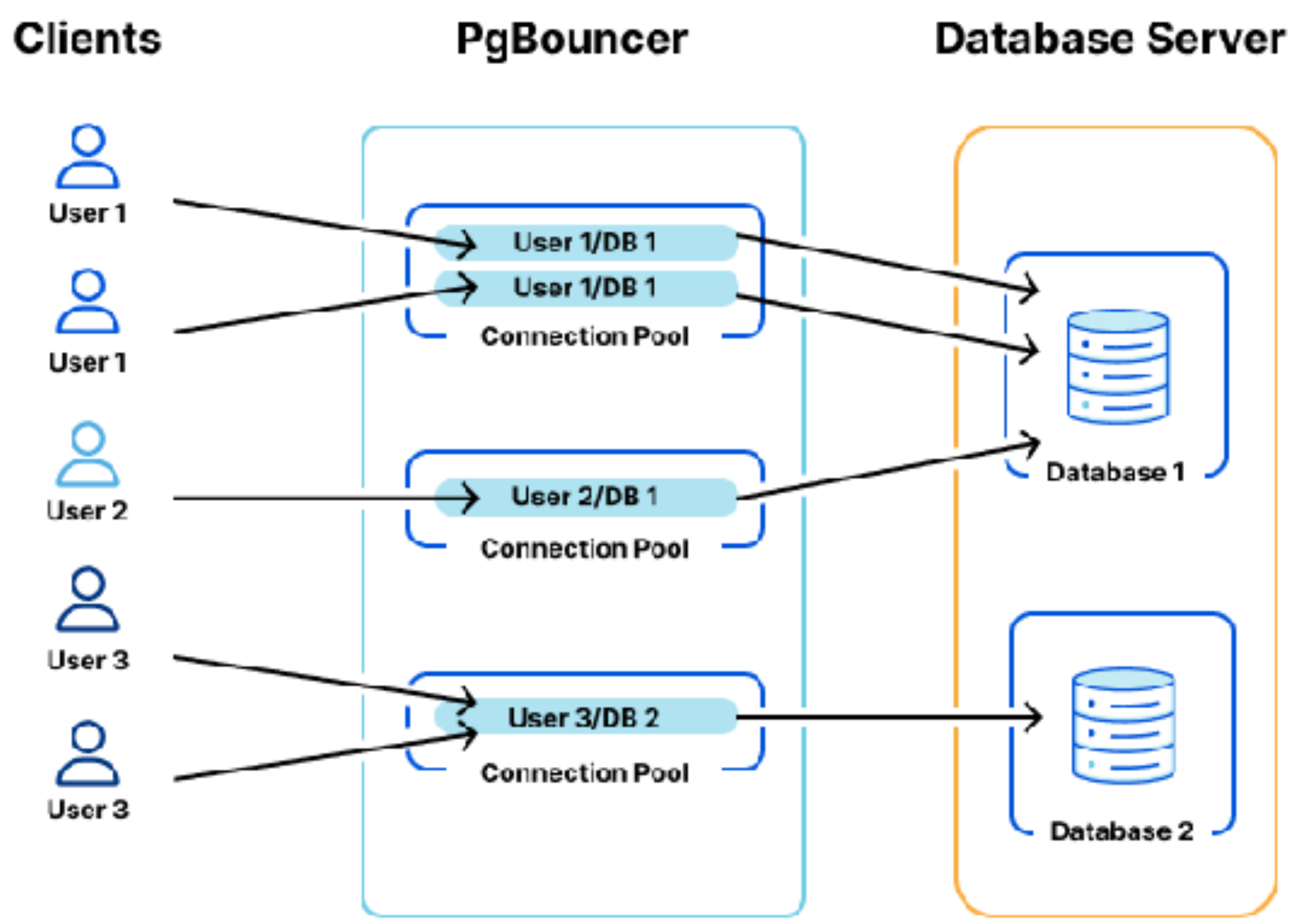
Impact

Too low: connection refused
Too high: excessive memory consumption, thrashing

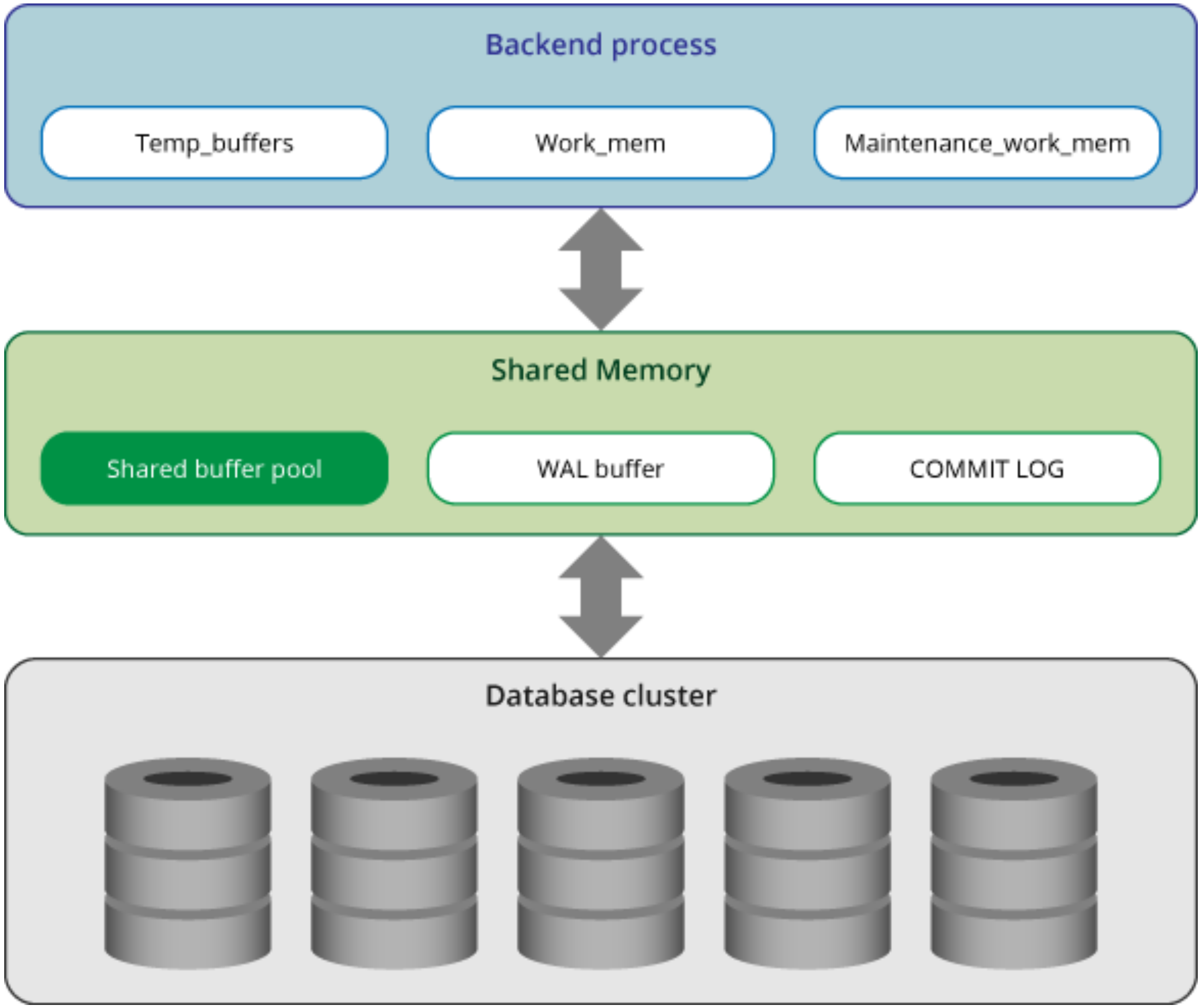
Recommendation

Start with a reasonable number based on application needs

Manage connection pooling ?



Memory architecture



Local memory

Shared memory

Local memory

Each backend process allocate local memory
Query processing

work_mem

temp_buffer

maintenance_work_mem

Shared memory

Allocate by server when it start up

shared_buffer

wal_buffer

Commit log (CLOG)

Work mem (not max memory)

Purpose

Memory used by internal sort operations and hash tables before spilling to disk

Impact

Crucial for query performance, especially with large sorts

Too low leads to temp file creation on disk

Recommendation

Start with a reasonable value (e.g., 4-64MB)

Tune based on EXPLAIN ANALYZE

Shared buffers

Purpose

Amount of memory dedicated to shared buffer pool for caching data pages

Impact

Directly affects read performance by reducing disk I/O

Recommendation

Typically 25% of total RAM, but can vary

Effective cache size

Purpose

Inform the query planner about the total amount of memory available for caching data (pg shared buffer + OS file cache)

Impact

Doesn’t allocate memory
Help the planner make better decision (index vs sequential)

Recommendation

50-75% og total RAM

Write-Ahead Log (WAL)

The foundation of **Durability**

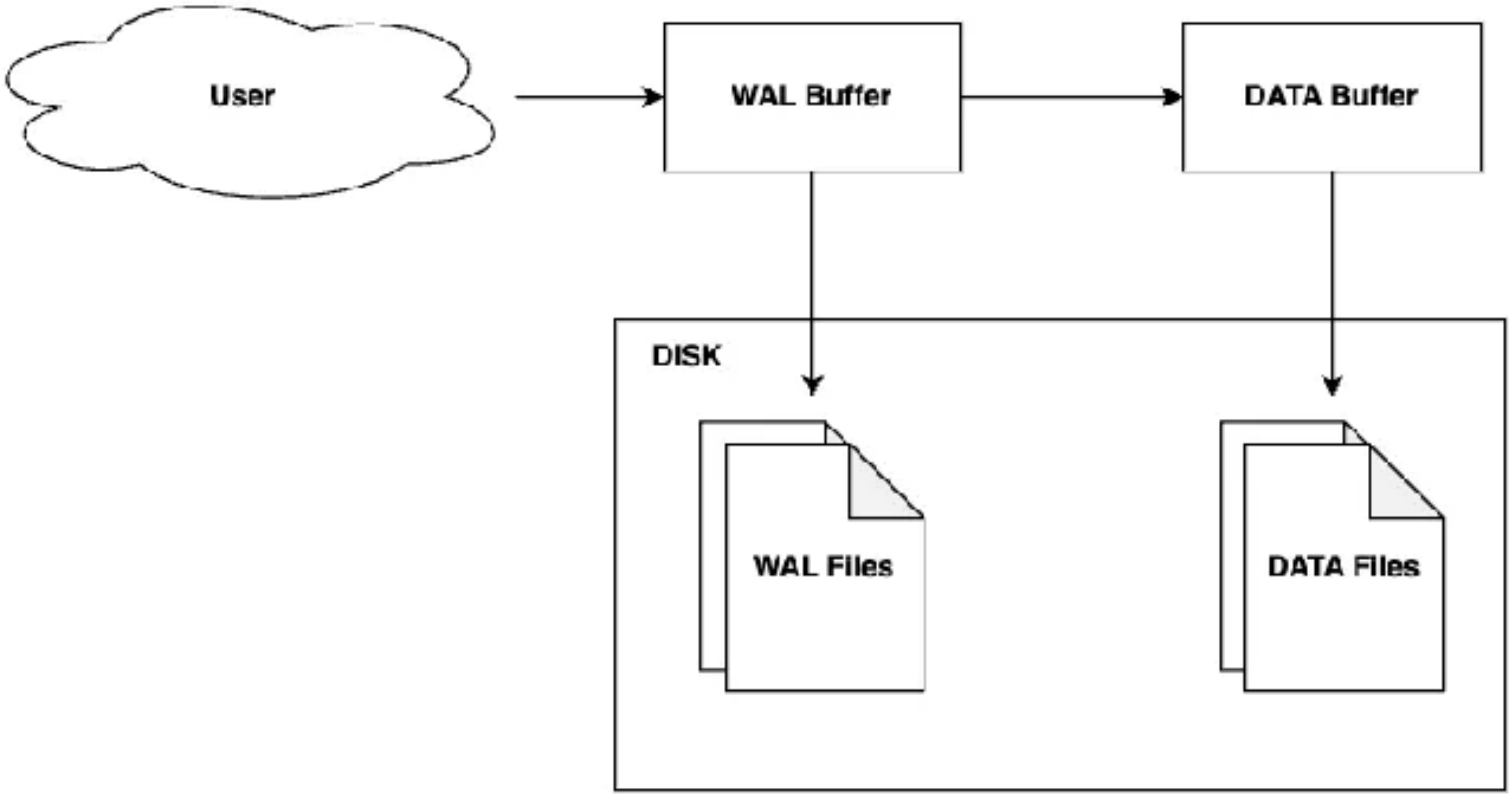
Write-Ahead Log (WAL) ?

Transaction log or XLOG

Use to ensure data durability and consistency

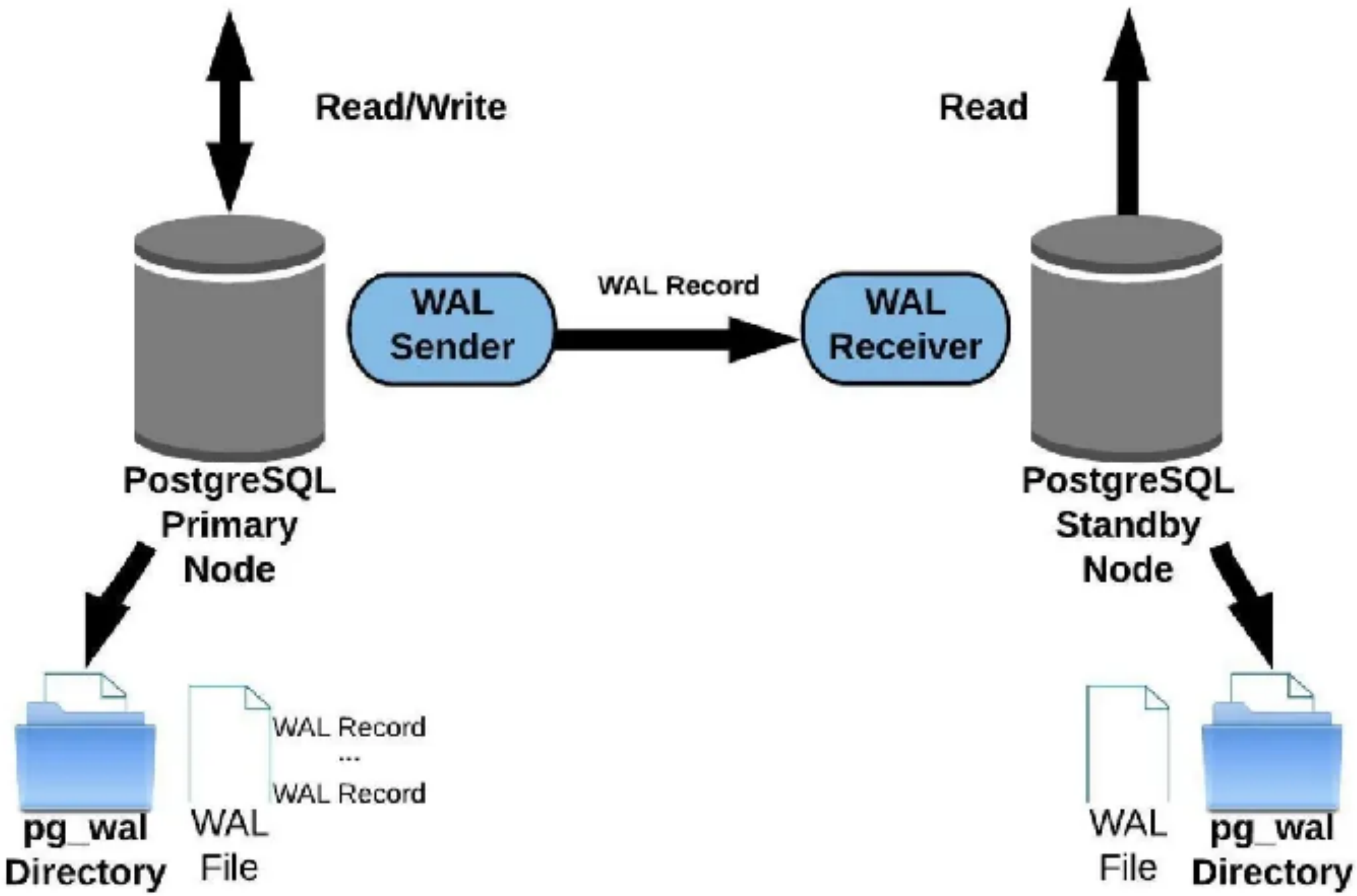
All changes to data must be written to the WAL before applied to the actual data files on disk

Write-Ahead Log (WAL) ?



https://levelup.gitconnected.com/when-databases-crash-wal-saves-the-day-write-ahead-log-wal-710e31aa9cdd

Async Replication



https://levelup.gitconnected.com/when-databases-crash-wal-saves-the-day-write-ahead-log-wal-710e31aa9cdd

Checkpoint timeout

Purpose

Controls how often checkpoints occur
Checkpoints flush dirty shared buffers to disk

Impact

Too frequent: I/O spikes during checkpoints
Too infrequent: Longer recovery time, large pg_wal directory

Recommendation

5-10 minutes

Max wal size

Purpose

Controls how often checkpoints occur
Checkpoints flush dirty shared buffers to disk

Impact

The maximum size of the WAL directory that triggers a checkpoint

Recommendation

Based on I/O capacity and recovery time tolerance
(e.g., 4GB-16GB+)

High volume tuning ?

Read heavy	Write heavy
Many concurrent user	Complex query

Principles for high volume tuning

Allocate more memory

Balance I/O (optimize WAL and checkpoint)

Parallelism

Monitoring (pg_stat_statement, EXPLAIN)

Hardware (Fast disk, RAM, CPU)

Connection pooling

Slow Query !!

Autovacuum

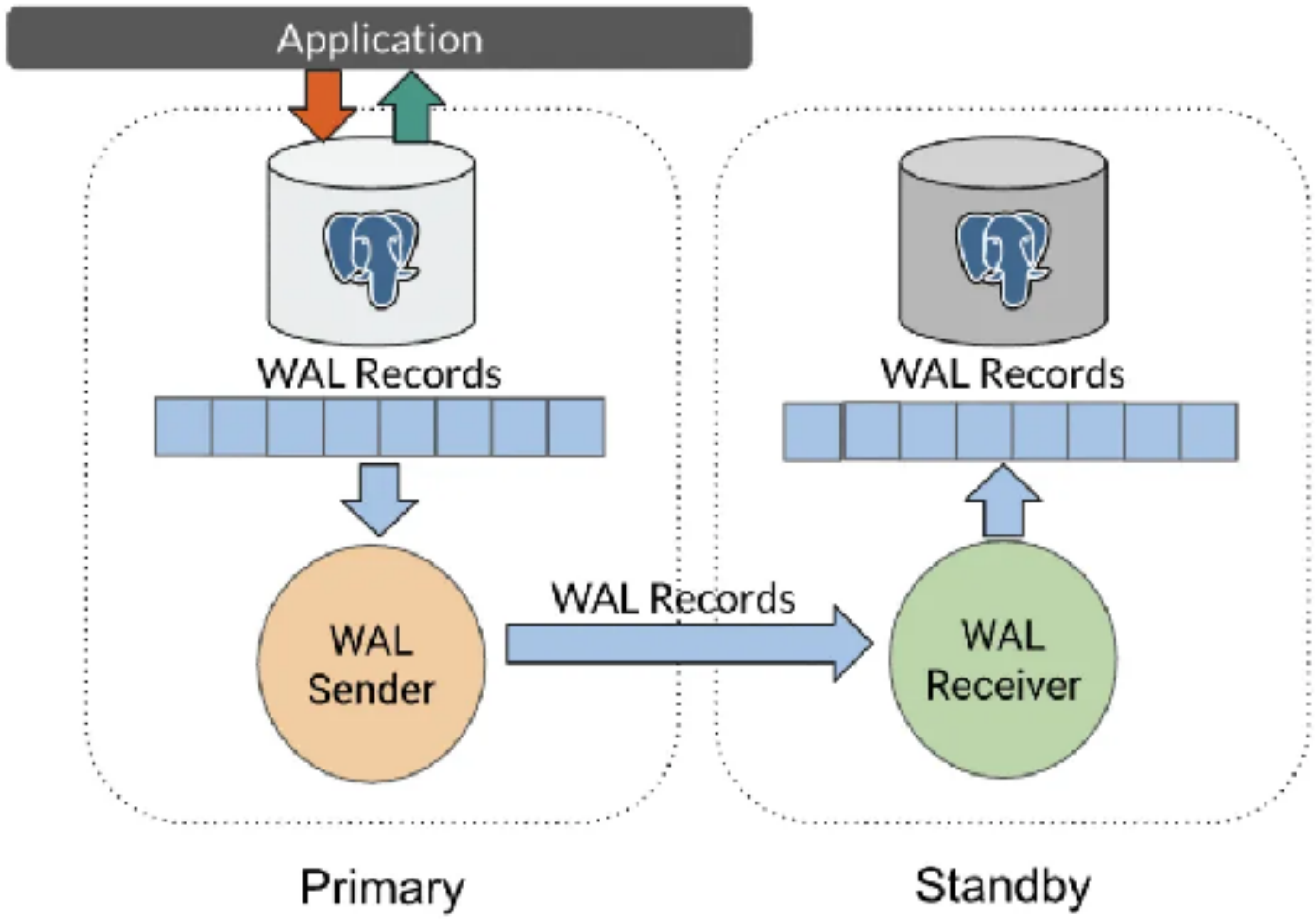
Prevent table bloat and ensure query performance

PostgreSQL Cluster

PostgreSQL
Stream Replication

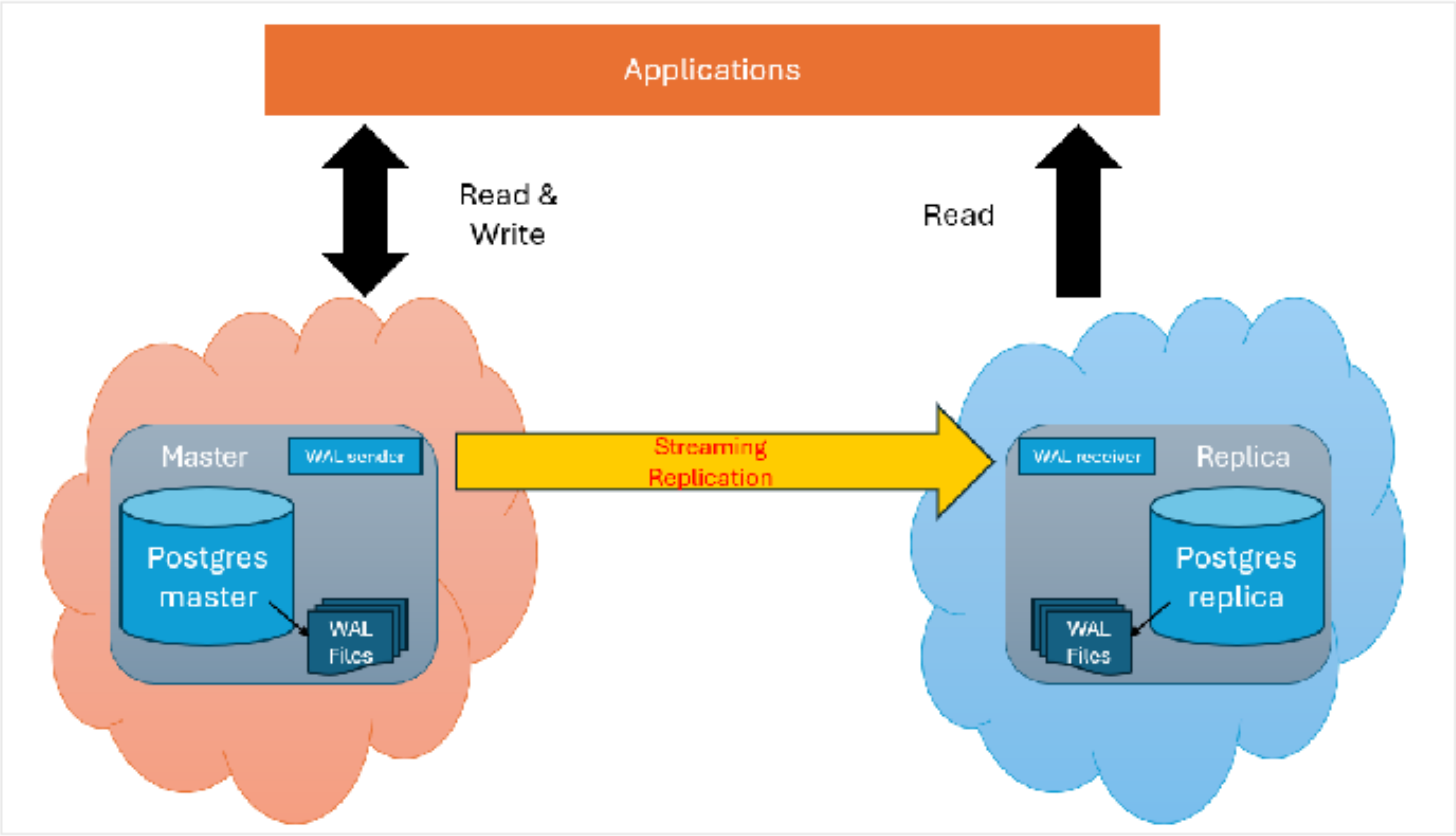
Stream Replication

Build-in sync or async replication
Simple DR site and backup cluster



Stream Replication

PostgreSQL stream replication (build-in)



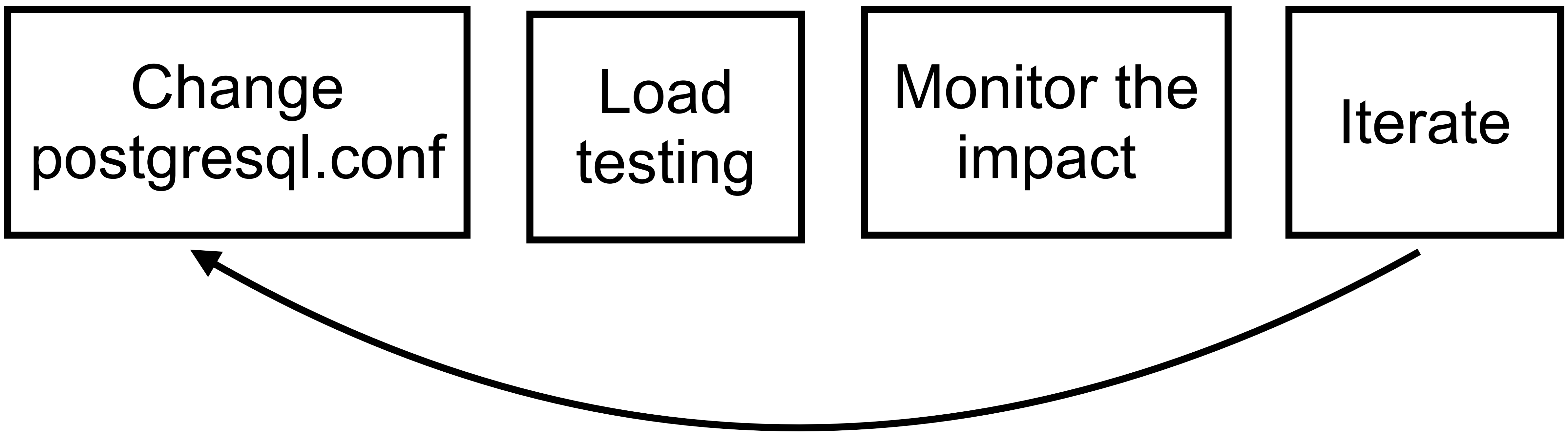
Stream Replication configuration

Memory allocation

Checkpoints

Query planning

Send WAL data from primary to standby server



Tuning parameters in primary

Name	Location	Description	Example value
wal_level	Primary	Sets the level of information written to the WAL	replica or logical
max_wal_senders	Primary	Maximum number of concurrent replication connections from standby servers	10
wal_keep_size	Primary	Minimum amount of WAL to retain in the pg_wal directory for standbys to fetch	4GB
max_slot_wal_keep_size	Primary	Maximum size of WAL files that replication slots are allowed to retain in pg_wal	32GB

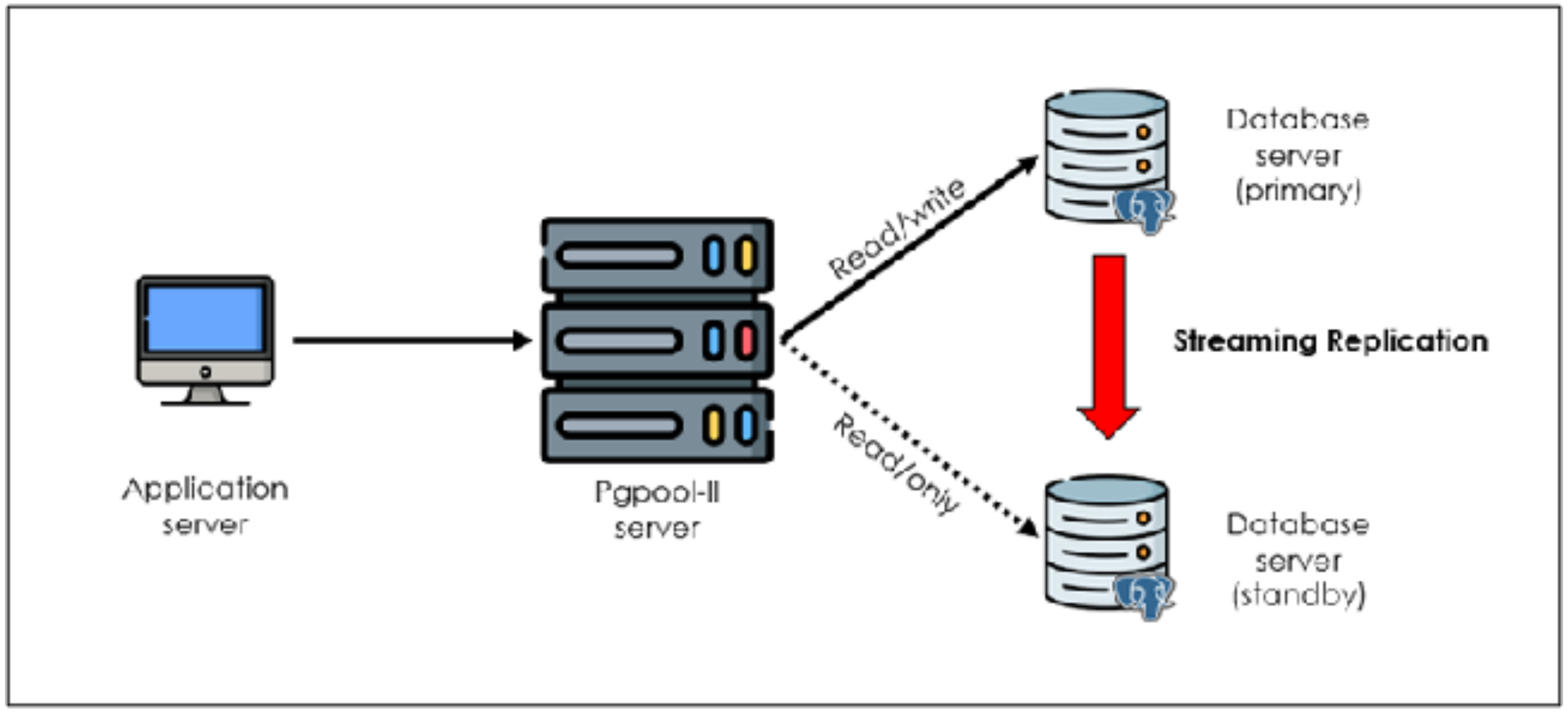
Tuning parameters in standby

Name	Location	Description	Example value
hot_standby	Standby	Allows read-only queries on the standby server while it's in recovery mode	on
max_standby_streaming_delay	Standby	Maximum time to wait before canceling standby queries that conflict with incoming WAL entries	30s
hot_standby_feedback	Standby	Sends feedback to the primary about the oldest transaction active on the standby	on
wal_receiver_Status_interval	Standby	Specifies the interval for the standby to report its replication status to the primary	10s

More ..

Scale with PgPool II

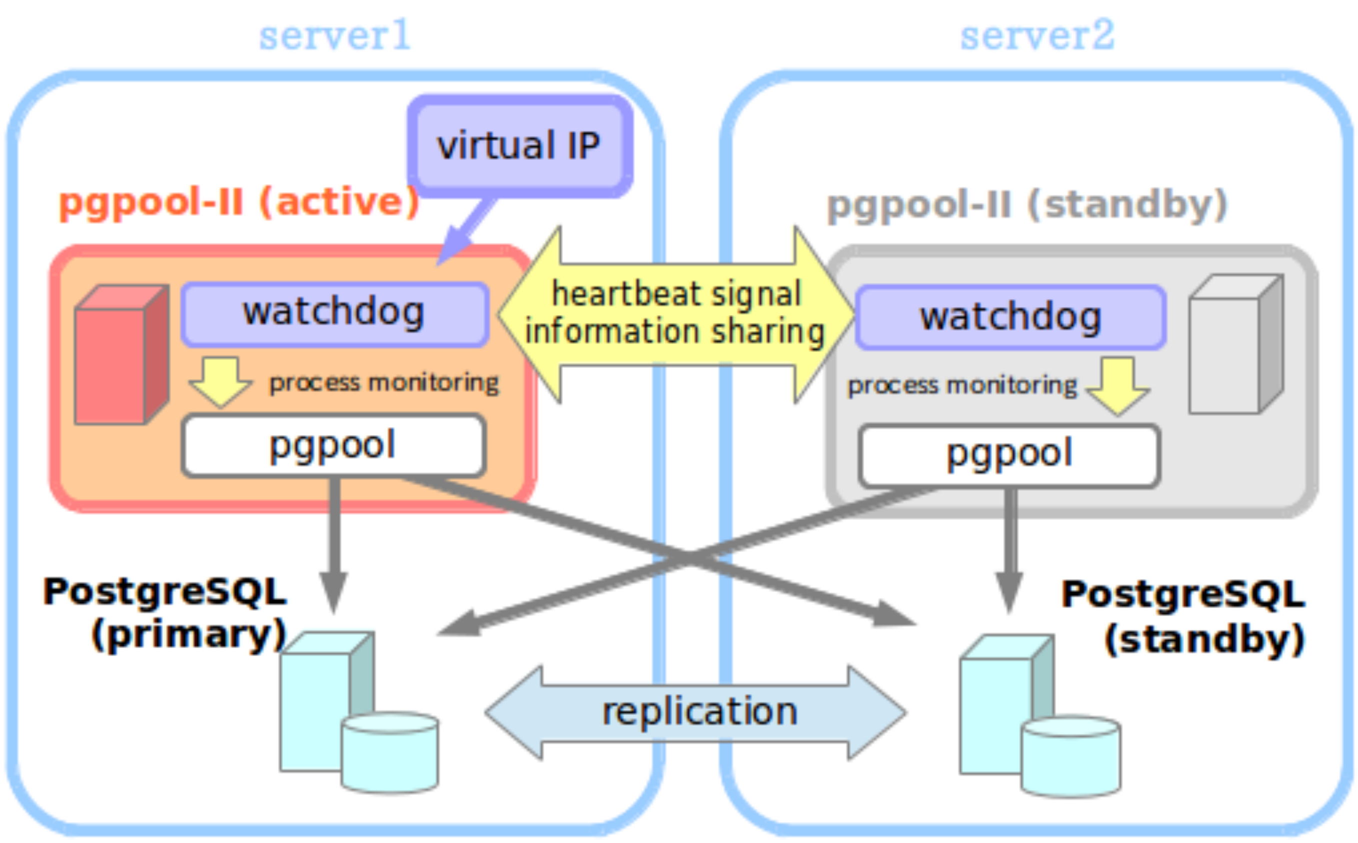
Connection pooling
Load balancing
In-memory query caching



https://www.pgpool.net/mediawiki/index.php/Main_Page

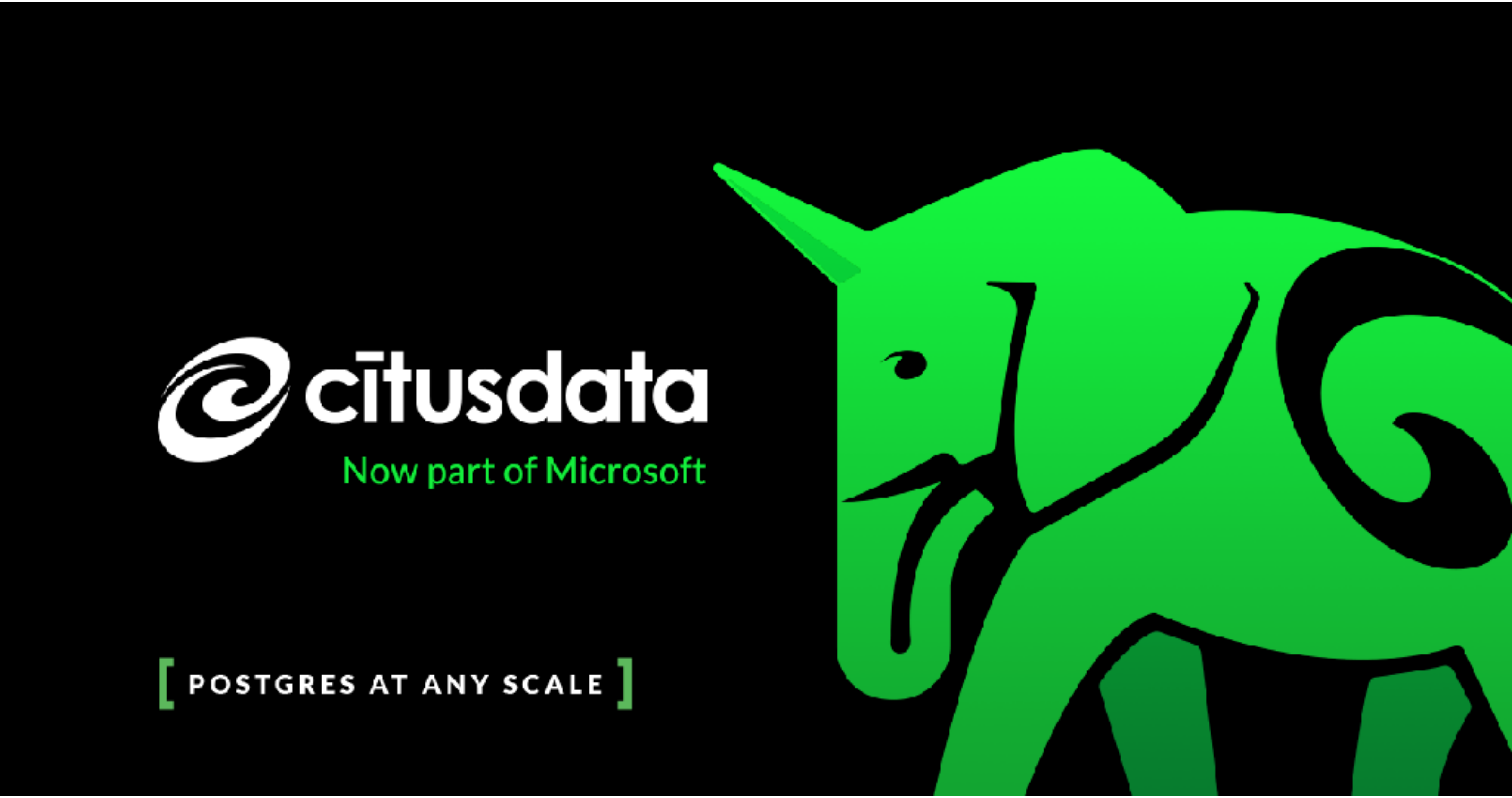
Scale with PgPool II

Master-slave mode with watchdog
Failure management



https://www.pgpool.net/mediawiki/index.php/Main_Page

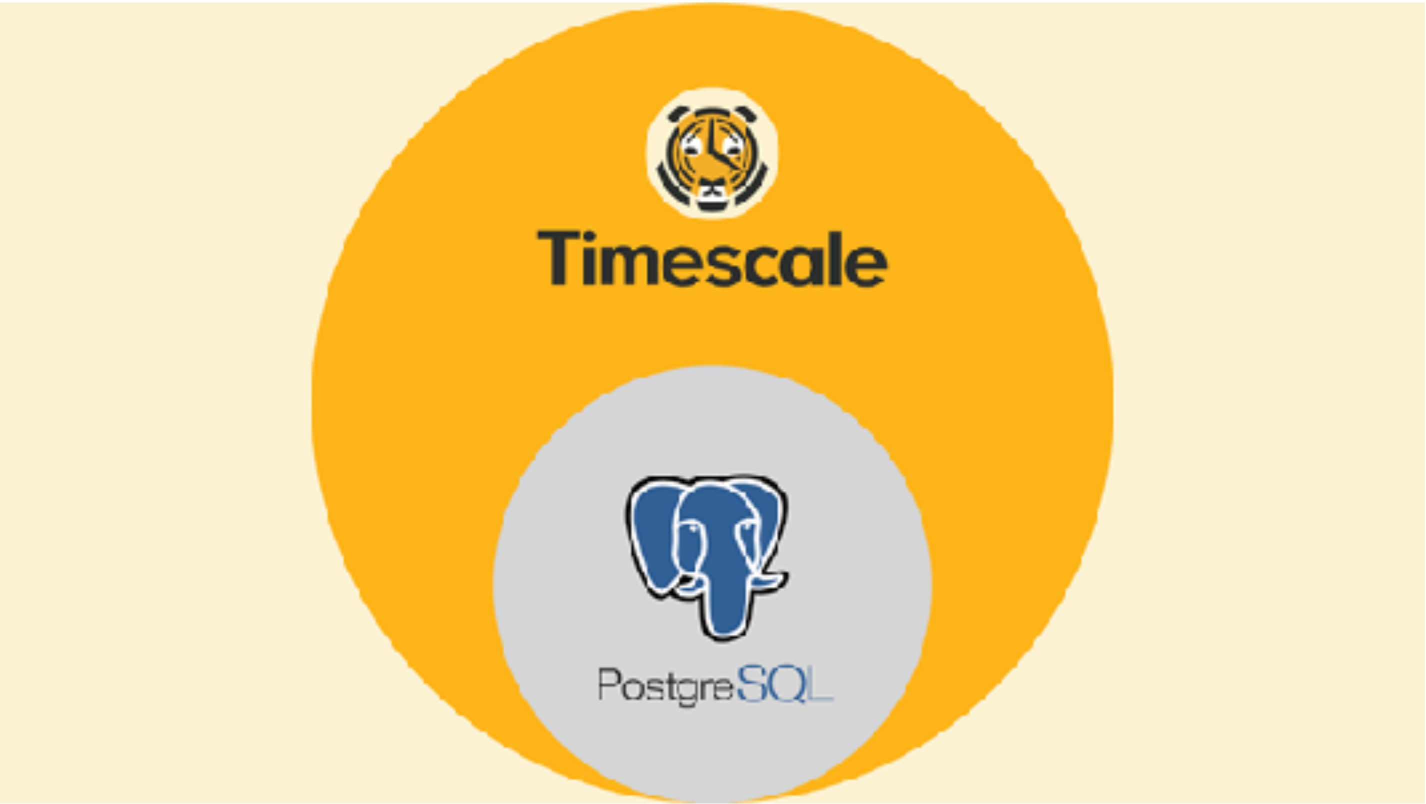
Citus = Distributed PostgreSQL



https://github.com/citusdata/citus

TimeScaleDB

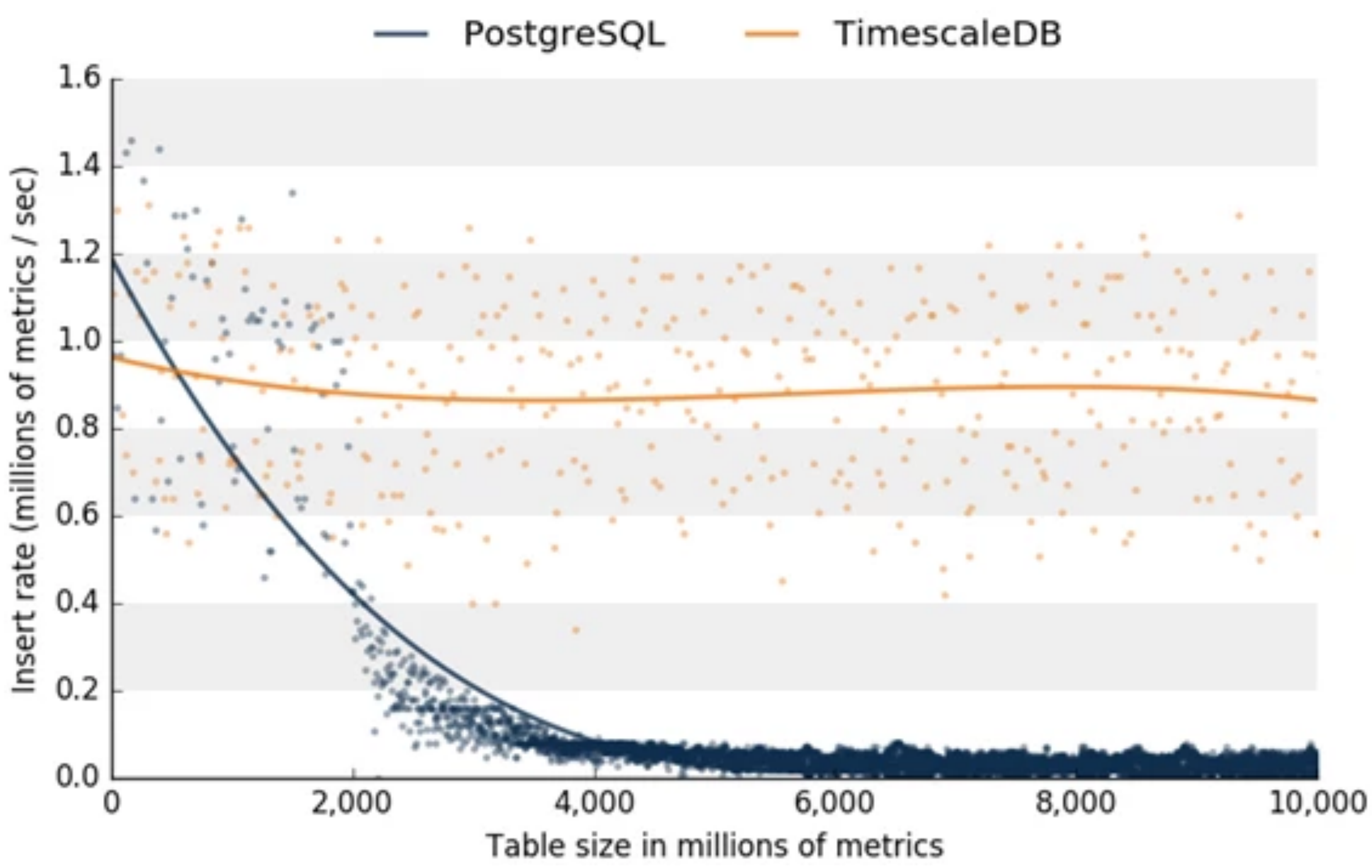
Time-series database for realtime analytic
PostgreSQL extension



https://github.com/timescale/timescaledb

TimeScaleDB

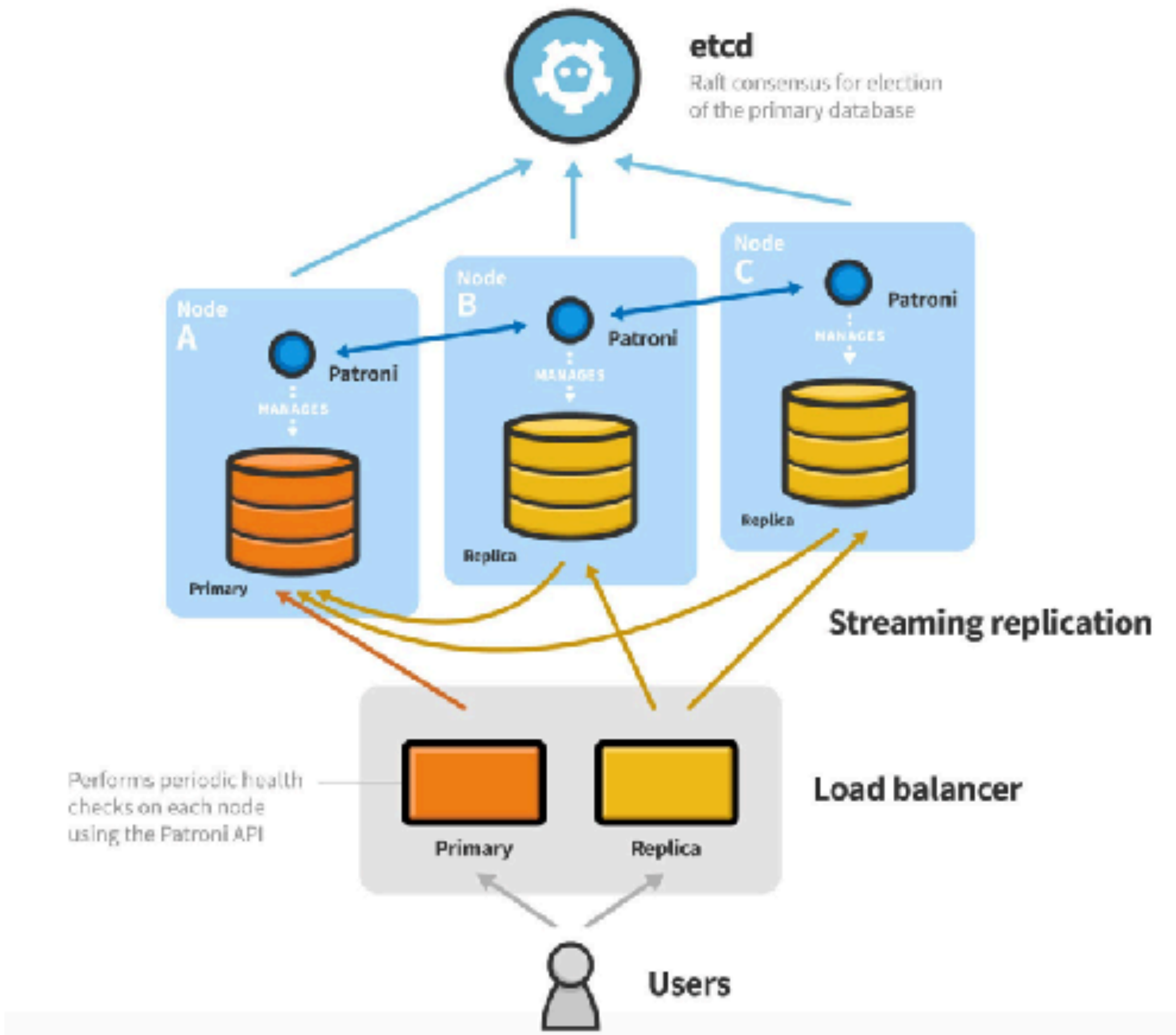
Insert operation



https://azure.microsoft.com/en-us/blog/power-iot-and-time-series-workloads-with-timescaledb-for-azure-database-for-postgresql/

Paroni (HA for PostgreSQL)

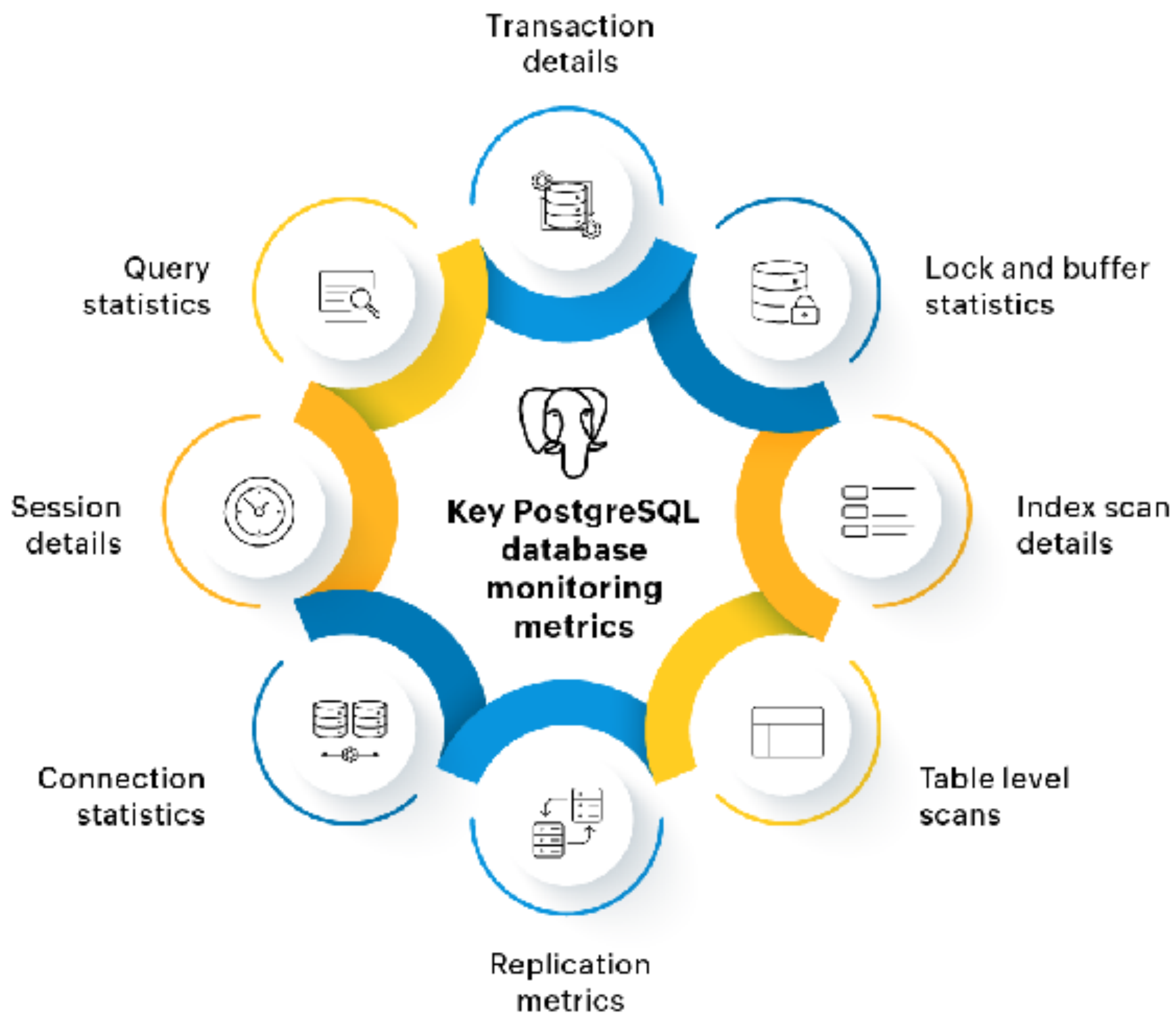
With Etcd, Consul and Zookeeper



https://github.com/patroni/patroni

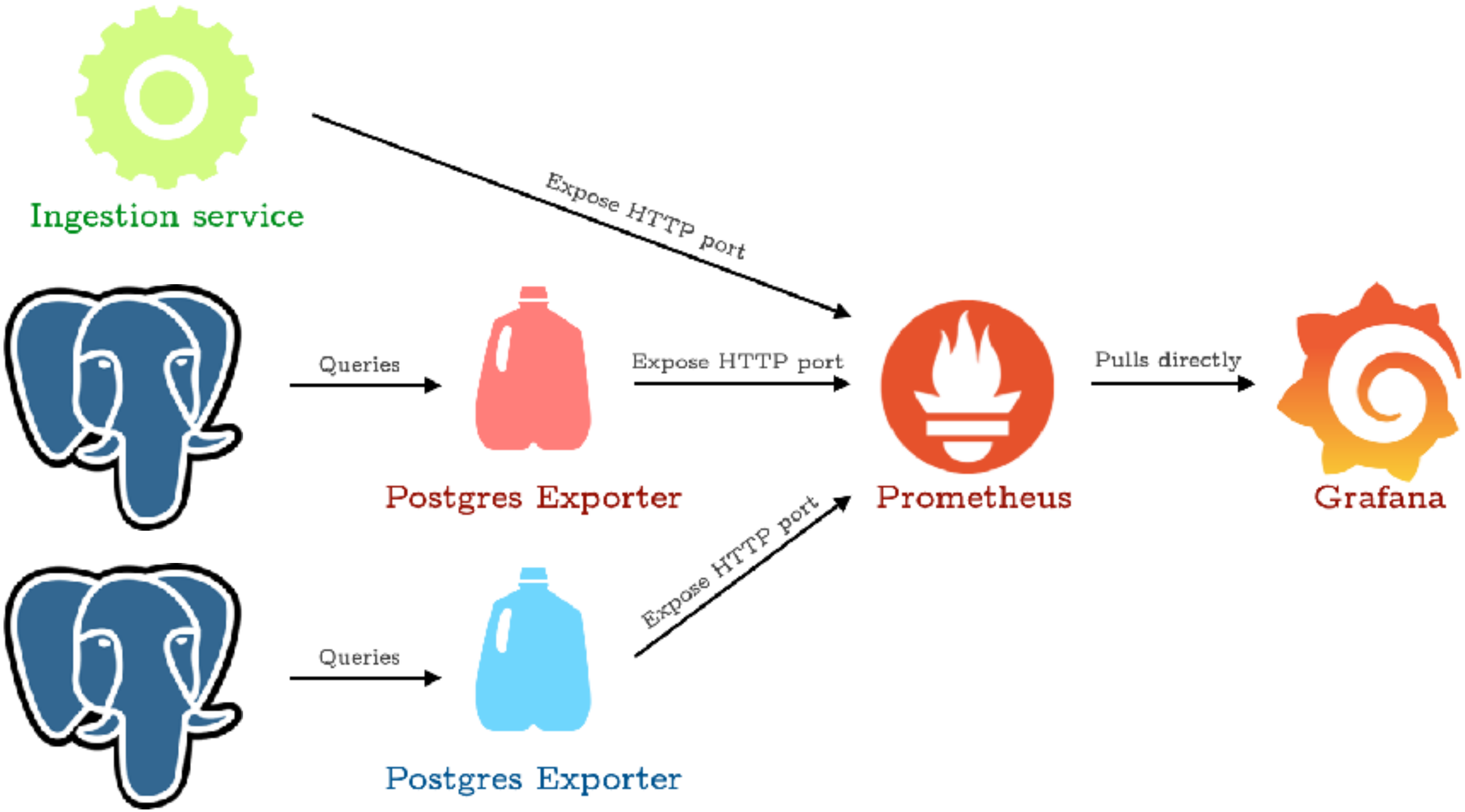
Monitoring

Basic Monitoring



https://blogs.manageengine.com/application-performance-2/appmanager/2024/04/19/postgres-performance-monitoring-best-practices-and-key-metrics.html

Basic Monitoring



https://github.com/up1/workshop-database/tree/main/workshop/monitoring-testing